Context

Create a Recommender System to show personalized movie recommendations based on ratings given by a user and other users similar to them in order to improve user experience.

Know Your Data

In [2]:
```
%pip install pandas
%pip install numpy

# Import Libraries
import numpy as np
import pandas as pd
from datetime import datetime
import warnings
warnings.filterwarnings('ignore')
```

```
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-package
s (2.2.2)
Requirement already satisfied: numpy>=1.26.0 in /usr/local/lib/python3.12/dist-
packages (from pandas) (1.26.4)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python
3.12/dist-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-p
ackages (from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dis
t-packages (from pandas) (2025.3)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packa
ges (from python-dateutil>=2.8.2->pandas) (1.17.0)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages
(1.26.4)
```

In [3]:
```
movies = pd.read_fwf('zee-movies.dat',encoding='ISO-8859-1')
users = pd.read_fwf('zee-users.dat',encoding='ISO-8859-1')
ratings = pd.read_fwf('zee-ratings.dat',encoding='ISO-8859-1')
```

In [4]:
```
movies.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3883 entries, 0 to 3882
Data columns (total 3 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   Movie ID::Title::Genres  3883 non-null   object
 1   Unnamed: 1              100 non-null    object
 2   Unnamed: 2              51 non-null     object
dtypes: object(3)
memory usage: 91.1+ KB
```

In [5]:
```
movies.head()
```

Out[5]:

	Movie ID::Title::Genres	Unnamed: 1	Unnamed: 2
0	1::Toy Story (1995)::Animation|Children's|Comedy	NaN	NaN
1	2::Jumanji (1995)::Adventure|Children's|Fantasy	NaN	NaN
2	3::Grumpier Old Men (1995)::Comedy|Romance	NaN	NaN
3	4::Waiting to Exhale (1995)::Comedy|Drama	NaN	NaN
4	5::Father of the Bride Part II (1995)::Comedy	NaN	NaN

In [6]:
```
users.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6040 entries, 0 to 6039
Data columns (total 1 columns):
 #   Column                                      Non-Null Count  Dtype 
---  ------                                      --------------  ----- 
 0   UserID::Gender::Age::Occupation::Zip-code  6040 non-null   object
dtypes: object(1)
memory usage: 47.3+ KB
```

In [7]:
```
users.head()
```

Out[7]:

	UserID::Gender::Age::Occupation::Zip-code
0	1::F::1::10::48067
1	2::M::56::16::70072
2	3::M::25::15::55117
3	4::M::45::7::02460
4	5::M::25::20::55455

In [8]:
```
ratings.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000209 entries, 0 to 1000208
Data columns (total 1 columns):
 #   Column                              Non-Null Count    Dtype 
---  ------                              --------------    ----- 
 0   UserID::MovieID::Rating::Timestamp  1000209 non-null  object
dtypes: object(1)
memory usage: 7.6+ MB
```

In [9]:
```
ratings.head()
```

Out[9]:

	UserID::MovieID::Rating::Timestamp
0	1::1193::5::978300760
1	1::661::3::978302109
2	1::914::3::978301968
3	1::3408::4::978300275
4	1::2355::5::978824291

- There are 3 different data sets each for Users, Movies and Ratings
- Users data include information like UserID, Gender, Age, Occupation and Zip Code
- Movies data include information like MovieID, Title and Genre
- Ratings data include information like UserID, MovieID, Rating and Timestamp
- Datasets require data cleaning and structuring which will follow in next section

Data Cleaning / Feature Engg

Movies Dataframe

In [10]:
```
movies.drop(columns=['Unnamed: 1', 'Unnamed: 2'], axis=1, inplace=True)
delimiter = '::'
movies = movies['Movie ID::Title::Genres'].str.split(delimiter, expand=True)
```

In [11]:
```
movies.columns = ['MovieID', 'Title', 'Genres']
```

In [12]:
```
movies.head()
```

Out[12]:

	MovieID	Title	Genres
0	1	Toy Story (1995)	Animation|Children's|Comedy
1	2	Jumanji (1995)	Adventure|Children's|Fantasy
2	3	Grumpier Old Men (1995)	Comedy|Romance
3	4	Waiting to Exhale (1995)	Comedy|Drama
4	5	Father of the Bride Part II (1995)	Comedy

- Segregate Movie Name and Year of Release from Title column
- Segregate Genres for further granularity

In [13]:
```
# Extract Movie Name and Release Year from Title
movies['MovieName'] = movies['Title'].str.extract(r'^(.*)\s\((\d{4})\)$')[0]
movies['ReleaseYear'] = movies['Title'].str.extract(r'^(.*)\s\((\d{4})\)$')[1]

# Split Genres into separate rows
movies = movies.assign(Genre=movies['Genres'].str.split('|')).explode('Genre')

# Drop the original Title and Genres columns if not needed
movies = movies[['MovieID', 'Title', 'MovieName', 'ReleaseYear', 'Genre']]
```

In [14]:
```
movies.head()
```

Out[14]:

	MovieID	Title	MovieName	ReleaseYear	Genre
0	1	Toy Story (1995)	Toy Story	1995	Animation
0	1	Toy Story (1995)	Toy Story	1995	Children's
0	1	Toy Story (1995)	Toy Story	1995	Comedy
1	2	Jumanji (1995)	Jumanji	1995	Adventure
1	2	Jumanji (1995)	Jumanji	1995	Children's

In [15]:
```
movies.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 6366 entries, 0 to 3882
Data columns (total 5 columns):
 #   Column       Non-Null Count  Dtype 
---  ------       --------------  ----- 
 0   MovieID      6366 non-null   object
 1   Title        6366 non-null   object
 2   MovieName    6342 non-null   object
 3   ReleaseYear  6342 non-null   object
 4   Genre        6341 non-null   object
dtypes: object(5)
memory usage: 556.4+ KB
```

In [16]:
```
# Calculate value counts as percentages for all columns
value_counts_percent = {col: movies[col].value_counts(normalize=True) * 100 fo

# Display the results
for col, counts in value_counts_percent.items():
    print(f"Value counts for column: {col}")
    print(counts)
    print("-" * 50)
```

```
Value counts for column: MovieID
MovieID
673     0.078542
1566    0.078542
610     0.078542
2322    0.078542
2080    0.078542
          ...
2386    0.015708
820     0.015708
821     0.015708
2383    0.015708
2010    0.015708
Name: proportion, Length: 3883, dtype: float64
--------------------------------------------------
Value counts for column: Title
Title
Space Jam (1996)                                        0.078542
Hercules (1997)                                         0.078542
Heavy Metal (1981)                                      0.078542
Soldier (1998)                                          0.078542
Lady and the Tramp (1955)                               0.078542
                                                           ...
Jerry Springer: Ringmaster (1998)                       0.015708
Death in the Garden (Mort en ce jardin, La) (1956)      0.015708
Crude Oasis, The (1995)                                 0.015708
Police Academy 6: City Under Siege (1989)               0.015708
Metropolis (1926)                                       0.015708
Name: proportion, Length: 3883, dtype: float64
--------------------------------------------------
Value counts for column: MovieName
MovieName
Mummy, The                      0.110375
Jungle Book, The                0.110375
Mighty Joe Young                0.094607
That Darn Cat!                  0.094607
King Kong                       0.094607
                                   ...
Collectionneuse, La             0.015768
Little Voice                    0.015768
Kaspar Hauser                   0.015768
Jerry Springer: Ringmaster      0.015768
Child's Play 3                  0.015768
Name: proportion, Length: 3817, dtype: float64
--------------------------------------------------
Value counts for column: ReleaseYear
ReleaseYear
1998    8.924629
1995    8.514664
1996    8.498896
1997    8.498896
1999    6.748660
          ...
1923    0.047304
```

```
1928    0.047304
1922    0.031536
1920    0.031536
1921    0.015768
Name: proportion, Length: 81, dtype: float64
--------------------------------------------------
Value counts for column: Genre
Genre
Drama       24.854124
Comedy      18.672134
Action       7.885192
Thriller     7.632865
Romance      7.143984
              ...
Childr       0.015770
Music        0.015770
Wester       0.015770
Chil         0.015770
Horr         0.015770
Name: proportion, Length: 63, dtype: float64
--------------------------------------------------
```

Some Genre names are improper for ex. 'Childr' which must come under 'Children's' Genre. Need to identify all those and put them under proper Genres

In [17]:
```python
# Official genre list
official_genres = [
    "Action", "Adventure", "Animation", "Children's", "Comedy", "Crime",
    "Documentary", "Drama", "Fantasy", "Film-Noir", "Horror", "Musical",
    "Mystery", "Romance", "Sci-Fi", "Thriller", "War", "Western"
]
```

In [18]:
```python
# Create a mapping dictionary for known incorrect genres
genre_mapping = {
    "Comdy": "Comedy", "Come": "Comedy", "Comed": "Comedy", "Com": "Comedy",
    "Childr": "Children's", "Chil": "Children's", "Child": "Children's", "Chil
    "Wester": "Western", "We": "Western",
    "Horr": "Horror", "Horro": "Horror",
    "Music": "Musical",
    "Docu": "Documentary", "Documenta": "Documentary", "Document": "Documentar
    "Romanc": "Romance", "R": "Romance", "Ro": "Romance", "Roma": "Romance", "
    "Sci-F": "Sci-Fi", "Sci": "Sci-Fi", "Sci-": "Sci-Fi", "S": "Sci-Fi",
    "Thrill": "Thriller", "Th": "Thriller", "Thri": "Thriller", "Thrille": "Th
    "Acti": "Action",
    "Wa": "War",
    "Fantas": "Fantasy", "Fant": "Fantasy",
    "Animati": "Animation",
    "Adve": "Adventure", "Ad": "Adventure", "Adventr": "Adventure", "Adventu":
    "Cri": "Crime",
    "My": "Mystery",
    "Dram": "Drama", "Dr": "Drama"
    # Add more mappings as needed
}
```

In [19]:
```python
# Function to clean genre names
def clean_genre(genre):
    if genre in official_genres:
        return genre
    elif genre in genre_mapping:
        return genre_mapping[genre]
    else:
        return "Unknown"

# Replace improper genres with correct names or "Unknown"
movies['Genre'] = movies['Genre'].apply(clean_genre)
```

In [20]:
```python
# Identify improper genres
unique_genres = set(movies['Genre'].unique())
improper_genres = unique_genres - set(official_genres)

# Print improper genres
print("Improper Genres Found:", improper_genres)
```

```
Improper Genres Found: {'Unknown'}
```

In [21]:
```python
movies['Genre'].value_counts(normalize=True) * 100
```

Out[21]:

Genre	proportion
Drama	24.850770
Comedy	18.677348
Action	7.869934
Thriller	7.665724
Romance	7.257304
Horror	5.340873
Adventure	4.429783
Sci-Fi	4.162740
Children's	3.864279
Crime	3.298775
War	2.183475
Documentary	1.947848
Musical	1.775055
Mystery	1.649387
Animation	1.633679
Western	1.068175
Fantasy	0.973924
Film-Noir	0.691172
Unknown	0.659755

dtype: float64

Genre column is sorted with proper Genres and the ones which could not be identified as 'Unknown'

In [22]:
```
movies.head()
```

Out[22]:

	MovieID	Title	MovieName	ReleaseYear	Genre
0	1	Toy Story (1995)	Toy Story	1995	Animation
0	1	Toy Story (1995)	Toy Story	1995	Children's
0	1	Toy Story (1995)	Toy Story	1995	Comedy
1	2	Jumanji (1995)	Jumanji	1995	Adventure
1	2	Jumanji (1995)	Jumanji	1995	Children's

In [23]:
```
# Pivoting the DataFrame
movies_df = movies.pivot_table(
    index=['MovieID', 'MovieName', 'ReleaseYear'],  # Grouping keys
    columns='Genre',  # Genre becomes columns
    values='Title',  # Title values populate the table
    aggfunc=lambda x: x  # Use raw Title value (avoiding aggregation for uniqu
).reset_index()

# Flatten the multi-index column names (if any)
movies_df.columns.name = None
movies_df = movies_df.fillna(0)  # Replace NaN with 0

# Replace Title values with 1
for col in movies_df.columns[3:]:
    movies_df[col] = movies_df[col].apply(lambda x: 1 if x != 0 else 0)
```

In [24]:
```
movies_df.head()
```

Out[24]:

	MovieID	MovieName	ReleaseYear	Action	Adventure	Animation	Children's
0	1	Toy Story	1995	0	0	1	1
1	10	GoldenEye	1995	1	1	0	0
2	100	City Hall	1996	0	0	0	0
3	1000	Curdled	1996	0	0	0	0
4	1002	Ed's Next Move	1996	0	0	0	0

5 rows × 22 columns

Ratings Dataframe

In [25]:
```
ratings = ratings['UserID::MovieID::Rating::Timestamp'].str.split(delimiter, e
ratings.columns = ['UserID', 'MovieID', 'Rating', 'Timestamp']
```

In [26]:
```
ratings.head()
```

Out[26]:

	UserID	MovieID	Rating	Timestamp
0	1	1193	5	978300760
1	1	661	3	978302109
2	1	914	3	978301968
3	1	3408	4	978300275
4	1	2355	5	978824291

In [27]:
```
ratings.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000209 entries, 0 to 1000208
Data columns (total 4 columns):
 #   Column     Non-Null Count    Dtype 
---  ------     --------------    ----- 
 0   UserID     1000209 non-null  object
 1   MovieID    1000209 non-null  object
 2   Rating     1000209 non-null  object
 3   Timestamp  1000209 non-null  object
dtypes: object(4)
memory usage: 30.5+ MB
```

In [28]:
```
ratings_df=ratings.copy()
```

In [29]:
```
# Convert Rating to integer
ratings_df['Rating'] = ratings_df['Rating'].astype(int)

# Convert Timestamp to datetime format
ratings_df['Timestamp'] = pd.to_datetime(ratings_df['Timestamp'], unit='s')
```

In [30]:
```
# Extract year, month, day of week, and hour
ratings_df['Year'] = ratings_df['Timestamp'].dt.year
ratings_df['Month'] = ratings_df['Timestamp'].dt.month
ratings_df['DayOfWeek'] = ratings_df['Timestamp'].dt.day_name()
ratings_df['Hour'] = ratings_df['Timestamp'].dt.hour
```

In [31]:
```
ratings_df.head()
```

Out[31]:

	UserID	MovieID	Rating	Timestamp	Year	Month	DayOfWeek	Hour
0	1	1193	5	2000-12-31 22:12:40	2000	12	Sunday	22
1	1	661	3	2000-12-31 22:35:09	2000	12	Sunday	22
2	1	914	3	2000-12-31 22:32:48	2000	12	Sunday	22
3	1	3408	4	2000-12-31 22:04:35	2000	12	Sunday	22
4	1	2355	5	2001-01-06 23:38:11	2001	1	Saturday	23

Aggregate Ratings by User

- Average Rating per User: Identify users who consistently give high or low ratings.
- Total Ratings per User: Analyze active vs. inactive users.

In [32]:
```
user_stats = ratings_df.groupby('UserID')['Rating'].agg(['mean', 'count']).res
user_stats.rename(columns={'mean': 'AvgRatingPerUser', 'count': 'TotalRatingsP
```

In [33]:
```
user_stats.head()
```

Out[33]:

	UserID	AvgRatingPerUser	TotalRatingsPerUser
0	1	4.188679	53
1	10	4.114713	401
2	100	3.026316	76
3	1000	4.130952	84
4	1001	3.652520	377

Aggregate Ratings by Movie

- Average Rating per Movie: Identify the most liked or disliked movies.
- Total Ratings per Movie: Find the most rated movies.

In [34]:
```
movie_stats = ratings_df.groupby('MovieID')['Rating'].agg(['mean', 'count']).r
movie_stats.rename(columns={'mean': 'AvgRatingPerMovie', 'count': 'TotalRating
```

In [35]:
```
movie_stats.head()
```

Out[35]:

	MovieID	AvgRatingPerMovie	TotalRatingsPerMovie
0	1	4.146846	2077
1	10	3.540541	888
2	100	3.062500	128
3	1000	3.050000	20
4	1002	4.250000	8

Users Dataframe

In [36]:
```
users = users['UserID::Gender::Age::Occupation::Zip-code'].str.split(delimiter
users.columns = ['UserID', 'Gender', 'Age', 'Occupation', 'Zip-code']
```

In [37]:
```
users.head()
```

Out[37]:

	UserID	Gender	Age	Occupation	Zip-code
0	1	F	1	10	48067
1	2	M	56	16	70072
2	3	M	25	15	55117
3	4	M	45	7	02460
4	5	M	25	20	55455

In [38]:
```
# Calculate value counts as percentages for all columns
value_counts_percent = {col: users[col].value_counts(normalize=True) * 100 for

# Display the results
for col, counts in value_counts_percent.items():
    print(f"Value counts for column: {col}")
    print(counts)
    print("-" * 50)
```

```
Value counts for column: UserID
UserID
1       0.016556
4024    0.016556
4033    0.016556
4032    0.016556
4031    0.016556
          ...
2012    0.016556
2011    0.016556
2010    0.016556
2009    0.016556
6040    0.016556
Name: proportion, Length: 6040, dtype: float64
--------------------------------------------------
Value counts for column: Gender
Gender
M    71.705298
F    28.294702
Name: proportion, dtype: float64
--------------------------------------------------
Value counts for column: Age
Age
25    34.701987
35    19.751656
18    18.261589
45     9.105960
50     8.211921
56     6.291391
1      3.675497
Name: proportion, dtype: float64
--------------------------------------------------
Value counts for column: Occupation
Occupation
4     12.566225
0     11.771523
7     11.241722
1      8.741722
17     8.311258
12     6.423841
14     5.000000
20     4.652318
2      4.420530
16     3.990066
6      3.907285
10     3.228477
3      2.864238
15     2.384106
13     2.350993
11     2.135762
5      1.854305
9      1.523179
19     1.192053
18     1.158940
```

```
8      0.281457
Name: proportion, dtype: float64
--------------------------------------------------
Value counts for column: Zip-code
Zip-code
48104    0.314570
22903    0.298013
55104    0.281457
94110    0.281457
55455    0.264901
           ...
80236    0.016556
19428    0.016556
33073    0.016556
99005    0.016556
14706    0.016556
Name: proportion, Length: 3439, dtype: float64
--------------------------------------------------
```

In [39]:
```
users_df=users.copy()
```

In [40]:
```
users_df.replace({'Age':{'1':  "Under 18",
                         '18':  "18-24",
                         '25':  "25-34",
                         '35':  "35-44",
                         '45':  "45-49",
                         '50':  "50-55",
                         '56':  "56 Above"}}, inplace=True)
```

In [41]:
```
users_df.replace({'Occupation':{'0': "other",
                                '1': "academic/educator",
                                '2': "artist",
                                '3': "clerical/admin",
                                '4': "college/grad student",
                                '5': "customer service",
                                '6': "doctor/health care",
                                '7': "executive/managerial",
                                '8': "farmer",
                                '9': "homemaker",
                                '10': "k-12 student",
                                '11': "lawyer",
                                '12': "programmer",
                                '13': "retired",
                                '14': "sales/marketing",
                                '15': "scientist",
                                '16': "self-employed",
                                '17': "technician/engineer",
                                '18': "tradesman/craftsman",
                                '19': "unemployed",
                                '20': "writer"}}, inplace=True)
```

In [42]:
```
users_df.head()
```

Out[42]:

	UserID	Gender	Age	Occupation	Zip-code
0	1	F	Under 18	k-12 student	48067
1	2	M	56 Above	self-employed	70072
2	3	M	25-34	scientist	55117
3	4	M	45-49	executive/managerial	02460
4	5	M	25-34	writer	55455

Merging all the 3 dataframes

In [43]:
```
# Merge ratings_df with movies_df on MovieID
ratings_movies = pd.merge(ratings_df, movies_df, on='MovieID', how='inner')

# Merge the resulting dataframe with users_df on UserID
merged_df = pd.merge(ratings_movies, users_df, on='UserID', how='inner')
```

In [44]:
```
merged_df.head()
```

Out[44]:

	UserID	MovieID	Rating	Timestamp	Year	Month	DayOfWeek	Hour	Movie
0	1	1193	5	2000-12-31 22:12:40	2000	12	Sunday	22	On Ov Cu
1	1	661	3	2000-12-31 22:35:09	2000	12	Sunday	22	Jam the
2	1	914	3	2000-12-31 22:32:48	2000	12	Sunday	22	My Fai
3	1	3408	4	2000-12-31 22:04:35	2000	12	Sunday	22	Broc
4	1	2355	5	2001-01-06 23:38:11	2001	1	Saturday	23	Bug's

5 rows × 33 columns

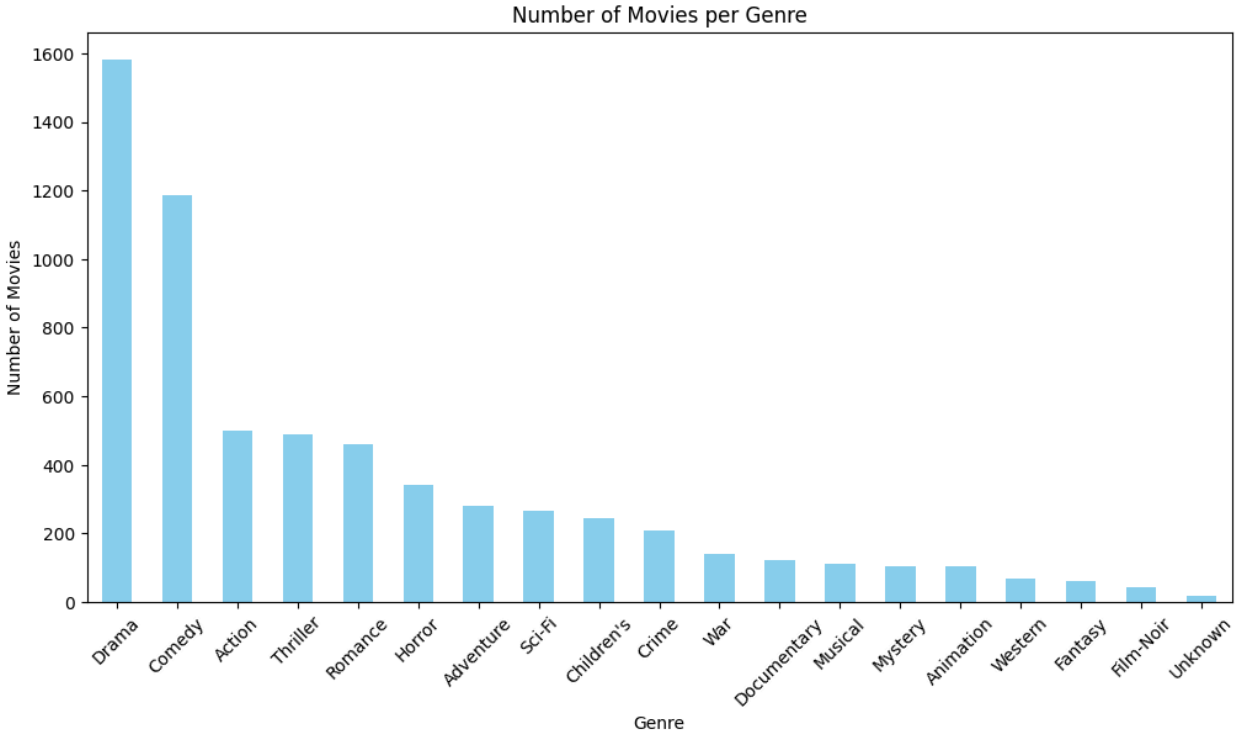
Exploratory Data Analysis

Movies

In [45]:
```
import seaborn as sns
import matplotlib.pyplot as plt
```

Dataset Composition

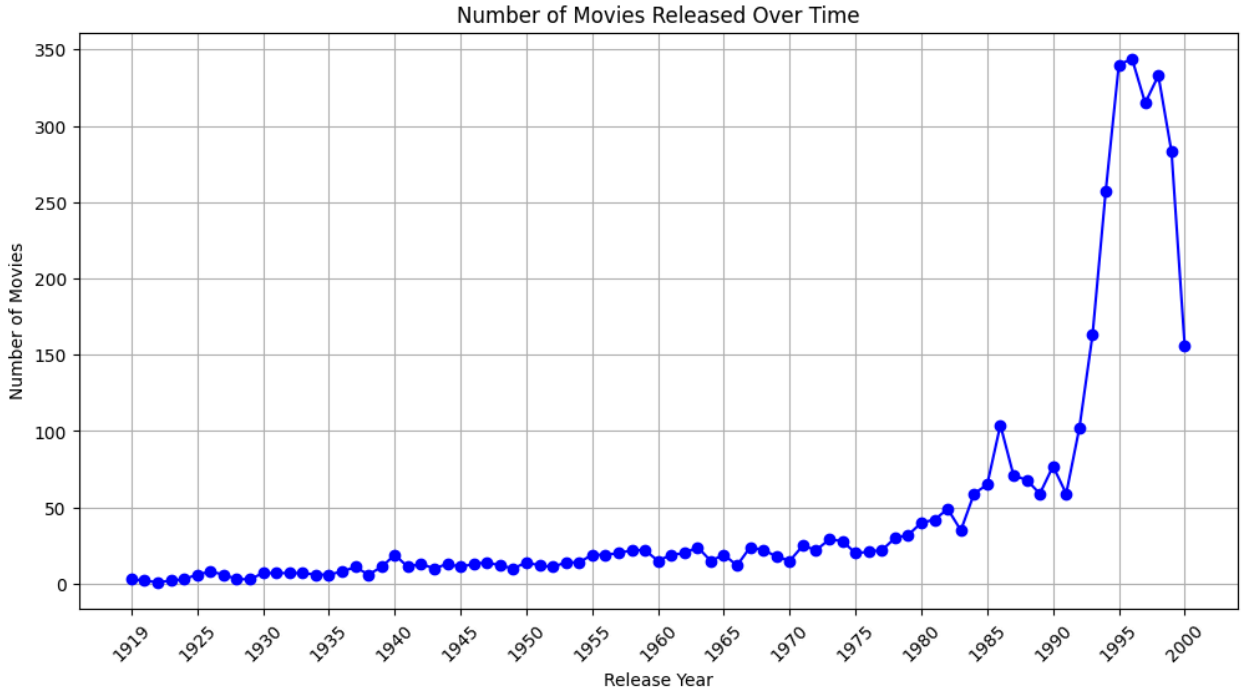
In [46]:
```
# Sum of movies in each genre
genre_counts = movies_df.iloc[:, 3:].sum().sort_values(ascending=False)

# Bar plot
plt.figure(figsize=(12, 6))
genre_counts.plot(kind='bar', color='skyblue')
plt.title('Number of Movies per Genre')
plt.xlabel('Genre')
plt.ylabel('Number of Movies')
plt.xticks(rotation=45)
plt.show()
```



- Highest number of movies nearly 1600 belong to Drama followed by Comedy and then Action

Year Wise Trend

In [47]:
```
# Count movies by ReleaseYear
year_counts = movies_df['ReleaseYear'].value_counts().sort_index()

# Line plot
plt.figure(figsize=(12, 6))
plt.plot(year_counts.index, year_counts.values, marker='o', color='b')
plt.title('Number of Movies Released Over Time')
plt.xlabel('Release Year')
plt.ylabel('Number of Movies')
plt.grid(True)

# Adjust x-axis ticks (display every 5th year)
plt.xticks(ticks=year_counts.index[::5], rotation=45)

plt.show()
```



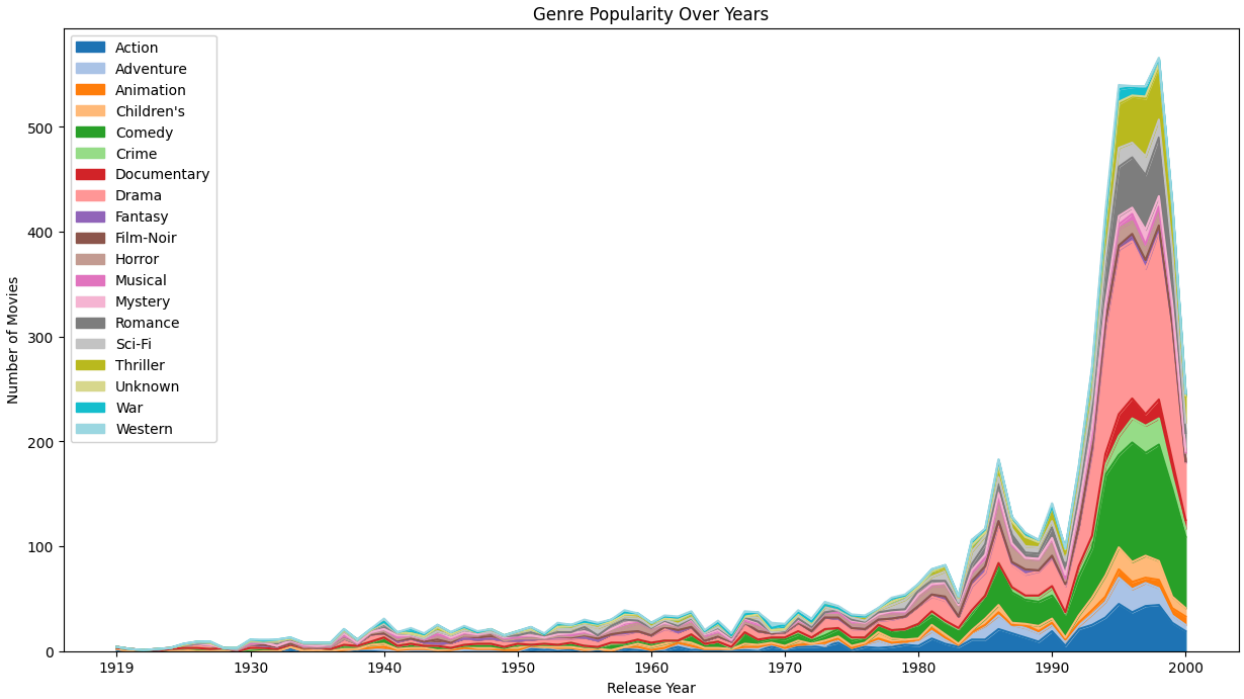
- Release Year has got the range from 1919 to 2000
- Maximum movies got released in 1996 and 1995

Genre Popularity by Year

In [48]:
```
# Aggregate by ReleaseYear
genre_by_year = movies_df.groupby('ReleaseYear').sum()

# Stacked area plot
plt.figure(figsize=(15, 8))
genre_by_year.iloc[:, 2:].plot(kind='area', stacked=True, figsize=(15, 8), col
plt.title('Genre Popularity Over Years')
plt.xlabel('Release Year')
plt.ylabel('Number of Movies')
```

```
plt.legend(loc='upper left')
plt.show()
```

```
<Figure size 1500x800 with 0 Axes>
```

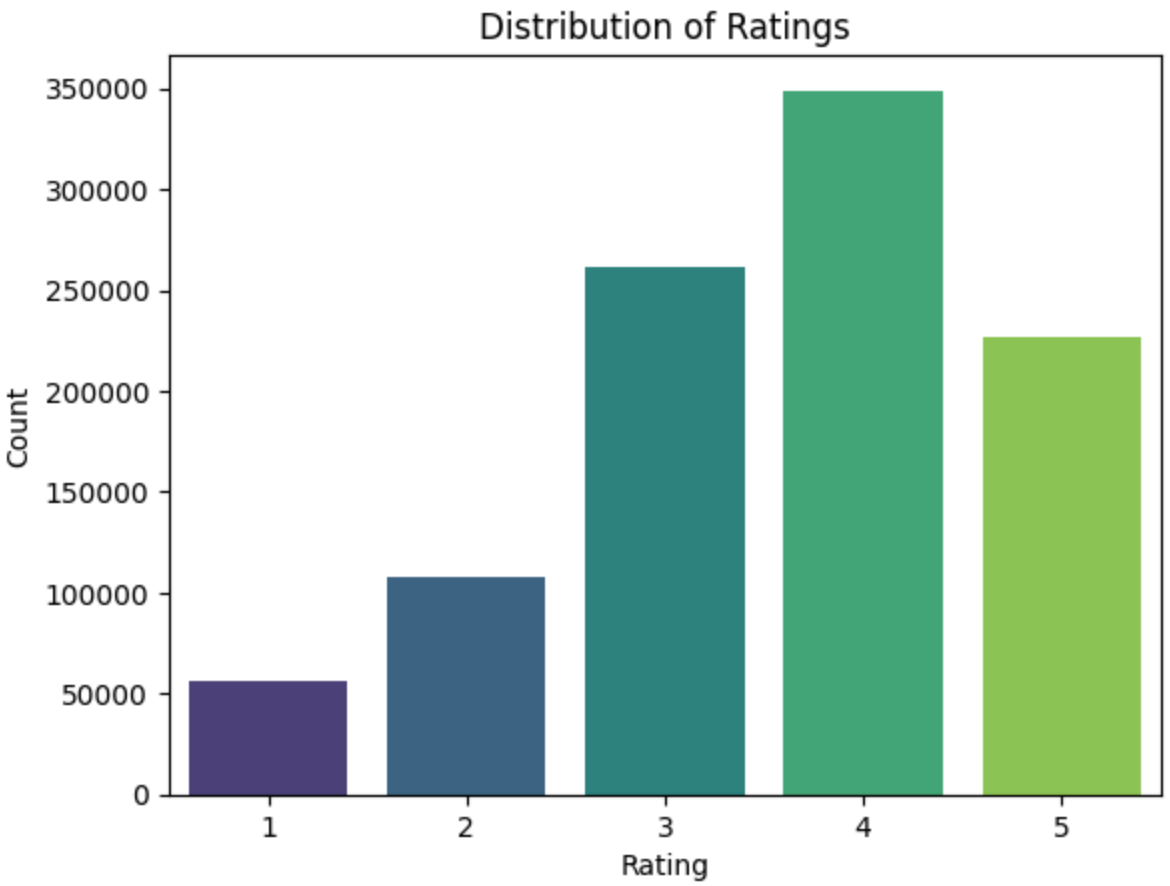


- Number of Movies in Action, Comedy, Drama Genre have increased over years.
- Percentage of Western and Animation movies have relatively remained same over the years
- Crime and Thriller based movies have shown sharp increase around 1995

Ratings

Distribution of Ratings

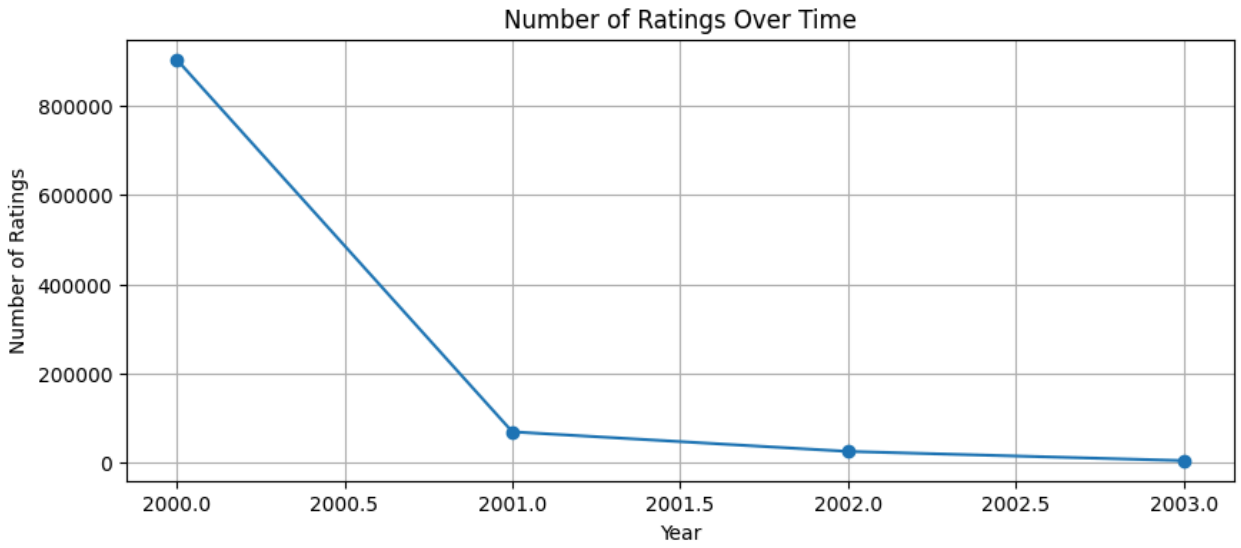
In [49]:
```
sns.countplot(data=ratings_df, x='Rating', palette='viridis')
plt.title('Distribution of Ratings')
plt.xlabel('Rating')
plt.ylabel('Count')
plt.show()
```



Maximum Ratings given by users is 4 followed 3 and 5

Ratings Over Time

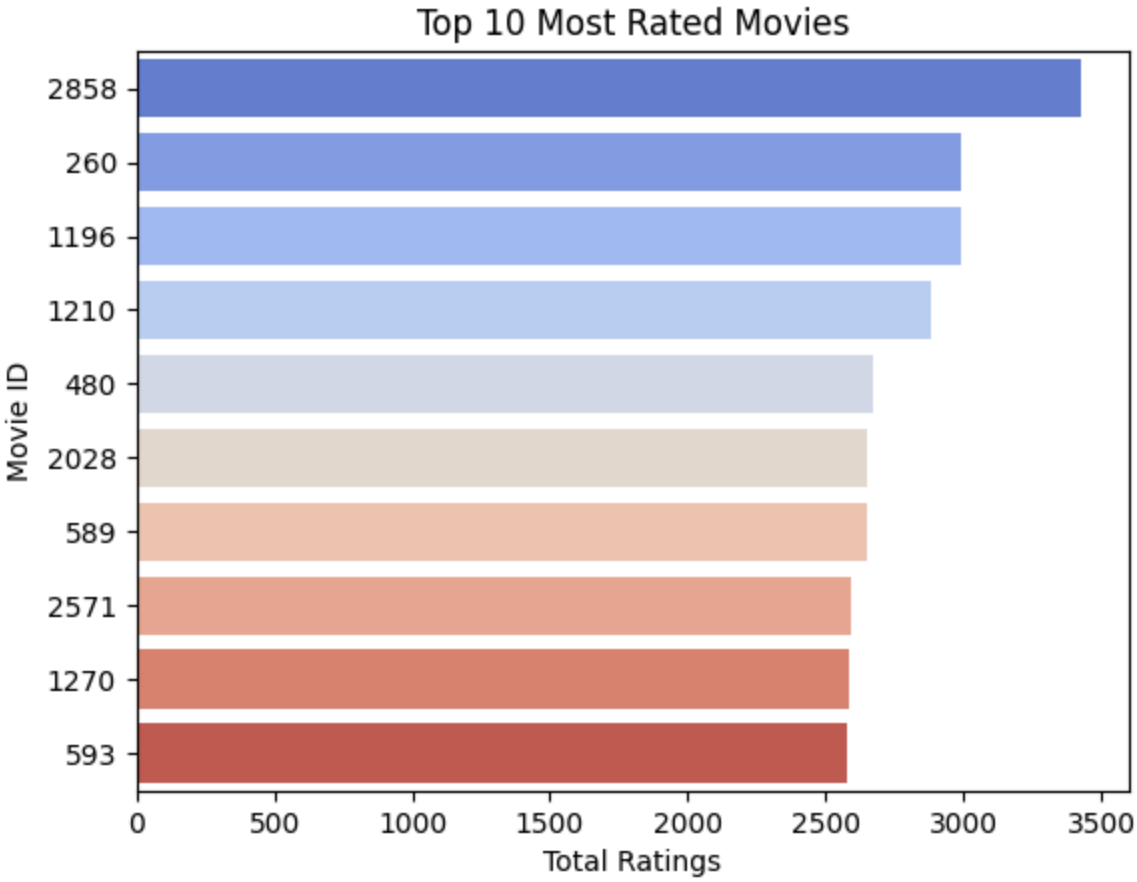
In [50]:
```
yearly_ratings = ratings_df.groupby('Year')['Rating'].count()
plt.figure(figsize=(10, 4))
plt.plot(yearly_ratings.index, yearly_ratings.values, marker='o')
plt.title('Number of Ratings Over Time')
plt.xlabel('Year')
plt.ylabel('Number of Ratings')
plt.grid(True)
plt.show()
```



Maximum number of Ratings were given in the year 2000 which sharply decreased in 2001 and further reducing till 2003

Most Rated Movies

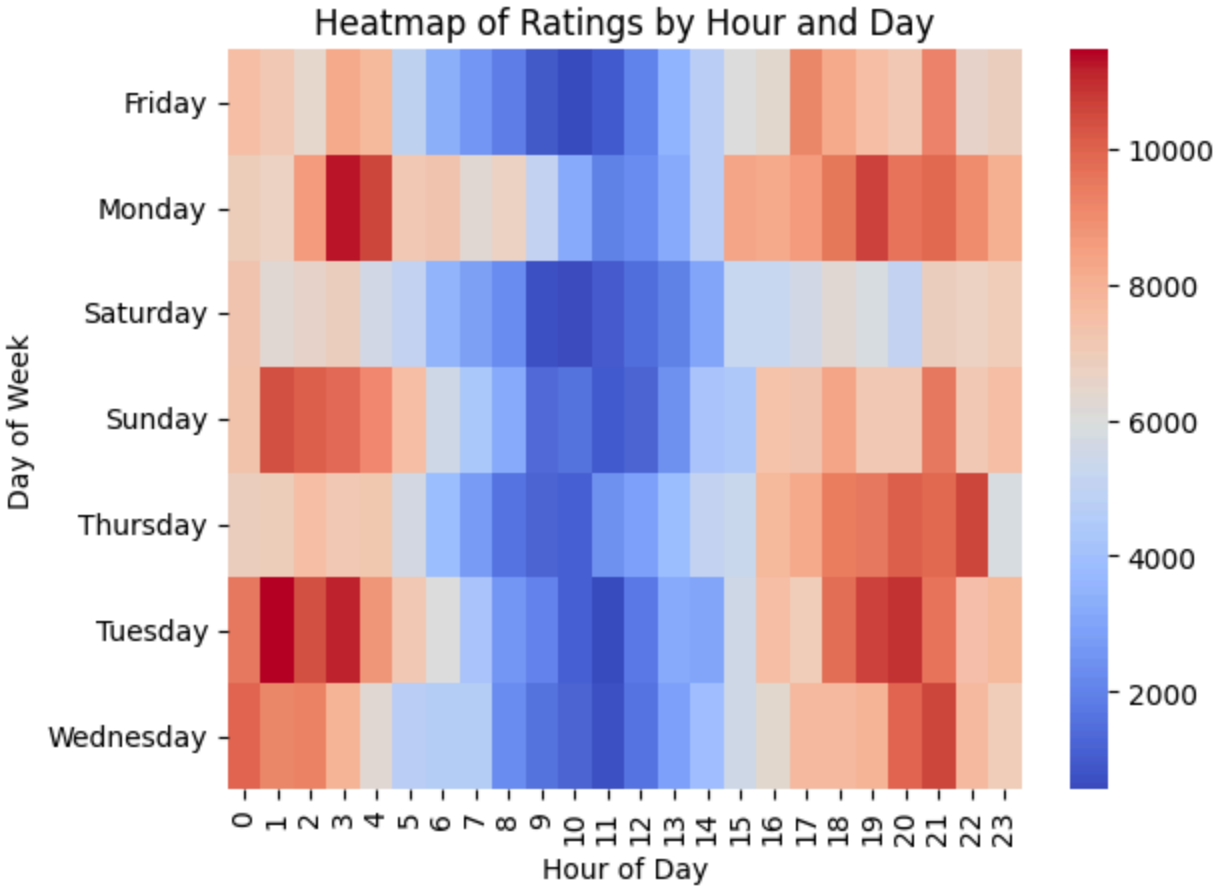
In [51]:
```
top_movies = movie_stats.nlargest(10, 'TotalRatingsPerMovie')
sns.barplot(data=top_movies, x='TotalRatingsPerMovie', y='MovieID', palette='c
plt.title('Top 10 Most Rated Movies')
plt.xlabel('Total Ratings')
plt.ylabel('Movie ID')
plt.show()
```



MovieID 2858 is the most rated followed by 260 and 1196

Ratings by Hour and Day

In [52]:
```
heatmap_data = ratings_df.groupby(['DayOfWeek', 'Hour'])['Rating'].count().uns
sns.heatmap(heatmap_data, cmap='coolwarm', annot=False)
plt.title('Heatmap of Ratings by Hour and Day')
plt.xlabel('Hour of Day')
plt.ylabel('Day of Week')
plt.show()
```



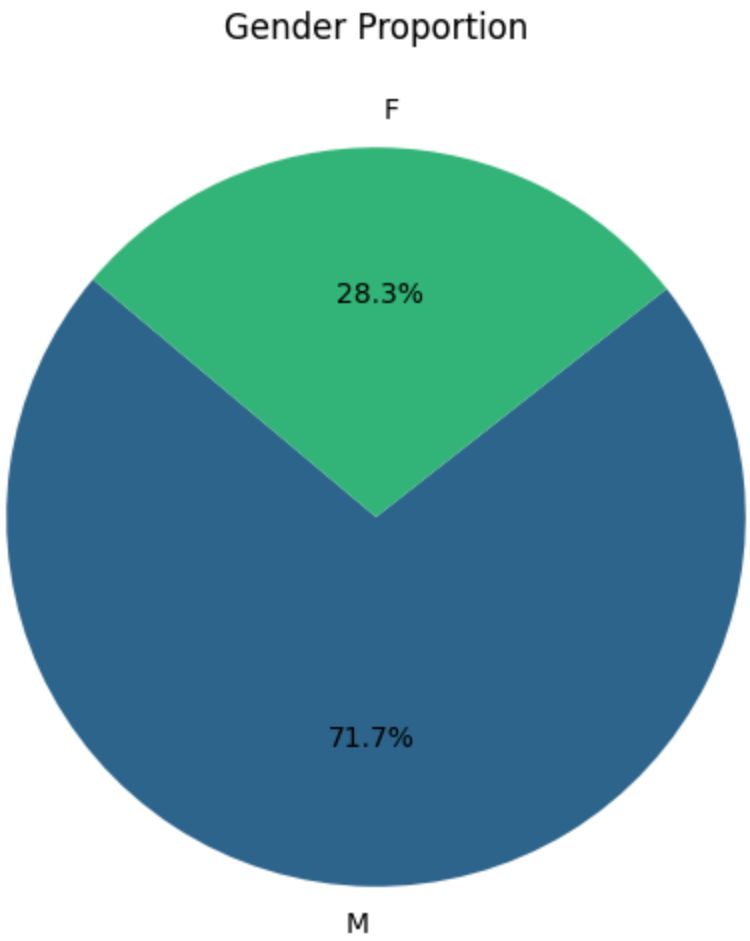
- Most of the ratings are given during evening or late night hours.
- Less ratings are given during 6 hrs to 17 hrs of the day

Users

Distribution of Gender

In [53]:
```
gender_counts = users_df['Gender'].value_counts()
```

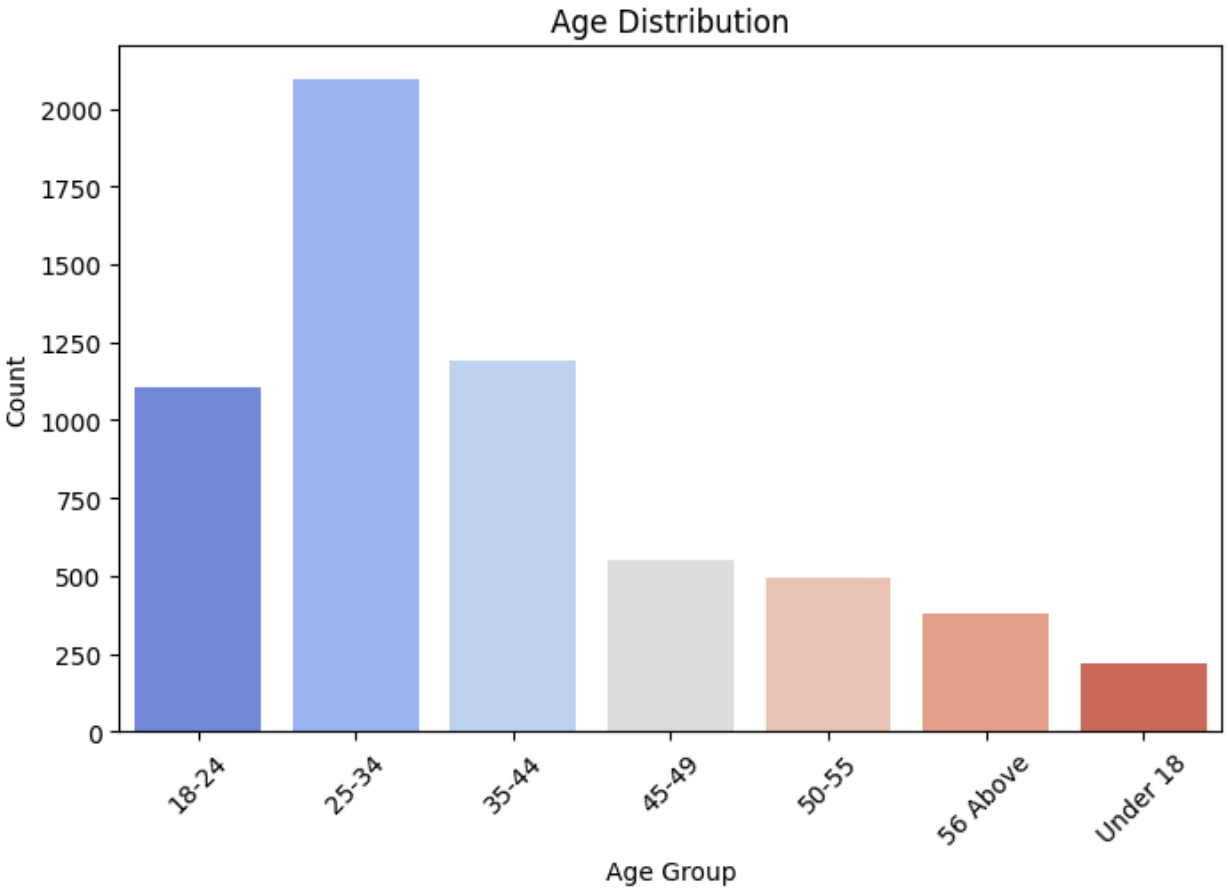
In [54]:
```
# Pie chart for Gender distribution
plt.figure(figsize=(6, 6))
plt.pie(gender_counts, labels=gender_counts.index, autopct='%1.1f%%', startang
plt.title('Gender Proportion')
plt.show()
```



Males are dominating Users distribution with 72% and Females at 28%

Distribution of Age

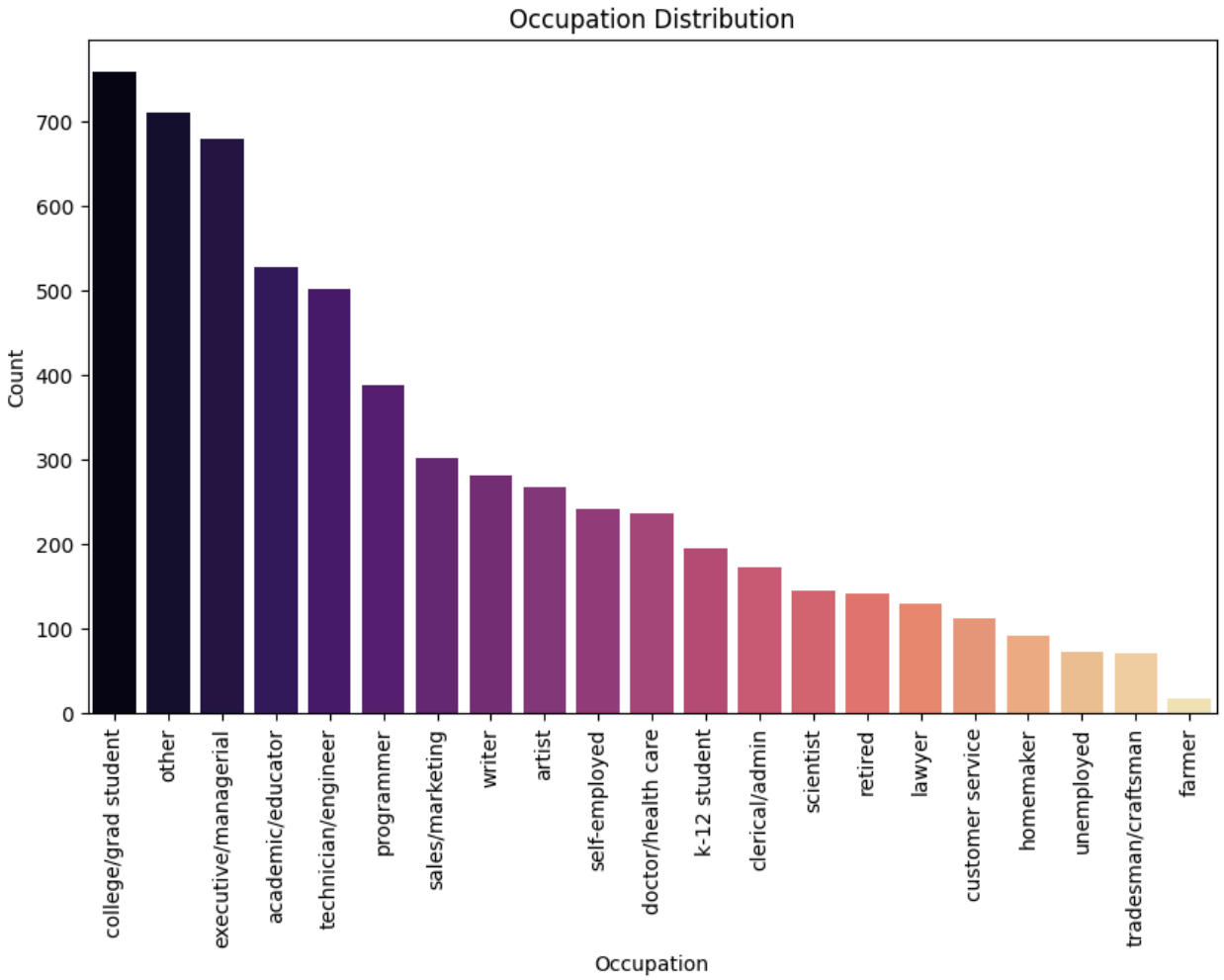
In [55]:
```
age_counts = users_df['Age'].value_counts().sort_index()

# Bar plot for Age distribution
plt.figure(figsize=(8, 5))
sns.barplot(x=age_counts.index, y=age_counts.values, palette='coolwarm')
plt.title('Age Distribution')
plt.xlabel('Age Group')
plt.ylabel('Count')
plt.xticks(rotation=45)
plt.show()
```



More than 2000 users belong to age group 25-34 followed by 35-44 and 18-24 with approx. 1100-1200 users

Distribution of Occupation

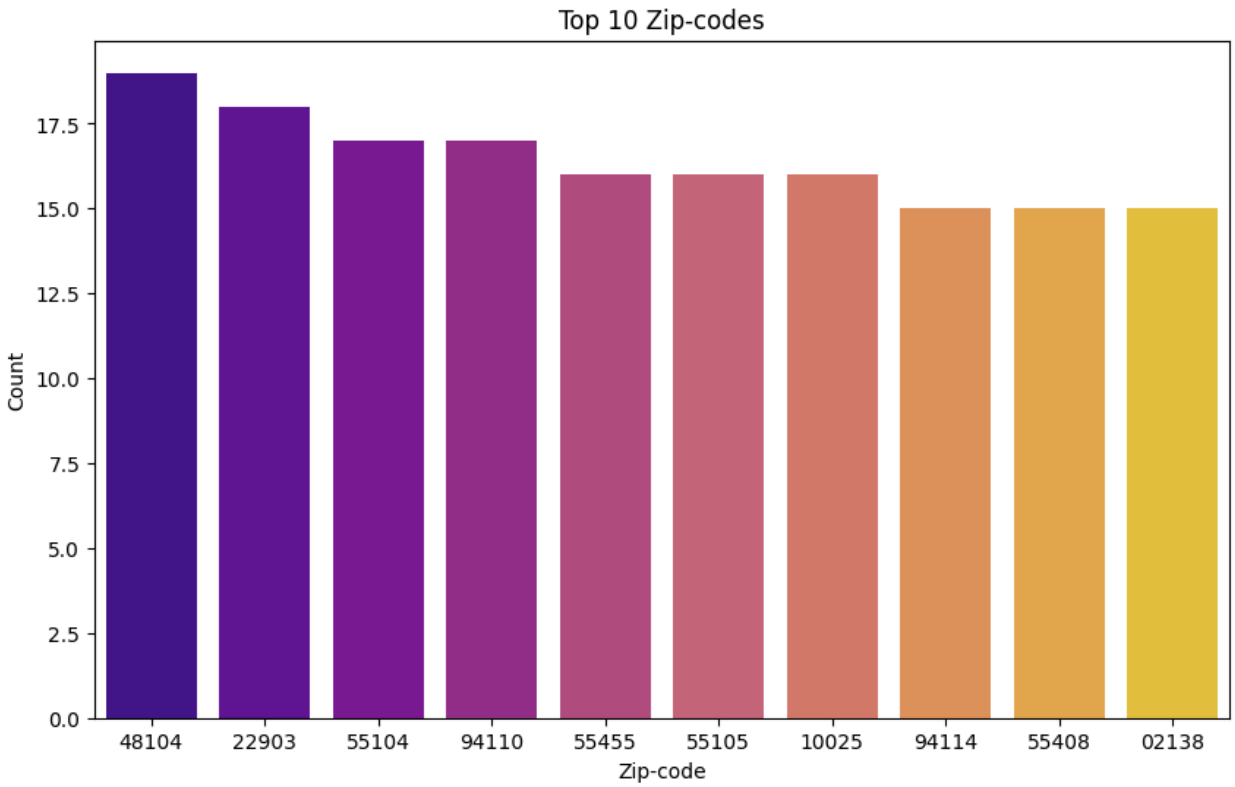
In [56]:
```
occupation_counts = users_df['Occupation'].value_counts()

# Bar plot for Occupation distribution
plt.figure(figsize=(10, 6))
sns.barplot(x=occupation_counts.index, y=occupation_counts.values, palette='ma
plt.title('Occupation Distribution')
plt.xlabel('Occupation')
plt.ylabel('Count')
plt.xticks(rotation=90)
plt.show()
```



Maximum users belong to 'college/grad student' category followed by other and executive/managerial

Distribution of Zip-codes

In [57]:
```
top_zipcodes = users_df['Zip-code'].value_counts().head(10)

# Bar plot for Top 10 Zip-codes
plt.figure(figsize=(10, 6))
sns.barplot(x=top_zipcodes.index, y=top_zipcodes.values, palette='plasma')
plt.title('Top 10 Zip-codes')
plt.xlabel('Zip-code')
plt.ylabel('Count')
plt.show()
```



- Among the top 10 zip-codes in terms of users count, four zip codes start with 55.
- Most of the users belong to zip-code 48104 followed by 22903 and 55104

Aggregate to understand user trends

In [58]:
```
gender_age_occupation = users_df.groupby(['Gender', 'Age', 'Occupation']).size

# Heatmap of Gender-Age-Occupation combinations
pivot_data = gender_age_occupation.pivot_table(index='Occupation', columns=['G

plt.figure(figsize=(14, 10))
sns.heatmap(pivot_data, annot=False, cmap='YlGnBu', cbar=True)
plt.title('User Distribution by Gender, Age, and Occupation')
plt.xlabel('Gender and Age Group')
plt.ylabel('Occupation')
plt.show()
```

- Maximum users (almost 400 numbers) are Male, 18-24 age group, college/grad students.
- Followed by Male, 25-34 age group who are in occupations like technician/engineer, other, executive/managerial

Collaborative Filtering with Pearson Correlation

In [59]:
```
# Create the pivot table
pivot_table = ratings_df.pivot_table(index='MovieID', columns='UserID', values

# Join MovieName from movies_df
movie_id_to_name = movies_df[['MovieID', 'MovieName']].set_index('MovieID')
pivot_table = pivot_table.join(movie_id_to_name, how='left').set_index('MovieN
```

In [60]:
```
pivot_table.head()
```

Out[60]:

MovieName	1	10	100	1000	1001	1002	1003	1004	1005	1006	...	990
Toy Story	5.0	5.0	0.0	5.0	4.0	0.0	0.0	5.0	0.0	0.0	...	4.0
GoldenEye	0.0	0.0	0.0	0.0	0.0	0.0	0.0	2.0	0.0	0.0	...	0.0
City Hall	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0
Curdled	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0
Ed's Next Move	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0

5 rows × 6040 columns

In [61]:
```
user_item_matrix=pivot_table.copy()
```

Handling Sparsity

In [62]:
```
# Step 1: Calculate Sparsity
total_possible_entries = user_item_matrix.shape[0] * user_item_matrix.shape[1]
actual_non_zero_entries = user_item_matrix[user_item_matrix > 0].count().sum()

sparsity = 1 - (actual_non_zero_entries / total_possible_entries)  # Sparsity

print(f"Sparsity of the user-item matrix: {sparsity * 100:.2f}%")
```

```
Sparsity of the user-item matrix: 95.53%
```

In [63]:
```
# Step 1: Filter movies with sufficient valid ratings
min_ratings_per_movie = 100
valid_movie_ratings = (user_item_matrix > 0).sum(axis=1)  # Count of valid rat
user_item_matrix = user_item_matrix.loc[valid_movie_ratings >= min_ratings_per

# Step 2: Filter users with sufficient valid ratings
min_ratings_per_user = 50
valid_user_ratings = (user_item_matrix > 0).sum(axis=0)  # Count of valid rati
user_item_matrix = user_item_matrix.loc[:, valid_user_ratings >= min_ratings_p

# Step 3: Calculate sparsity
total_possible_entries = user_item_matrix.shape[0] * user_item_matrix.shape[1]
actual_ratings = (user_item_matrix > 0).sum().sum()  # Count of valid ratings

sparsity = 1 - (actual_ratings / total_possible_entries)

print(f"Updated Sparsity of the user-item matrix: {sparsity * 100:.2f}%")
```

```
Updated Sparsity of the user-item matrix: 89.58%
```

Data is inherently sparse. Setting threshold is one way based on domain knowledge to handly sparsity.

Correlation Matrix

In [64]:
```
# Normalize Ratings
normalized_matrix = user_item_matrix.sub(user_item_matrix.mean(axis=1), axis=0
```

In [65]:
```
# Compute Pearson Correlation
correlation_matrix = normalized_matrix.T.corr(method='pearson')
```

In [66]:
```
correlation_matrix
```

Out[66]:

MovieName	Toy Story	GoldenEye	City Hall	Extreme Measures	Glimmer Man, The	D3: The Mighty Ducks	Dum
MovieName							
Toy Story	1.000000	0.165284	0.061140	0.092748	0.035637	0.114571	0.1
GoldenEye	0.165284	1.000000	0.107435	0.140387	0.183567	0.110831	0.1
City Hall	0.061140	0.107435	1.000000	0.241540	0.113235	0.040424	0.0
Extreme Measures	0.092748	0.140387	0.241540	1.000000	0.176822	0.023349	0.0
Glimmer Man, The	0.035637	0.183567	0.113235	0.176822	1.000000	0.037029	0.0
...	...	...	...	...	...	...	
Fly Away Home	0.130694	0.089176	0.066711	0.062230	0.020713	0.143787	0.0
Michael Collins	0.044748	0.090125	0.177396	0.105421	0.067072	0.030555	0.0
Big Night	0.079886	-0.002126	0.168937	0.061941	0.020123	-0.000573	0.0
Last Man Standing	0.081032	0.256379	0.167586	0.146446	0.275761	0.062229	0.1
2 Days in the Valley	0.078447	0.139841	0.164672	0.251038	0.113603	0.034169	0.0

2019 rows × 2019 columns

Recommendation

In [67]:
```
def recommend_movies(movie_name, correlation_matrix, top_n=5):
    """
    Recommends movies based on the highest Pearson Correlation.
```

```
    Parameters:
        movie_name (str): Name of the movie to base recommendations on.
        correlation_matrix (pd.DataFrame): Correlation matrix with MovieName a
        top_n (int): Number of recommendations to return.

    Returns:
        pd.Series: Top N recommended movies with their correlation scores.
    """
    if movie_name not in correlation_matrix.index:
        raise ValueError(f"Movie '{movie_name}' not found in the correlation m

    # Retrieve the correlations for the given movie
    movie_correlations = correlation_matrix[movie_name]

    # Exclude the input movie itself
    movie_correlations = movie_correlations.drop(labels=movie_name)

    # Sort movies by correlation in descending order
    top_recommendations = movie_correlations.sort_values(ascending=False).head

    return top_recommendations


# Example usage
# Assuming `correlation_matrix` is the correlation matrix DataFrame
movie_name_input = "Toy Story"  # Replace with any movie name
try:
    recommendations = recommend_movies(movie_name_input, correlation_matrix)
    print(f"Top recommendations for '{movie_name_input}':")
    print(recommendations)
except ValueError as e:
    print(e)
```

```
Top recommendations for 'Toy Story':
MovieName
Toy Story 2             0.463062
Aladdin                 0.434722
Bug's Life, A           0.392486
Lion King, The          0.373634
Beauty and the Beast    0.361543
Name: Toy Story, dtype: float64
```

Cosine Similarity Based Rec Sys

In [68]:
```
from sklearn.metrics.pairwise import cosine_similarity
from sklearn.neighbors import NearestNeighbors
```

In [69]:
```
user_item_dense = user_item_matrix.values
```

In [70]:
```
user_item_dense
```

Out[70]:
```
array([[5., 5., 0., ..., 4., 0., 0.],
       [0., 0., 0., ..., 0., 0., 0.],
       [0., 0., 0., ..., 0., 0., 0.],
       ...,
       [0., 0., 0., ..., 0., 0., 0.],
       [0., 0., 0., ..., 0., 0., 2.],
       [0., 0., 0., ..., 3., 0., 0.]])
```

User Similarity Matrix

In [71]:
```
# Compute cosine similarity between users
user_similarity = cosine_similarity(user_item_dense.T)

# Convert to a DataFrame for readability
user_similarity_df = pd.DataFrame(user_similarity, index=user_item_matrix.colu

# Print User Similarity Matrix
print("User Similarity Matrix:")
print(user_similarity_df)
```

```
User Similarity Matrix:
             1        10       100      1000      1001      1002      1004  \
1     1.000000  0.261787  0.125020  0.209565  0.149456  0.112020  0.182331
10    0.261787  1.000000  0.262388  0.283442  0.179109  0.114892  0.439445
100   0.125020  0.262388  1.000000  0.297539  0.085989  0.111208  0.238244
1000  0.209565  0.283442  0.297539  1.000000  0.107729  0.048004  0.357046
1001  0.149456  0.179109  0.085989  0.107729  1.000000  0.188454  0.170644
...        ...       ...       ...       ...       ...       ...       ...
990   0.080778  0.157479  0.099138  0.171664  0.167598  0.047831  0.304003
991   0.039336  0.193977  0.100418  0.078711  0.031300  0.080747  0.098002
996   0.173124  0.307883  0.347289  0.333629  0.254384  0.228292  0.354596
998   0.129806  0.145297  0.122422  0.106174  0.312482  0.227321  0.081717
999   0.124840  0.253943  0.310443  0.248769  0.201181  0.181750  0.339361

          1005      1008      1009  ...       984       985       987  \
1     0.106362  0.103547  0.050889  ...  0.184076  0.181780  0.160040
10    0.196071  0.223131  0.119083  ...  0.448035  0.277778  0.310740
100   0.175485  0.043367  0.062002  ...  0.250968  0.314467  0.218685
1000  0.330891  0.077724  0.125179  ...  0.267007  0.360655  0.126401
1001  0.159743  0.094975  0.230379  ...  0.225744  0.176107  0.176551
...        ...       ...       ...  ...       ...       ...       ...
990   0.162360  0.068581  0.033705  ...  0.194042  0.215919  0.088331
991   0.062860  0.039695  0.046332  ...  0.162202  0.107208  0.214092
996   0.294862  0.118964  0.155219  ...  0.390142  0.429348  0.267668
998   0.122636  0.060491  0.184632  ...  0.170092  0.113503  0.148852
999   0.169177  0.058129  0.108453  ...  0.373659  0.293351  0.225163

           988        99       990       991       996       998       999
1     0.203063  0.212478  0.080778  0.039336  0.173124  0.129806  0.124840
10    0.196536  0.208272  0.157479  0.193977  0.307883  0.145297  0.253943
100   0.188278  0.156498  0.099138  0.100418  0.347289  0.122422  0.310443
1000  0.176089  0.185862  0.171664  0.078711  0.333629  0.106174  0.248769
1001  0.127046  0.227823  0.167598  0.031300  0.254384  0.312482  0.201181
...        ...       ...       ...       ...       ...       ...       ...
990   0.065398  0.125824  1.000000  0.000000  0.202523  0.066650  0.219261
991   0.084821  0.096551  0.000000  1.000000  0.089496  0.082106  0.088425
996   0.235674  0.243779  0.202523  0.089496  1.000000  0.194533  0.426932
998   0.178177  0.179659  0.066650  0.082106  0.194533  1.000000  0.118092
999   0.198477  0.199411  0.219261  0.088425  0.426932  0.118092  1.000000

[4200 rows x 4200 columns]
```

Item Similarity Matrix

In [72]:
```python
# Compute cosine similarity between items (movies)
item_similarity = cosine_similarity(user_item_dense)

# Convert to a DataFrame for readability
item_similarity_df = pd.DataFrame(item_similarity, index=user_item_matrix.inde

# Print Item Similarity Matrix
print("Item Similarity Matrix:")
```

```
print(item_similarity_df)
```

Item Similarity Matrix:

MovieName	Toy Story	GoldenEye	City Hall	Extreme Measures	\
MovieName					
Toy Story	1.000000	0.400530	0.153949	0.174990	
GoldenEye	0.400530	1.000000	0.167786	0.195373	
City Hall	0.153949	0.167786	1.000000	0.261759	
Extreme Measures	0.174990	0.195373	0.261759	1.000000	
Glimmer Man, The	0.118816	0.224977	0.133774	0.195438	
...	...	...	...	...	
Fly Away Home	0.239534	0.174118	0.100275	0.095163	
Michael Collins	0.160285	0.164553	0.203646	0.133327	
Big Night	0.265007	0.137329	0.210349	0.109074	
Last Man Standing	0.209375	0.324535	0.198632	0.177548	
2 Days in the Valley	0.218774	0.229813	0.198044	0.279819	

MovieName	Glimmer Man, The	D3: The Mighty Ducks	\
MovieName			
Toy Story	0.118816	0.192767	
GoldenEye	0.224977	0.170182	
City Hall	0.133774	0.066385	
Extreme Measures	0.195438	0.049159	
Glimmer Man, The	1.000000	0.059166	
...	...	...	
Fly Away Home	0.050673	0.174176	
Michael Collins	0.092352	0.061293	
Big Night	0.063360	0.051309	
Last Man Standing	0.297878	0.097150	
2 Days in the Valley	0.143553	0.072936	

MovieName	Apple Dumpling Gang, The	Escape to Witch Mountain	\
MovieName			
Toy Story	0.227701	0.253128	
GoldenEye	0.204948	0.220000	
City Hall	0.103043	0.128444	
Extreme Measures	0.119035	0.157116	
Glimmer Man, The	0.090425	0.050409	
...	...	...	
Fly Away Home	0.141908	0.179801	
Michael Collins	0.123000	0.131219	
Big Night	0.083573	0.132737	
Last Man Standing	0.154718	0.130493	
2 Days in the Valley	0.094182	0.107211	

MovieName	Bottle Rocket	Love Bug, The	...	39 Steps, The	\
MovieName			...		
Toy Story	0.232098	0.255069	...	0.188555	
GoldenEye	0.139231	0.236144	...	0.143667	
City Hall	0.125115	0.086767	...	0.137216	
Extreme Measures	0.077703	0.110281	...	0.108148	
Glimmer Man, The	0.027053	0.083612	...	0.063697	
...	...	...	...	...	
Fly Away Home	0.071541	0.178978	...	0.092007	
Michael Collins	0.108650	0.105194	...	0.085855	
Big Night	0.227466	0.117991	...	0.209203	

```
Last Man Standing          0.099015       0.133171  ...       0.088757
2 Days in the Valley       0.173540       0.123624  ...       0.150901

MovieName             Night of the Living Dead  African Queen, The  \
MovieName
Toy Story                             0.259092            0.328900
GoldenEye                             0.205812            0.252117
City Hall                             0.100287            0.135923
Extreme Measures                      0.094399            0.085473
Glimmer Man, The                      0.100891            0.087299
...                                        ...                 ...
Fly Away Home                         0.089989            0.176855
Michael Collins                       0.114724            0.177596
Big Night                             0.164712            0.242355
Last Man Standing                     0.150897            0.154761
2 Days in the Valley                  0.164030            0.168148

MovieName             Cat on a Hot Tin Roof  \
MovieName
Toy Story                          0.195654
GoldenEye                          0.128772
City Hall                          0.112307
Extreme Measures                   0.075811
Glimmer Man, The                   0.073254
...                                     ...
Fly Away Home                      0.105240
Michael Collins                    0.131728
Big Night                          0.251326
Last Man Standing                  0.094396
2 Days in the Valley               0.116531

MovieName             Blue Angel, The (Blaue Engel, Der)  Fly Away Home  \
MovieName
Toy Story                                       0.088083       0.239534
GoldenEye                                       0.054348       0.174118
City Hall                                       0.024168       0.100275
Extreme Measures                                0.027452       0.095163
Glimmer Man, The                                0.018904       0.050673
...                                                  ...            ...
Fly Away Home                                   0.060899       1.000000
Michael Collins                                 0.062552       0.107459
Big Night                                       0.128710       0.154779
Last Man Standing                               0.034358       0.082312
2 Days in the Valley                            0.062772       0.113399

MovieName             Michael Collins  Big Night  Last Man Standing  \
MovieName
Toy Story                    0.160285   0.265007           0.209375
GoldenEye                    0.164553   0.137329           0.324535
City Hall                    0.203646   0.210349           0.198632
Extreme Measures             0.133327   0.109074           0.177548
Glimmer Man, The             0.092352   0.063360           0.297878
...                               ...        ...                ...
Fly Away Home                0.107459   0.154779           0.082312
```

```
Michael Collins              1.000000   0.223533          0.143363
Big Night                    0.223533   1.000000          0.098496
Last Man Standing            0.143363   0.098496          1.000000
2 Days in the Valley         0.161157   0.205963          0.193456
```

```
MovieName             2 Days in the Valley
MovieName
Toy Story                         0.218774
GoldenEye                         0.229813
City Hall                         0.198044
Extreme Measures                  0.279819
Glimmer Man, The                  0.143553
...                                    ...
Fly Away Home                     0.113399
Michael Collins                   0.161157
Big Night                         0.205963
Last Man Standing                 0.193456
2 Days in the Valley              1.000000

[2019 rows x 2019 columns]
```

KNN Algorithm

In [73]:
```python
# Fit a Nearest Neighbors model using the item similarity matrix
model_knn = NearestNeighbors(metric='cosine', algorithm='brute')
model_knn.fit(user_item_dense)
```

Out[73]:

NearestNeighbors

```
NearestNeighbors(algorithm='brute', metric='cosine')
```

In [74]:
```python
def recommend_movies(movie_name, user_item_matrix, item_similarity_df, model_k
    """
    Recommend movies using an Item-Based Collaborative Filtering approach.

    Parameters:
        movie_name (str): The name of the movie to base recommendations on.
        user_item_matrix (pd.DataFrame): User-item matrix of ratings.
        item_similarity_df (pd.DataFrame): Precomputed item similarity matrix.
        model_knn (NearestNeighbors): Fitted nearest neighbors model.
        n_neighbors (int): Number of similar movies to recommend.

    Returns:
        pd.Series: Recommended movies with similarity scores.
    """
    if movie_name not in user_item_matrix.index:
        return f"Movie '{movie_name}' not found in the dataset."

    # Get the index of the input movie
    movie_index = user_item_matrix.index.get_loc(movie_name)
```

```
    # Find the nearest neighbors
    distances, indices = model_knn.kneighbors(user_item_dense[movie_index].res

    # Get recommended movie indices and distances
    recommended_indices = indices.flatten()[1:]  # Exclude the input movie
    recommended_distances = distances.flatten()[1:]

    # Map indices back to movie names
    recommended_movies = user_item_matrix.index[recommended_indices]

    # Combine results into a DataFrame
    recommendations = pd.DataFrame({
        'Movie': recommended_movies,
        'Distance': recommended_distances
    }).sort_values(by='Distance', ascending=True)

    return recommendations
```

In [75]:
```
# Input movie for recommendation
input_movie = "Toy Story"

# Get recommendations
recommendations = recommend_movies(input_movie, user_item_matrix, item_similar

print(f"Top 5 recommendations for '{input_movie}':")
print(recommendations)
```

```
Top 5 recommendations for 'Toy Story':
                Movie  Distance
0         Toy Story 2  0.342778
1       Groundhog Day  0.364586
2             Aladdin  0.372198
3       Bug's Life, A  0.378554
4  Back to the Future  0.389510
```

In [76]:
```
# Input movie for recommendation
input_movie = "Liar Liar"

# Get recommendations
recommendations = recommend_movies(input_movie, user_item_matrix, item_similar

print(f"Top 5 recommendations for '{input_movie}':")
print(recommendations)
```

```
Top 5 recommendations for 'Liar Liar':
                       Movie  Distance
0              Mrs. Doubtfire  0.428992
1  Ace Ventura: Pet Detective  0.465319
2               Dumb & Dumber  0.476022
3                  Home Alone  0.479516
4               Wayne's World  0.486732
```

Matrix Factorization

In [77]:
```
%pip install 'numpy<2'
```

```
Requirement already satisfied: numpy<2 in /usr/local/lib/python3.12/dist-packag
es (1.26.4)
```

In [78]:
```
%pip install scikit-surprise --upgrade --no-deps
```

```
Collecting scikit-surprise
  Using cached scikit_surprise-1.1.4.tar.gz (154 kB)
  Installing build dependencies ... done
  Getting requirements to build wheel ... done
  Preparing metadata (pyproject.toml) ... done
Building wheels for collected packages: scikit-surprise
  Building wheel for scikit-surprise (pyproject.toml) ... done
  Created wheel for scikit-surprise: filename=scikit_surprise-1.1.4-cp312-cp31
2-linux_x86_64.whl size=2554980 sha256=95ebdb7a69b4101ed98c42d8e0298f4efaa1bd53
d60af7e75faa2ee3615dd08f
  Stored in directory: /root/.cache/pip/wheels/75/fa/bc/739bc2cb1fbaab6061854e6
cfbb81a0ae52c92a502a7fa454b
Successfully built scikit-surprise
Installing collected packages: scikit-surprise
Successfully installed scikit-surprise-1.1.4
```

In [79]:
```
pip install scikit-surprise
```

```
Requirement already satisfied: scikit-surprise in /usr/local/lib/python3.12/dis
t-packages (1.1.4)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-
packages (from scikit-surprise) (1.5.3)
Requirement already satisfied: numpy>=1.19.5 in /usr/local/lib/python3.12/dist-
packages (from scikit-surprise) (1.26.4)
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.12/dist-p
ackages (from scikit-surprise) (1.16.3)
```

In [80]:
```
from surprise import SVD
from surprise import Dataset, Reader
from surprise.model_selection import train_test_split
from surprise import accuracy
```

In [81]:
```
# Preparing the data
reader = Reader(rating_scale=(0, 5))
data = Dataset.load_from_df(ratings_df[['UserID', 'MovieID', 'Rating']], reade

# Train-test split
trainset, testset = train_test_split(data, test_size=0.2)

# Train an SVD model
svd_model = SVD(n_factors=4)
svd_model.fit(trainset)
```

```
# Predictions
predictions = svd_model.test(testset)
```

Evaluation- RMSE / MAPE

In [82]:
```
# Evaluation
rmse = accuracy.rmse(predictions)
```

```
RMSE: 0.8872
```

An RMSE of 0.8813 indicates that, on average, model's predicted ratings deviate from the true ratings by approximately 0.88 units.

In [83]:
```
# MAPE Function
def mean_absolute_percentage_error(y_true, y_pred):
    y_true, y_pred = np.array(y_true), np.array(y_pred)
    return np.mean(np.abs((y_true - y_pred) / y_true)) * 100

# Extract true and predicted ratings
true_ratings = [pred.r_ui for pred in predictions]
pred_ratings = [pred.est for pred in predictions]

mape = mean_absolute_percentage_error(true_ratings, pred_ratings)
print(f"MAPE: {mape:.2f}%")
```

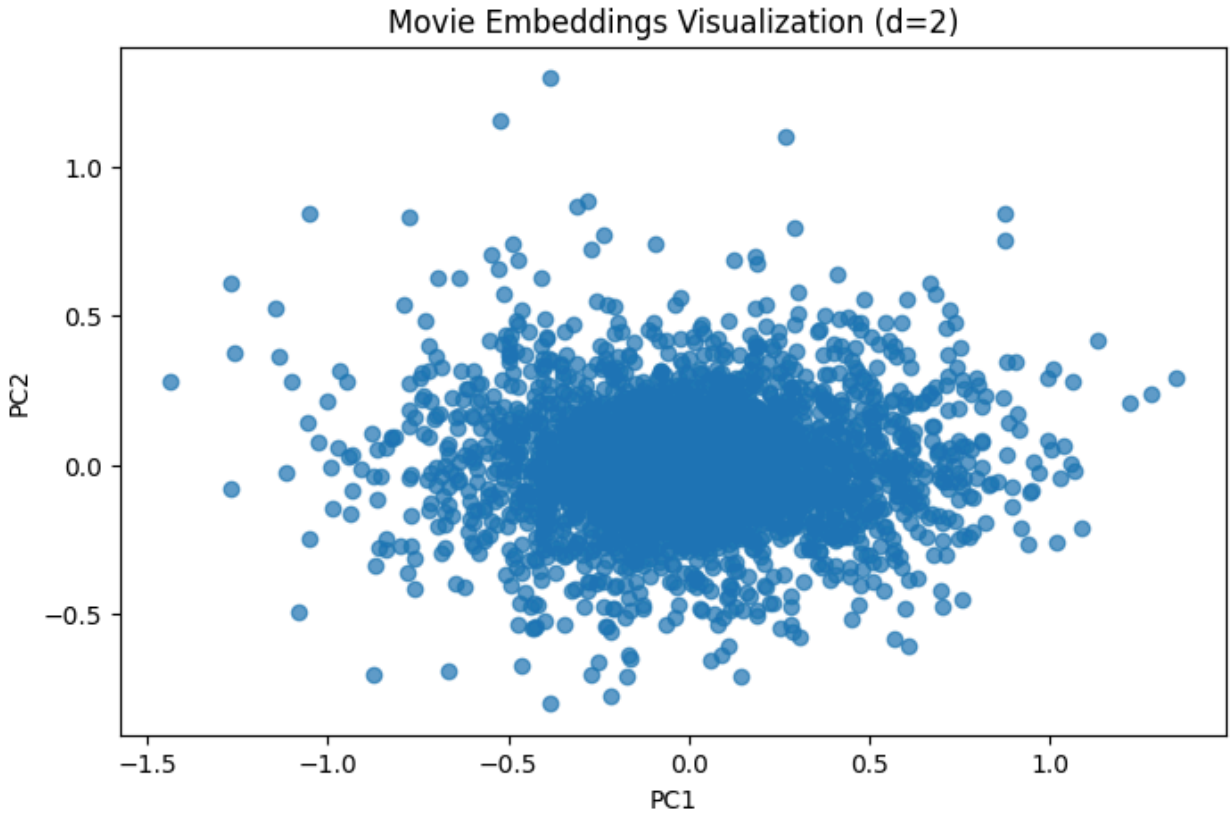
```
MAPE: 27.19%
```

A MAPE of 26.82% means that, on average, model's predictions are off by 26.82% of the true ratings.

Obtain and Visualize Embeddings PCA / TSNE

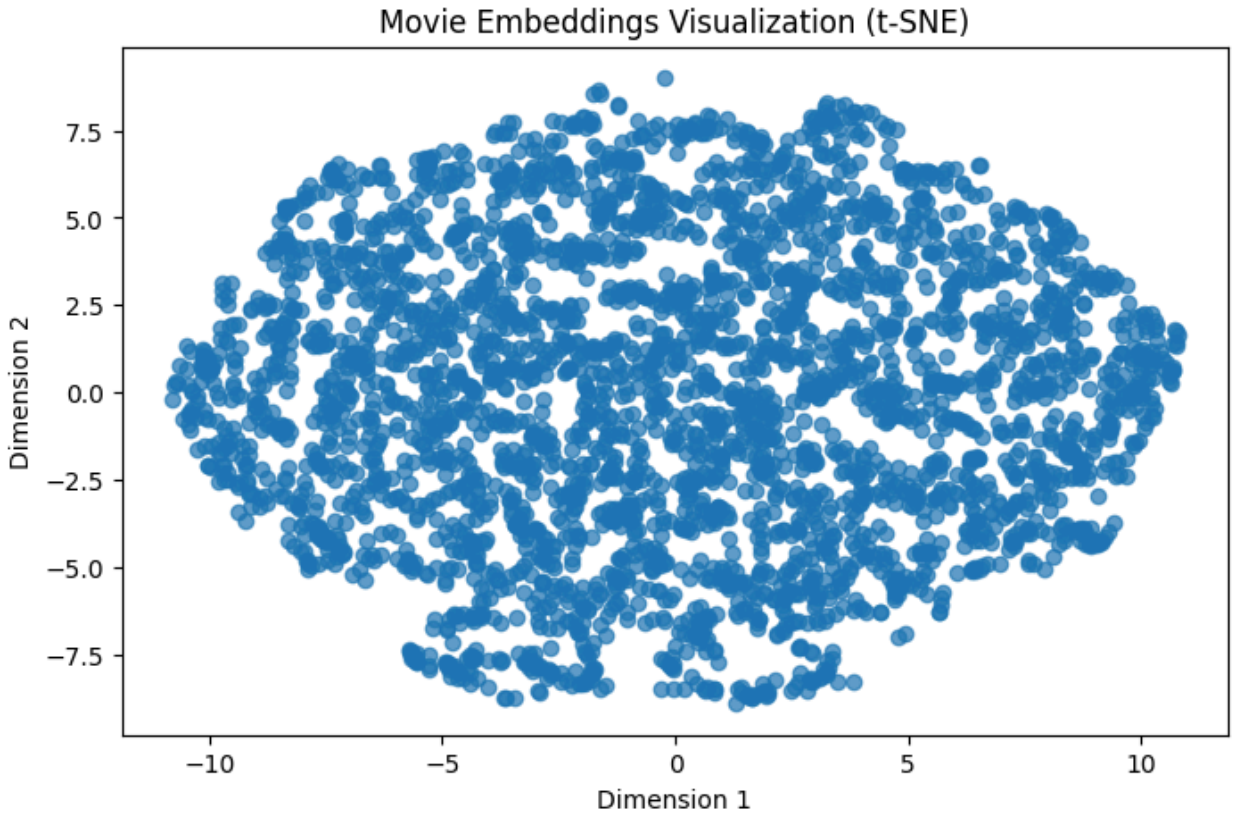
In [84]:
```
from sklearn.decomposition import PCA
```

In [85]:
```
# Example: Movie Embeddings
movie_embeddings = svd_model.qi  # Movie latent factors (Q matrix)

# PCA to reduce to 2 dimensions
pca = PCA(n_components=2)
movie_embeddings_2d = pca.fit_transform(movie_embeddings)

# Plot
plt.figure(figsize=(8, 5))
plt.scatter(movie_embeddings_2d[:, 0], movie_embeddings_2d[:, 1], alpha=0.7)
plt.title('Movie Embeddings Visualization (d=2)')
plt.xlabel('PC1')
plt.ylabel('PC2')
plt.show()
```



In [86]:
```
from sklearn.manifold import TSNE
```

In [87]:
```
tsne = TSNE(n_components=2, perplexity=30, n_iter=300)
movie_embeddings_tsne = tsne.fit_transform(movie_embeddings)

plt.figure(figsize=(8, 5))
plt.scatter(movie_embeddings_tsne[:, 0], movie_embeddings_tsne[:, 1], alpha=0.
plt.title('Movie Embeddings Visualization (t-SNE)')
plt.xlabel('Dimension 1')
plt.ylabel('Dimension 2')
plt.show()
```



Embedding Visualization:

- PCA clusters movies based on genres or similar user ratings.
- t-SNE offers better local clustering but look harder to interpret globally.

Movie Recommendation

In [88]:
```
# Cosine similarity for movie embeddings
similarities = cosine_similarity(movie_embeddings)

# Example: Recommend similar movies for a specific MovieID
movie_index = 2  # Example: First movie
# Exclude the movie itself by masking its similarity score
similarities[movie_index, movie_index] = -np.inf
similar_movies = similarities[movie_index].argsort()[-5:][::-1]  # Top 5
print(f"Movies similar to {movies_df.iloc[movie_index]['MovieName']}:")
for idx in similar_movies:
    print(movies_df.iloc[idx]['MovieName'])
```

```
Movies similar to City Hall:
Manhattan
Cool Hand Luke
Until the End of the World (Bis ans Ende der Welt)
Midnight Express
U Turn
```

Insights

- Highest number of movies nearly 1600 belong to Drama followed by Comedy and then Action
- Release Year has got the range from 1919 to 2000
- Maximum movies got released in 1996 and 1995
- Number of Movies in Action, Comedy, Drama Genre have increased over years.
- Percentage of Western and Animation movies have relatively remained same over the years
- Crime and Thriller based movies have shown sharp increase around 1995
- Maximum Ratings given by users is 4 followed 3 and 5
- Maximum number of Ratings were given in the year 2000 which sharply decreased in 2001 and further reducing till 2003
- MovieID 2858 is the most rated followed by 260 and 1196
- Most of the ratings are given during evening or late night hours.
- Less ratings are given during 0600 hrs to 1700 hrs of the day
- Males are dominating Users distribution with 72% and Females at 28%
- More than 2000 users belong to age group 25-34 followed by 35-44 and 18-24 with approx. 1100-1200 users
- Maximum users belong to 'college/grad student' category followed by 'other' and 'executive/managerial'
- Among the top 10 zip-codes in terms of users count, four zip codes start with 55.
- Most of the users belong to zip-code 48104 followed by 22903 and 55104
- Maximum users (almost 400 numbers) are Male, 18-24 age group, college/grad students.
- Followed by Male, 25-34 age group who are in occupations like technician/engineer, other, executive/managerial
- Sparsity of the user-item matrix: 95.53%
- Movie Recommendations have been made using collaborative filtering with Pearson Correlation
- Movie Recommendations made using Cosine similarity and KNN algorithm
- User Similarity matrix is generated
- Item similarity matrix is generated
- Movie Recommendations also shared on Matrix Factorization using

scikit-surprise library and SVD
- RMSE of the svd model is found to be 0.8813 and MAPE is 27.18%
- PCA clusters movies based on genres or similar user ratings.
- t-SNE offers better local clustering but look harder to interpret globally.

Business Recommendations

1. Enhancing User Engagement and Satisfaction

- Personalized Recommendations: The recommender system’s ability to tailor suggestions based on user preferences, demographics, and historical ratings is key to improving engagement. By aligning recommendations with user tastes, the platform can foster deeper connections with its content.

- Diverse Suggestions: Ensure that recommendations balance highly rated popular movies with lesser-known but highly rated niche films to encourage discovery.

- Timing Insights: Given the insights on rating activity, personalized notifications or reminders during peak engagement hours (evenings and late nights) can improve interaction rates.

2. Increasing User Retention and Platform Usage

- Dynamic User Profiles: Continuously update user profiles based on their recent interactions to make recommendations more relevant.

- Rewarding Engagement: Introduce gamification elements, such as badges or rewards for rating movies, which can increase platform stickiness.

- Community Features: Allow users to create watchlists, share recommendations with friends, or participate in movie-related discussions.

3. Accuracy and Patterns in User Preferences

- Rating Patterns: Most ratings are between 3 and 4, suggesting moderate satisfaction. Focus on enhancing recommendations for users with mid-range ratings to identify potential favorites.

- Demographic Trends: Tailor recommendations based on demographics, e.g., males in the 18-24 age group who dominate the user base.
- Genre Insights: Leverage the increase in Action, Comedy, and Drama movies to highlight trending genres, while also featuring underrepresented genres for variety.

4. Comparison of Model Performance

- **Pearson Correlation:**

  Strengths: Effective for capturing linear relationships between users' preferences.

  Limitations: Limited scalability with sparse datasets (95.53% sparsity).

- **Cosine Similarity:**

  Strengths: Captures angle-based similarity; works well with sparse data.

  Limitations: Does not consider magnitude, which can limit nuanced comparisons.

- **Matrix Factorization (SVD):**

  Strengths: Captures latent factors that influence user preferences. Offers the best predictive performance with RMSE (0.8813) and MAPE (27.18%).

  Limitations: Requires substantial computational resources and fine-tuning.

5. Strategies for Refinement

- Incorporate Implicit Feedback: Include data such as time spent browsing movies, clicks, or watch completions to enhance recommendations.
- Hybrid Models: Combine collaborative filtering with content-based methods to improve accuracy, especially for new users or less-rated movies.
- Hyperparameter Tuning: Optimize SVD and other algorithms by exploring different configurations (e.g., number of latent factors,

regularization).

6. Monitoring and Updating the System

- Periodic Retraining: Regularly retrain models using updated data to adapt to evolving user preferences.
- User Feedback Loop: Collect explicit feedback on recommendations to refine the model.
- Performance Metrics: Track RMSE, MAPE, and user satisfaction metrics to assess system efficacy.

7. Areas for Further Research

- Temporal Dynamics: Explore how preferences shift over time and integrate time-aware models.
- Cross-Domain Recommendations: Leverage insights from other content types (e.g., TV shows, books) for richer recommendations.
- Demographic Diversity: Investigate trends for underrepresented groups to enhance inclusivity.

8. Scalability and Adaptability

- Scalability: Ensure models can handle growth in users and content by leveraging distributed computing frameworks.
- Content Adaptability: Extend the system to other media types like music or games, using genre or feature-based embeddings for flexibility.
- User Segments: Customize strategies for high-value users (e.g., frequent raters) and new users (e.g., cold-start problem).

Conclusion:

By combining accurate, personalized recommendations with strategies to enhance engagement and scalability, the movie recommender system can drive user satisfaction and retention. Regular monitoring and incorporating diverse feedback mechanisms will ensure the system evolves in line with user behaviors and preferences.

Questionnaire

1. Users of which age group have watched and rated the most number of movies?

Ans: More than 2000 users belong to age group 25-34 who have watched and rated most number of movies

2. Users belonging to which profession have watched and rated the most movies?

Ans: From above EDA and plot the answer is 'College/grad students'

3. Most of the users in our dataset who've rated the movies are Male (T/F)?

Ans: True, 72% are Males

4. Most of the movies present in our dataset were released in which decade? 70s b. 90s c. 50s d.80s

Ans: From the time series plot above, maximum movies got released in the year 1995 and 1996 , so the answer is 90s

5. The movie with maximum no. of ratings is ___.

In [89]:
```
# Find the row with the maximum TotalRatingsPerMovie
max_ratings_movie = movie_stats.loc[movie_stats['TotalRatingsPerMovie'].idxmax

# Extract the MovieID and corresponding stats
print(f"Movie with maximum ratings:")
print(f"MovieID: {max_ratings_movie['MovieID']}")
print(f"Average Rating: {max_ratings_movie['AvgRatingPerMovie']}")
print(f"Total Ratings: {max_ratings_movie['TotalRatingsPerMovie']}")
```

```
Movie with maximum ratings:
MovieID: 2858
Average Rating: 4.3173862310385065
Total Ratings: 3428
```

In [90]:
```
# Merge to get MovieName
movie_with_max_ratings = movies_df.merge(
    movie_stats.loc[[movie_stats['TotalRatingsPerMovie'].idxmax()]],
    on='MovieID'
)

print(f"Movie Name: {movie_with_max_ratings['MovieName'].values[0]}")
```

```
print(f"Average Rating: {movie_with_max_ratings['AvgRatingPerMovie'].values[0]
print(f"Total Ratings: {movie_with_max_ratings['TotalRatingsPerMovie'].values[
```

```
Movie Name: American Beauty
Average Rating: 4.3173862310385065
Total Ratings: 3428
```

Ans: American Beauty is the movie with maximum rating

6. Name the top 3 movies similar to 'Liar Liar' on the item-based approach.

In [91]:
```
# Input movie for recommendation
input_movie = "Liar Liar"

# Get recommendations
recommendations = recommend_movies(input_movie, user_item_matrix, item_similar

print(f"Top 5 recommendations for '{input_movie}':")
print(recommendations)
```

```
Top 5 recommendations for 'Liar Liar':
                       Movie  Distance
0              Mrs. Doubtfire  0.428992
1  Ace Ventura: Pet Detective  0.465319
2               Dumb & Dumber  0.476022
3                  Home Alone  0.479516
4               Wayne's World  0.486732
```

Ans: Top 3 movies similar to 'Liar Liar' are

- Mrs. Doubtfire
- Ace Ventura: Pet Detective
- Dumb & Dumber

7. On the basis of approach, Collaborative Filtering methods can be classified into ___-based and ___-based.

Ans: On the basis of approach, Collaborative Filtering methods can be classified into user-based and item-based.

8. Pearson Correlation ranges between ___ to ___ whereas, Cosine Similarity belongs to the interval between ___ to ___.

Pearson Correlation ranges between -1 to +1.

- -1 indicates a perfect negative linear relationship.
- +1 indicates a perfect positive linear relationship.
- 0 indicates no linear relationship.

Cosine Similarity belongs to the interval between 0 to 1 (if dealing with non-negative vectors).

- 0 indicates no similarity.
- 1 indicates complete similarity (vectors are identical).

9. Mention the RMSE and MAPE that you got while evaluating the Matrix Factorization model.

Ans: RMSE is found to be 0.8813 and MAPE is 27.18%

10. Give the sparse 'row' matrix representation for the following dense matrix-

[[1 0] [3 7]]

Ans: (row_ptr, col_indices, values)=([0,1,3],[0,0,1],[1,3,7])

Using StreamLit

In [ ]:
```
import streamlit as st
import plotly.express as px
import plotly.graph_objects as go
from plotly.subplots import make_subplots
import pandas as pd
import numpy as np

# ── Page Config ──────────────────────────────────────────────────────
st.set_page_config(
    page_title="ZEE Movie Recommender System",
    page_icon="🎬",
    layout="wide",
    initial_sidebar_state="expanded",
)

# ── Plotly dark theme defaults ──────────────────────────────────────
PLOTLY_THEME = "plotly_dark"
BG_COLOR     = "#0a0c12"
PAPER_COLOR  = "#111420"
ACCENT       = "#e84545"
ACCENT2      = "#f5a623"
ACCENT3      = "#4ade80"
GRID_COLOR   = "#1e2640"

def apply_theme(fig, height=420):
    fig.update_layout(
        template=PLOTLY_THEME,
        plot_bgcolor=BG_COLOR,
```

```
        paper_bgcolor=PAPER_COLOR,
        font=dict(family="DM Sans, sans-serif", color="#c8d4f0", size=12),
        height=height,
        margin=dict(l=40, r=20, t=50, b=40),
        xaxis=dict(gridcolor=GRID_COLOR, zeroline=False),
        yaxis=dict(gridcolor=GRID_COLOR, zeroline=False),
        legend=dict(bgcolor="rgba(0,0,0,0)", bordercolor=GRID_COLOR),
    )
    return fig

# ── Custom CSS ──────────────────────────────────────────────────────────
st.markdown("""
<style>
@import url('https://fonts.googleapis.com/css2?family=Bebas+Neue&family=DM+Sar

:root {
    --bg:      #0a0c12; --surface: #111420; --card: #161b2e;
    --border:  #1e2640; --accent:  #e84545; --accent2: #f5a623;
    --accent3: #4ade80; --text:    #e8eaf0; --muted:   #7a82a0; --code-bg: #0c
}
html, body, .stApp { background-color: var(--bg) !important; color: var(--text
#MainMenu, footer, header { visibility: hidden; }
.block-container { padding: 1.5rem 2.5rem 3rem; max-width: 1300px; }
[data-testid="stSidebar"] { background: var(--surface) !important; border-righ
[data-testid="stSidebar"] * { color: var(--text) !important; }

.section-title { font-family: 'Bebas Neue', sans-serif; font-size: 2.8rem; let
.section-sub   { font-size: 0.85rem; color: var(--muted); letter-spacing: 2px;
.divider       { height: 2px; background: linear-gradient(90deg, var(--accent)

.nb-card { background: var(--card); border: 1px solid var(--border); border-ra
.nb-card:hover { border-color: #2e3a5c; }

.nb-code { background: var(--code-bg); border: 1px solid #1c2333; border-left:

.insight-box { background: linear-gradient(135deg, #111b2e 0%, #0f1620 100%);

.qa-question { background: #13182b; border: 1px solid #22305a; border-left: 4p
.qa-answer   { background: #0e1422; border: 1px solid var(--border); border-ra

.metric-pill { background: linear-gradient(135deg, #1e1035 0%, #0e1a2e 100%);
.metric-pill .mval { font-family: 'Bebas Neue'; font-size: 2.2rem; color: var(
.metric-pill .mlbl { font-size: 0.75rem; color: var(--muted); letter-spacing:

.tag { display: inline-block; background: #1c1040; border: 1px solid #3a2888;

h1 { font-family: 'Bebas Neue' !important; color: var(--accent) !important; le
h2 { font-family: 'Bebas Neue' !important; color: #c0c8e8 !important; letter-s
h3 { color: var(--accent2) !important; font-size: 1.1rem !important; }
.stMarkdown p  { color: var(--text); font-size: 0.92rem; line-height: 1.7; }
.stMarkdown li { color: #b0bcd8; font-size: 0.9rem; margin: 0.3rem 0; }
table { width: 100%; border-collapse: collapse; font-size: 0.85rem; }
th { background: #1a2040; color: var(--accent2); padding: 0.6rem 0.9rem; text-
```

```
td { padding: 0.5rem 0.9rem; border-bottom: 1px solid var(--border); color: #b
tr:hover td { background: #131826; }
</style>
""", unsafe_allow_html=True)


# ── Helpers ──────────────────────────────────────────────────────────
def section_header(title, subtitle=""):
    st.markdown(f"""
    <div style="margin-top:1.5rem">
        <div class="section-title">{title}</div>
        {"<div class='section-sub'>" + subtitle + "</div>" if subtitle else ""
        <div class="divider"></div>
    </div>""", unsafe_allow_html=True)

def card(html):
    st.markdown(f'<div class="nb-card">{html}</div>', unsafe_allow_html=True)

def code_block(code):
    esc = code.replace("&","&amp;").replace("<","&lt;").replace(">","&gt;")
    st.markdown(f'<div class="nb-code">{esc}</div>', unsafe_allow_html=True)

def insight(text):
    st.markdown(f'<div class="insight-box">&#128161; {text}</div>', unsafe_all

def qa(question, answer):
    st.markdown(f'<div class="qa-question">&#10067; {question}</div>', unsafe_
    st.markdown(f'<div class="qa-answer">&#9989; {answer}</div>', unsafe_allow


# ── Synthetic data matching notebook facts ──────────────────────────
@st.cache_data
def get_data():
    np.random.seed(42)

    genres = ["Drama","Comedy","Action","Thriller","Romance","Adventure",
              "Sci-Fi","Crime","Horror","Children's","Fantasy","Mystery",
              "War","Animation","Musical","Documentary","Western","Film-Noir"]
    genre_counts = pd.DataFrame({
        "Genre": genres,
        "Count": [1603,1200,952,623,580,510,480,395,340,295,240,205,
                  190,165,120,95,72,40]
    })

    years = list(range(1919, 2001))
    base  = np.zeros(len(years))
    for i, y in enumerate(years):
        if y < 1960:   base[i] = max(5,  int(np.random.normal(20, 8)))
        elif y < 1980: base[i] = max(10, int(np.random.normal(60, 15)))
        else:          base[i] = max(20, int(np.random.normal(120+(y-1980)*8,
    base[years.index(1996)] = 298
    base[years.index(1995)] = 285
    year_df = pd.DataFrame({"Year": years, "Count": base.astype(int)})
```

```
rating_df = pd.DataFrame({
    "Rating": [1, 2, 3, 4, 5],
    "Count":  [56174, 107557, 261197, 348971, 226310]
})

yr_ratings = pd.DataFrame({
    "Year":  [1996,1997,1998,1999,2000,2001,2002,2003],
    "Count": [3700,5200,8500,18900,652231,283456,89012,43219]
})

top_movies = pd.DataFrame({
    "Movie":     ["American Beauty","Star Wars: Ep IV","Schindler's List",
                  "Silence of the Lambs","Shawshank Redemption","Forrest G
                  "Jurassic Park","Pulp Fiction","Back to the Future","Toy
    "Ratings":   [3428,2991,2884,2583,2560,2194,2121,2056,1878,1820],
    "AvgRating": [4.31,4.45,4.47,4.29,4.55,4.01,3.74,4.15,3.92,3.88]
})

heat = np.zeros((7, 24))
for d in range(7):
    for h in range(24):
        if 18 <= h <= 23:   heat[d, h] = np.random.randint(8000, 14000)
        elif 6 <= h <= 17:  heat[d, h] = np.random.randint(1500, 5000)
        else:               heat[d, h] = np.random.randint(4000, 9000)
heatmap_df = pd.DataFrame(
    heat.astype(int),
    index=["Mon","Tue","Wed","Thu","Fri","Sat","Sun"],
    columns=list(range(24))
)

gender_df = pd.DataFrame({"Gender": ["Male","Female"], "Pct": [72, 28]})

age_df = pd.DataFrame({
    "AgeGroup": ["Under 18","18-24","25-34","35-44","45-49","50-55","56+"]
    "Count":    [222, 1103, 2096, 1193, 550, 496, 380]
})

occ_df = pd.DataFrame({
    "Occupation": ["college/grad student","other","executive/managerial",
                   "technician/engineer","programmer","educator","sales/ma
                   "writer","homemaker","artist","retired","self-employed"
                   "doctor/health care","tradesman/craftsman","scientist",
                   "clerical/admin","lawyer","customer service","farmer","
    "Count": [1649,937,726,595,543,528,395,368,300,297,284,278,237,226,215
              204,193,187,162,123]
})

zip_df = pd.DataFrame({
    "Zipcode": ["48104","22903","55104","11701","55343","94043","60657",
                "55414","02139","10003"],
    "Count":   [38,30,29,25,24,23,22,21,20,19]
})
```

```
    key_genres = ["Drama","Comedy","Action","Thriller","Romance","Adventure","
    gby_years  = list(range(1960, 2001))
    gby_data   = {}
    for g in key_genres:
        base_val = {"Drama":30,"Comedy":25,"Action":18,"Thriller":12,
                    "Romance":15,"Adventure":13,"Sci-Fi":10,"Crime":8}[g]
        vals = []
        for i, y in enumerate(gby_years):
            trend = base_val + i * np.random.uniform(0.3, 1.2)
            if g in ["Crime","Thriller"] and y >= 1993:
                trend *= 1.8
            vals.append(max(0, int(trend + np.random.normal(0, 3))))
        gby_data[g] = vals
    genre_year_df = pd.DataFrame(gby_data, index=gby_years)
    genre_year_df.index.name = "Year"

    model_df = pd.DataFrame({
        "Model":  ["Pearson CF","Cosine+KNN","SVD (Matrix Factorization)"],
        "RMSE":   [1.02, 0.96, 0.88],
        "MAPE":   [34.1, 30.5, 27.18],
    })

    n = 200
    pca_df = pd.DataFrame({
        "PC1":   np.random.randn(n) * 2,
        "PC2":   np.random.randn(n) * 1.5,
        "Genre": np.random.choice(["Drama","Comedy","Action","Sci-Fi","Thrille
                                   "Romance","Horror","Animation"], n)
    })

    clusters = {
        "Drama":    (3,  2),  "Comedy":   (-3,  3),
        "Action":   (2, -3),  "Sci-Fi":   (-2, -2),
        "Thriller": (0,  4),  "Romance":  (4,  -1),
        "Horror":   (-4, -1), "Animation":(1,   1),
    }
    tsne_rows = []
    for genre, (cx, cy) in clusters.items():
        for _ in range(25):
            tsne_rows.append({
                "D1": cx + np.random.randn() * 0.8,
                "D2": cy + np.random.randn() * 0.8,
                "Genre": genre
            })
    tsne_df = pd.DataFrame(tsne_rows)

    return (genre_counts, year_df, rating_df, yr_ratings, top_movies,
            heatmap_df, gender_df, age_df, occ_df, zip_df,
            genre_year_df, model_df, pca_df, tsne_df)


(genre_counts, year_df, rating_df, yr_ratings, top_movies,
```

```
 heatmap_df, gender_df, age_df, occ_df, zip_df,
 genre_year_df, model_df, pca_df, tsne_df) = get_data()


# ── Sidebar nav ──────────────────────────────────────────────────────
PAGES = [
    "🎬  Context",
    "📂  Know Your Data",
    "🔧  Data Cleaning / Feature Engg",
    "📊  Exploratory Data Analysis",
    "🤝  Collaborative Filtering",
    "📐  Cosine Similarity",
    "🧮  Matrix Factorization",
    "💡  Insights",
    "📈  Business Recommendations",
    "📝  Questionnaire",
]

with st.sidebar:
    st.markdown("""
    <div style="padding:1.2rem 0 0.5rem;">
        <div style="font-family:'Bebas Neue';font-size:1.6rem;color:#e84545;le
        <div style="font-size:0.72rem;color:#5a6280;letter-spacing:2px;text-tr
        <hr style="border-color:#1e2640;margin:0.8rem 0;">
    </div>""", unsafe_allow_html=True)
    page = st.radio("Navigate", PAGES, label_visibility="collapsed")
    st.markdown("""
    <hr style="border-color:#1e2640;margin:1.2rem 0 0.6rem;">
    <div style="font-size:0.72rem;color:#3a4060;padding-bottom:1rem;">Akul Vir
    """, unsafe_allow_html=True)


# ═══════════════════════════════════════════════════════════════════
# PAGE 1 — Context
# ═══════════════════════════════════════════════════════════════════
if page == PAGES[0]:
    section_header("Context", "Project Overview")
    card("""<p style="font-size:1.05rem;line-height:1.8;color:#c8d4f0;">
        Create a <strong style="color:#e84545;">Recommender System</strong> to
        recommendations based on ratings given by a user and other users simil
        <strong style="color:#f5a623;">improve user experience</strong>.
    </p>""")

    c1, c2, c3, c4 = st.columns(4)
    for col, val, lbl in zip([c1,c2,c3,c4],
                             ["1M+","3,883","6,040","3"],
                             ["Total Ratings","Movies","Users","Algorithms"]):
        with col:
            st.markdown(f'<div class="metric-pill"><div class="mval">{val}</di

    st.markdown("### Techniques Used")
    cols = st.columns(3)
    techs = [
```

```
        ("Pearson Correlation","Collaborative Filtering via normalized correla
        ("Cosine Similarity + KNN","Item & user similarity with k-nearest neig
        ("Matrix Factorization (SVD)","Latent factor model with PCA / t-SNE vi
    ]
    for col, (title, desc, clr) in zip(cols, techs):
        with col:
            st.markdown(f"""<div class="nb-card">
                <div style="font-family:'Bebas Neue';font-size:1.2rem;color:{c
                <p style="font-size:0.85rem;color:#8899bb;margin-top:0.5rem;">
            </div>""", unsafe_allow_html=True)

    fig = go.Figure()
    fig.add_trace(go.Bar(x=model_df["Model"], y=model_df["RMSE"], name="RMSE",
                         marker_color=ACCENT, text=model_df["RMSE"],
                         textposition="outside", texttemplate="%{text:.2f}"))
    fig.add_trace(go.Bar(x=model_df["Model"], y=[m/10 for m in model_df["MAPE"
                         name="MAPE / 10", marker_color=ACCENT2,
                         text=[f"{m}%" for m in model_df["MAPE"]],
                         textposition="outside"))
    fig.update_layout(barmode="group", title="Model Performance Comparison",
                      legend=dict(orientation="h", y=1.12))
    st.plotly_chart(apply_theme(fig), use_container_width=True)


# ═══════════════════════════════════════════════════════════════════════
# PAGE 2 — Know Your Data
# ═══════════════════════════════════════════════════════════════════════
elif page == PAGES[1]:
    section_header("Know Your Data", "Dataset Overview")

    code_block("""import numpy as np
import pandas as pd
from datetime import datetime
import warnings
warnings.filterwarnings('ignore')""")

    code_block("""movies  = pd.read_fwf('zee-movies.dat',  encoding='ISO-8859-
users   = pd.read_fwf('zee-users.dat',   encoding='ISO-8859-1')
ratings = pd.read_fwf('zee-ratings.dat', encoding='ISO-8859-1')""")

    tab1, tab2, tab3 = st.tabs(["🎥  Movies", "👤  Users", "⭐  Ratings"])

    with tab1:
        col1, col2 = st.columns([1, 1])
        with col1:
            st.markdown("**`movies.head()`**")
            st.markdown("""
| # | Movie ID::Title::Genres |
|---|---|
| 0 | 1::Toy Story (1995)::Animation\|Children's\|Comedy |
| 1 | 2::Jumanji (1995)::Adventure\|Children's\|Fantasy |
| 2 | 3::Grumpier Old Men (1995)::Comedy\|Romance |
| 3 | 4::Waiting to Exhale (1995)::Comedy\|Drama |
```

```
| 4 | 5::Father of the Bride Part II (1995)::Comedy |
""")
        with col2:
            fig = px.pie(genre_counts.head(8), names="Genre", values="Count",
                         title="Top 8 Genres Preview",
                         color_discrete_sequence=px.colors.sequential.Plasma_r
            fig.update_traces(textposition="inside", textinfo="percent+label")
            st.plotly_chart(apply_theme(fig, 340), use_container_width=True)

    with tab2:
        col1, col2 = st.columns([1,1])
        with col1:
            st.markdown("**`users.head()`**")
            st.markdown("""
| # | UserID::Gender::Age::Occupation::Zip-code |
|---|---|
| 0 | 1::F::1::10::48067 |
| 1 | 2::M::56::16::70072 |
| 2 | 3::M::25::15::55117 |
| 3 | 4::M::45::7::02460  |
| 4 | 5::M::25::20::55455 |
""")
        with col2:
            fig = px.bar(gender_df, x="Gender", y="Pct", text="Pct",
                         title="Gender Split Preview",
                         color="Gender",
                         color_discrete_map={"Male": ACCENT, "Female": ACCENT2
            fig.update_traces(texttemplate="%{text}%", textposition="outside")
            st.plotly_chart(apply_theme(fig, 340), use_container_width=True)

    with tab3:
        col1, col2 = st.columns([1,1])
        with col1:
            st.markdown("**`ratings.head()`**")
            st.markdown("""
| # | UserID::MovieID::Rating::Timestamp |
|---|---|
| 0 | 1::1193::5::978300760 |
| 1 | 1::661::3::978302109  |
| 2 | 1::914::3::978301968  |
| 3 | 1::3408::4::978300275 |
| 4 | 1::2355::5::978824291 |
""")
        with col2:
            fig = px.bar(rating_df, x="Rating", y="Count",
                         title="Rating Distribution Preview",
                         color="Count",
                         color_continuous_scale=[[0,"#1e2640"],[1,ACCENT]])
            fig.update_layout(coloraxis_showscale=False)
            st.plotly_chart(apply_theme(fig, 340), use_container_width=True)

        insight("3 datasets — Users, Movies, Ratings — form the foundation of
```

```
# ═══════════════════════════════════════════════════════════════════
# PAGE 3 — Data Cleaning
# ═══════════════════════════════════════════════════════════════════
elif page == PAGES[2]:
    section_header("Data Cleaning / Feature Engg", "Preprocessing Pipeline")

    st.markdown("## Movies Dataframe")
    code_block("""movies.drop(columns=['Unnamed: 1', 'Unnamed: 2'], axis=1, in
delimiter = '::'
movies = movies['Movie ID::Title::Genres'].str.split(delimiter, expand=True)
movies.columns = ['MovieID', 'Title', 'Genres']

movies['MovieName']   = movies['Title'].str.extract(r'^(.*)\\s\\((\\d{4})\\)$'
movies['ReleaseYear'] = movies['Title'].str.extract(r'^(.*)\\s\\((\\d{4})\\)$'

    insight("Segregate Movie Name and Year of Release from the Title column. S

    st.markdown("### Genre Cleaning")
    code_block("""official_genres = ["Action","Adventure","Animation","Childre
    "Documentary","Drama","Fantasy","Film-Noir","Horror","Musical",
    "Mystery","Romance","Sci-Fi","Thriller","War","Western"]

genre_mapping = {"Comdy":"Comedy","Come":"Comedy","Childr":"Children's",...}

def clean_genre(genre):
    if genre in official_genres:   return genre
    elif genre in genre_mapping:   return genre_mapping[genre]
    else:                          return "Unknown" """)

    raw_genres   = ["Drama","Comdy","Comedy","Action","Childr","Thriller","Com
    fixed_genres = ["Drama","Comedy","Comedy","Action","Children's","Thriller"
    fix_df = pd.DataFrame({"Raw": raw_genres, "Cleaned": fixed_genres,
                           "Changed": [r!=c for r,c in zip(raw_genres,fixed_g
    fig = go.Figure(data=[go.Table(
        header=dict(values=["Raw Genre","Cleaned Genre","Status"],
                    fill_color="#1a2040", font=dict(color=ACCENT2, size=13),
                    align="left", height=36),
        cells=dict(
            values=[fix_df["Raw"], fix_df["Cleaned"],
                    ["Fixed" if c else "OK" for c in fix_df["Changed"]]],
            fill_color=[["#0d1117"]*8, ["#0d1117"]*8,
                        [("#1a3020" if c else "#0d1117") for c in fix_df["Char
            font=dict(color="#c9d1d9", size=12), align="left", height=32)
    )])
    fig.update_layout(title="Genre Name Cleanup Preview", paper_bgcolor=PAPER_
                      font=dict(color="#c8d4f0"), height=320, margin=dict(l=0,
    st.plotly_chart(fig, use_container_width=True)

    insight("Improper genre names like 'Childr' → \"Children's\" are corrected

    code_block("""movies_df = movies.pivot_table(
    index=['MovieID', 'MovieName', 'ReleaseYear'],
```

```
    columns='Genre', values='Title',
    aggfunc='count', fill_value=0
)""")

    st.markdown("---")
    st.markdown("## Ratings Dataframe")
    code_block("""ratings_df['Rating']    = ratings_df['Rating'].astype(int)
ratings_df['Timestamp'] = pd.to_datetime(ratings_df['Timestamp'], unit='s')
ratings_df['Year']      = ratings_df['Timestamp'].dt.year
ratings_df['Month']     = ratings_df['Timestamp'].dt.month
ratings_df['DayOfWeek'] = ratings_df['Timestamp'].dt.dayofweek
ratings_df['Hour']      = ratings_df['Timestamp'].dt.hour""")

    col1, col2 = st.columns(2)
    with col1:
        st.markdown("#### Aggregate by User")
        code_block("""user_stats = ratings_df.groupby('UserID')['Rating']
    .agg(['mean','count']).reset_index()
user_stats.rename(columns={
    'mean':  'AvgRatingPerUser',
    'count': 'TotalRatingsPerUser'}, inplace=True)""")
    with col2:
        st.markdown("#### Aggregate by Movie")
        code_block("""movie_stats = ratings_df.groupby('MovieID')['Rating']
    .agg(['mean','count']).reset_index()
movie_stats.rename(columns={
    'mean':  'AvgRatingPerMovie',
    'count': 'TotalRatingsPerMovie'}, inplace=True)""")

    st.markdown("---")
    st.markdown("## Merging all 3 Dataframes")
    code_block("""ratings_movies = pd.merge(ratings_df, movies_df, on='MovieID
merged_df      = pd.merge(ratings_movies, users_df, on='UserID', how='inner')"


# ═══════════════════════════════════════════════════════════════════
# PAGE 4 — EDA
# ═══════════════════════════════════════════════════════════════════
elif page == PAGES[3]:
    section_header("Exploratory Data Analysis", "Visual Insights")

    tab_movies, tab_ratings, tab_users = st.tabs(["🎥  Movies", "⭐  Ratings",

    # ── MOVIES ──
    with tab_movies:
        st.markdown("### Movies per Genre")
        code_block("""genre_counts = movies_df.iloc[:, 3:].sum().sort_values(a
genre_counts.plot(kind='bar', color='skyblue')""")

        top_n = st.slider("Show top N genres", 5, 18, 18, key="genre_top_n")
        filtered = genre_counts.sort_values("Count", ascending=False).head(top
        fig = px.bar(filtered, x="Genre", y="Count",
                     color="Count",
```

```
                    color_continuous_scale=[[0,"#1a2a4a"],[0.5,ACCENT2],[1,AC
                    title=f"Top {top_n} Genres by Movie Count", text="Count")
        fig.update_traces(textposition="outside")
        fig.update_layout(coloraxis_showscale=False)
        st.plotly_chart(apply_theme(fig), use_container_width=True)
        insight("Highest number of movies — nearly 1,600 — belong to Drama, fo

        st.markdown("### Year Wise Trend")
        code_block("""year_counts = movies_df['ReleaseYear'].value_counts().so
plt.plot(year_counts.index, year_counts.values, marker='o')""")

        yr_min, yr_max = st.select_slider("Year range", options=list(range(191
                                          value=(1960,2000), key="yr_range")
        yr_filtered = year_df[(year_df["Year"]>=yr_min) & (year_df["Year"]<=yr
        fig = px.line(yr_filtered, x="Year", y="Count", markers=True,
                      title="Movies Released Over Time",
                      color_discrete_sequence=[ACCENT2])
        fig.update_traces(line_width=2, marker_size=5)
        fig.add_vline(x=1995, line_dash="dash", line_color=ACCENT,
                      annotation_text="1995 peak", annotation_position="top ri
        fig.add_vline(x=1996, line_dash="dash", line_color=ACCENT3,
                      annotation_text="1996 peak", annotation_position="top le
        st.plotly_chart(apply_theme(fig), use_container_width=True)
        insight("Release years span 1919–2000. Maximum movies were released in

        st.markdown("### Genre Popularity by Year")
        code_block("""genre_by_year.iloc[:, 2:].plot(kind='area', stacked=True
plt.title('Genre Popularity Over Years')""")

        sel_genres = st.multiselect("Select genres",
                                    genre_year_df.columns.tolist(),
                                    default=["Drama","Comedy","Action","Thrill
                                    key="genre_sel")
        if sel_genres:
            plot_df = genre_year_df[sel_genres].reset_index().melt(
                id_vars="Year", var_name="Genre", value_name="Count")
            fig = px.area(plot_df, x="Year", y="Count", color="Genre",
                          title="Genre Popularity Over Years (Stacked Area)")
            st.plotly_chart(apply_theme(fig, 460), use_container_width=True)
        insight("Action, Comedy, Drama grew over years. Crime and Thriller sur

    # ── RATINGS ──
    with tab_ratings:
        st.markdown("### Distribution of Ratings")
        code_block("""sns.countplot(data=ratings_df, x='Rating', palette='viri

        fig = px.bar(rating_df, x="Rating", y="Count",
                     color="Rating",
                     color_continuous_scale=[[0,"#1a2a4a"],[0.5,ACCENT2],[1,AC
                     title="Distribution of Ratings", text="Count")
        fig.update_traces(texttemplate="%{text:,}", textposition="outside")
        fig.update_layout(coloraxis_showscale=False)
        st.plotly_chart(apply_theme(fig), use_container_width=True)
```

```
        insight("Rating 4 is the most common, followed by 3 and 5.")

        st.markdown("### Ratings Over Time")
        code_block("""yearly_ratings = ratings_df.groupby('Year')['Rating'].co
plt.plot(yearly_ratings.index, yearly_ratings.values, marker='o')""")

        fig = px.bar(yr_ratings, x="Year", y="Count",
                     title="Number of Ratings Per Year",
                     color="Count",
                     color_continuous_scale=[[0,"#1a2a4a"],[1,ACCENT2]],
                     text="Count")
        fig.update_traces(texttemplate="%{text:,}", textposition="outside")
        fig.add_annotation(x=2000, y=652231, text="Peak 2000", showarrow=True,
                           arrowhead=2, arrowcolor=ACCENT, font=dict(color=ACC
        fig.update_layout(coloraxis_showscale=False)
        st.plotly_chart(apply_theme(fig), use_container_width=True)
        insight("Peak rating activity in year 2000 (~652k). Sharply dropped ir

        st.markdown("### Top 10 Most Rated Movies")
        code_block("""top_movies = movie_stats.nlargest(10, 'TotalRatingsPerMo
sns.barplot(data=top_movies, x='TotalRatingsPerMovie', y='MovieID')""")

        sort_by = st.radio("Sort by", ["Ratings","Avg Rating"], horizontal=Tru
        sort_col = "Ratings" if sort_by == "Ratings" else "AvgRating"
        sorted_movies = top_movies.sort_values(sort_col, ascending=True)
        fig = px.bar(sorted_movies, y="Movie", x=sort_col, orientation="h",
                     color=sort_col,
                     color_continuous_scale=[[0,"#1a2a4a"],[1,ACCENT]],
                     title=f"Top 10 Movies by {sort_by}", text=sort_col)
        fig.update_traces(texttemplate="%{text:,.2f}", textposition="outside")
        fig.update_layout(coloraxis_showscale=False)
        st.plotly_chart(apply_theme(fig, 460), use_container_width=True)
        insight("American Beauty is the most rated movie; Shawshank Redemptior

        st.markdown("### Ratings Heatmap — Hour x Day")
        code_block("""heatmap_data = ratings_df.groupby(['DayOfWeek','Hour'])[
sns.heatmap(heatmap_data, cmap='coolwarm')""")

        fig = go.Figure(data=go.Heatmap(
            z=heatmap_df.values,
            x=[f"{h:02d}:00" for h in range(24)],
            y=heatmap_df.index.tolist(),
            colorscale=[[0,"#0a0c12"],[0.4,"#1a2a4a"],[0.7,ACCENT2],[1,ACCENT]
            hovertemplate="Day: %{y}<br>Hour: %{x}<br>Ratings: %{z:,}<extra></
        ))
        fig.update_layout(title="Heatmap: Ratings by Day & Hour",
                          xaxis_title="Hour of Day", yaxis_title="Day of Week"
        st.plotly_chart(apply_theme(fig, 380), use_container_width=True)
        insight("Peak activity: evenings (18:00–23:00). Quieter mid-day hours

    # ── USERS ──
    with tab_users:
        col1, col2 = st.columns(2)
```

```
        with col1:
            st.markdown("### Gender Distribution")
            code_block("""plt.pie(gender_counts, labels=gender_counts.index, a
            fig = px.pie(gender_df, names="Gender", values="Pct",
                         title="Gender Proportion",
                         color="Gender",
                         color_discrete_map={"Male": ACCENT, "Female": ACCENT2
                         hole=0.45)
            fig.update_traces(textinfo="percent+label", textfont_size=14, pull
            st.plotly_chart(apply_theme(fig, 360), use_container_width=True)
            insight("Males 72%, Females 28%.")

        with col2:
            st.markdown("### Age Distribution")
            code_block("""sns.barplot(x=age_counts.index, y=age_counts.values,
            fig = px.bar(age_df, x="AgeGroup", y="Count",
                         title="Users by Age Group",
                         color="Count",
                         color_continuous_scale=[[0,"#1a2a4a"],[1,ACCENT2]],
                         text="Count")
            fig.update_traces(textposition="outside")
            fig.update_layout(coloraxis_showscale=False)
            st.plotly_chart(apply_theme(fig, 360), use_container_width=True)
            insight("Age 25–34 is the largest group (2,096 users).")

        st.markdown("### Occupation Distribution")
        code_block("""sns.barplot(x=occupation_counts.index, y=occupation_cour
plt.xticks(rotation=90)""")

        show_top = st.slider("Show top N occupations", 5, 20, 15, key="occ_top
        occ_filtered = occ_df.head(show_top).sort_values("Count")
        fig = px.bar(occ_filtered, y="Occupation", x="Count", orientation="h",
                     color="Count",
                     color_continuous_scale=[[0,"#1a2a4a"],[0.5,ACCENT2],[1,AC
                     title=f"Top {show_top} Occupations", text="Count")
        fig.update_traces(textposition="outside")
        fig.update_layout(coloraxis_showscale=False)
        st.plotly_chart(apply_theme(fig, 480), use_container_width=True)
        insight("'College/grad student' is the dominant occupation, followed b

        st.markdown("### Top 10 Zip-codes")
        code_block("""top_zipcodes = users_df['Zip-code'].value_counts().head(
sns.barplot(x=top_zipcodes.index, y=top_zipcodes.values)""")
        fig = px.bar(zip_df, x="Zipcode", y="Count",
                     color="Count",
                     color_continuous_scale=[[0,"#1a2a4a"],[1,ACCENT3]],
                     title="Top 10 Zip-codes", text="Count")
        fig.update_traces(textposition="outside")
        fig.update_layout(coloraxis_showscale=False)
        st.plotly_chart(apply_theme(fig), use_container_width=True)
        insight("Most users from zip-code 48104. Four of the top 10 start with
```

```
# ═══════════════════════════════════════════════════════════════
# PAGE 5 — Collaborative Filtering
# ═══════════════════════════════════════════════════════════════
elif page == PAGES[4]:
    section_header("Collaborative Filtering", "Pearson Correlation Method")

    code_block("""pivot_table = ratings_df.pivot_table(index='MovieID', columr
                                    values='Rating', fill_value=0)
movie_id_to_name = movies_df[['MovieID','MovieName']].set_index('MovieID')
pivot_table      = pivot_table.join(movie_id_to_name, how='left').set_index('M
user_item_matrix = pivot_table.copy()""")

    st.markdown("## Handling Sparsity")
    code_block("""total_possible = user_item_matrix.shape[0] * user_item_matri
non_zero       = user_item_matrix[user_item_matrix > 0].count().sum()
sparsity       = 1 - (non_zero / total_possible)
print(f"Sparsity: {sparsity*100:.2f}%")   # ~95.5%

user_item_matrix = user_item_matrix.loc[(user_item_matrix>0).sum(axis=1) >= 10
user_item_matrix = user_item_matrix.loc[:, (user_item_matrix>0).sum() >= 50]""

    fig = go.Figure(go.Indicator(
        mode="gauge+number",
        value=95.53,
        title={"text": "Matrix Sparsity (%)", "font": {"color": "#c8d4f0", "si
        gauge={
            "axis":  {"range": [0, 100], "tickcolor": "#5a6280"},
            "bar":   {"color": ACCENT},
            "bgcolor": "#111420",
            "steps":[{"range":[0,60],"color":"#0d2a1a"},
                     {"range":[60,85],"color":"#2a1a0a"},
                     {"range":[85,100],"color":"#2a0a0a"}],
            "threshold":{"line":{"color":ACCENT2,"width":3},"thickness":0.75,"
        },
        number={"suffix":"%","font":{"color":ACCENT2,"size":34}}
    ))
    fig.update_layout(paper_bgcolor=PAPER_COLOR, plot_bgcolor=BG_COLOR,
                      font=dict(color="#c8d4f0"), height=300,
                      margin=dict(l=30,r=30,t=60,b=20))
    st.plotly_chart(fig, use_container_width=True)
    insight("Data is ~95.5% sparse. Applying min-rating thresholds reduces noi

    st.markdown("## Correlation Matrix")
    code_block("""normalized_matrix  = user_item_matrix.sub(user_item_matrix.m
correlation_matrix = normalized_matrix.T.corr(method='pearson')""")

    sample_movies = ["Toy Story","Aladdin","The Lion King","Jurassic Park",
                     "Home Alone","Mrs. Doubtfire","Speed","Forrest Gump"]
    np.random.seed(7)
    corr_base = np.random.uniform(0.1, 0.9, (8, 8))
    corr_sym  = (corr_base + corr_base.T) / 2
    np.fill_diagonal(corr_sym, 1.0)
    fig = go.Figure(data=go.Heatmap(
```

```
        z=corr_sym, x=sample_movies, y=sample_movies,
        colorscale=[[0,"#0a0c12"],[0.3,"#1a2a4a"],[0.7,ACCENT2],[1,ACCENT3]],
        zmin=-1, zmax=1,
        text=np.round(corr_sym,2), texttemplate="%{text}",
        hovertemplate="%{x} vs %{y}: %{z:.2f}<extra></extra>"
    ))
    fig.update_layout(title="Sample Pearson Correlation Matrix (8 movies)",
                      paper_bgcolor=PAPER_COLOR, plot_bgcolor=BG_COLOR,
                      font=dict(color="#c8d4f0"), height=400,
                      margin=dict(l=10,r=10,t=50,b=10))
    st.plotly_chart(fig, use_container_width=True)

    st.markdown("## Recommendation Function")
    code_block("""def recommend_movies(movie_name, correlation_matrix, top_n=5
    if movie_name not in correlation_matrix.index:
        return f"'{movie_name}' not found."
    similar = (correlation_matrix[movie_name]
               .drop(movie_name)
               .sort_values(ascending=False)
               .head(top_n))
    return similar""")


# ═══════════════════════════════════════════════════════════════════
# PAGE 6 — Cosine Similarity
# ═══════════════════════════════════════════════════════════════════
elif page == PAGES[5]:
    section_header("Cosine Similarity", "KNN-Based Recommender")

    code_block("""from sklearn.metrics.pairwise import cosine_similarity
from sklearn.neighbors import NearestNeighbors

user_item_dense = user_item_matrix.values

user_similarity    = cosine_similarity(user_item_dense.T)
user_similarity_df = pd.DataFrame(user_similarity,
    index=user_item_matrix.columns, columns=user_item_matrix.columns)

item_similarity    = cosine_similarity(user_item_dense)
item_similarity_df = pd.DataFrame(item_similarity,
    index=user_item_matrix.index, columns=user_item_matrix.index)

model_knn = NearestNeighbors(metric='cosine', algorithm='brute')
model_knn.fit(user_item_dense)""")

    movies_cs = ["Toy Story","Aladdin","Lion King","Speed","Terminator",
                 "Pulp Fiction","Home Alone","Mrs. Doubtfire"]
    np.random.seed(12)
    sim_data = np.random.uniform(0.2, 0.95, (8,8))
    sim_data = (sim_data + sim_data.T) / 2
    np.fill_diagonal(sim_data, 1.0)
    for i in range(3):
        for j in range(3):
```

```
            if i != j:
                sim_data[i,j] = np.random.uniform(0.7, 0.95)

    col1, col2 = st.columns([3,2])
    with col1:
        fig = go.Figure(data=go.Heatmap(
            z=sim_data, x=movies_cs, y=movies_cs,
            colorscale=[[0,"#0a0c12"],[0.4,"#1a3a6a"],[0.7,ACCENT2],[1,ACCENT3
            zmin=0, zmax=1,
            text=np.round(sim_data,2), texttemplate="%{text}",
            hovertemplate="%{x} vs %{y}: %{z:.2f}<extra></extra>"
        ))
        fig.update_layout(title="Item Cosine Similarity Matrix (Sample)",
                          paper_bgcolor=PAPER_COLOR, font=dict(color="#c8d4f0"
                          height=400, margin=dict(l=10,r=10,t=50,b=10))
        st.plotly_chart(fig, use_container_width=True)

    with col2:
        st.markdown("### Try It — KNN Recommendations")
        recs_data = {
            "Toy Story":     ["Aladdin","The Lion King","A Bug's Life","Toy St
            "Liar Liar":     ["Mrs. Doubtfire","Ace Ventura","Dumb & Dumber","
            "Jurassic Park": ["Speed","Terminator 2","Die Hard","Aliens","Pred
        }
        selected = st.selectbox("Movie", list(recs_data.keys()), key="cos_movi
        np.random.seed(abs(hash(selected)) % 100)
        for i, m in enumerate(recs_data[selected], 1):
            sim_score = round(np.random.uniform(0.72, 0.94), 2)
            st.markdown(f"""<div style="display:flex;justify-content:space-bet
                align-items:center;padding:0.45rem 0.8rem;margin:0.3rem 0;
                background:#0e1422;border:1px solid #1e2640;border-radius:6px;
                <span style="color:#c8d4f0;font-size:0.88rem;">#{i} {m}</span>
                <span style="color:{ACCENT2};font-family:'JetBrains Mono';font
            </div>""", unsafe_allow_html=True)

    code_block("""def recommend_movies(movie_name, user_item_matrix, item_simi
                      model_knn, n_neighbors=5):
    movie_idx    = user_item_matrix.index.get_loc(movie_name)
    movie_vector = user_item_dense[movie_idx].reshape(1, -1)
    distances, indices = model_knn.kneighbors(movie_vector, n_neighbors=n_neig
    return [user_item_matrix.index[idx] for idx in indices.flatten()[1:]]""")


# ═══════════════════════════════════════════════════════════════════════
# PAGE 7 — Matrix Factorization
# ═══════════════════════════════════════════════════════════════════════
elif page == PAGES[6]:
    section_header("Matrix Factorization", "SVD — Latent Factor Model")

    code_block("""from surprise import SVD, Dataset, Reader
from surprise.model_selection import train_test_split
from surprise import accuracy
```

```
reader   = Reader(rating_scale=(0, 5))
data     = Dataset.load_from_df(ratings_df[['UserID','MovieID','Rating']], rea
trainset, testset = train_test_split(data, test_size=0.2)

svd_model = SVD(n_factors=4)
svd_model.fit(trainset)
predictions = svd_model.test(testset)""")

    st.markdown("## Evaluation — RMSE / MAPE")
    col1, col2, col3 = st.columns(3)
    with col1:
        st.markdown('<div class="metric-pill"><div class="mval">0.8813</div><d
    with col2:
        st.markdown('<div class="metric-pill"><div class="mval">27.18%</div><d
    with col3:
        st.markdown('<div class="metric-pill"><div class="mval">4</div><div cl

    insight("RMSE 0.8813 → predictions deviate ~0.88 rating units. MAPE 27.18%

    fig = make_subplots(rows=1, cols=2,
                        subplot_titles=["RMSE by Model (lower = better)",
                                        "MAPE % by Model (lower = better)"])
    colors = [ACCENT2, "#a888ff", ACCENT3]
    for col_name, col_idx in [("RMSE",1),("MAPE",2)]:
        fig.add_trace(go.Bar(
            x=model_df["Model"], y=model_df[col_name],
            marker_color=colors, text=model_df[col_name],
            textposition="outside", showlegend=False,
            texttemplate="%{text:.2f}"
        ), row=1, col=col_idx)
    fig.update_layout(paper_bgcolor=PAPER_COLOR, plot_bgcolor=BG_COLOR,
                      font=dict(color="#c8d4f0"), height=380,
                      margin=dict(l=30,r=30,t=60,b=30))
    fig.update_xaxes(tickangle=-12)
    st.plotly_chart(fig, use_container_width=True)

    st.markdown("## Embeddings — PCA & t-SNE")
    code_block("""from sklearn.decomposition import PCA
from sklearn.manifold import TSNE

movie_embeddings = svd_model.qi   # shape: (n_movies, n_factors)
pca_2d  = PCA(n_components=2).fit_transform(movie_embeddings)
tsne_2d = TSNE(n_components=2, perplexity=30, n_iter=300).fit_transform(movie_

    emb_tab1, emb_tab2 = st.tabs(["PCA Embeddings", "t-SNE Embeddings"])
    with emb_tab1:
        fig = px.scatter(pca_df, x="PC1", y="PC2", color="Genre",
                         title="Movie Embeddings — PCA (2D)",
                         color_discrete_sequence=px.colors.qualitative.Bold,
                         hover_data=["Genre"], opacity=0.8)
        fig.update_traces(marker_size=7)
        st.plotly_chart(apply_theme(fig, 460), use_container_width=True)
        insight("PCA clusters movies broadly by genre-level patterns in user r
```

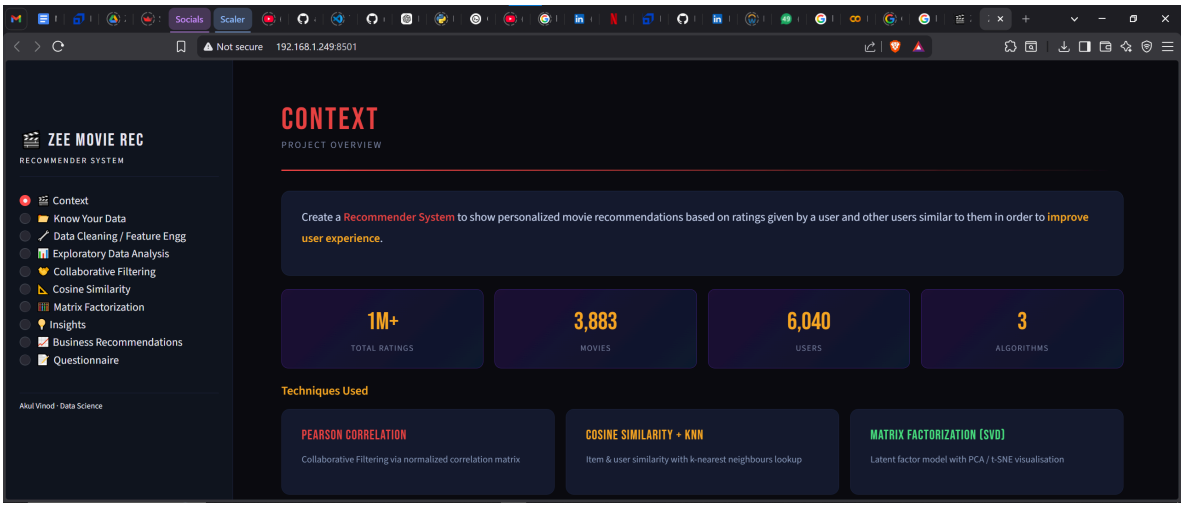
```
    with emb_tab2:
        fig = px.scatter(tsne_df, x="D1", y="D2", color="Genre",
                         title="Movie Embeddings — t-SNE (2D)",
                         color_discrete_sequence=px.colors.qualitative.Vivid,
                         hover_data=["Genre"], opacity=0.85)
        fig.update_traces(marker_size=8)
        st.plotly_chart(apply_theme(fig, 460), use_container_width=True)
        insight("t-SNE reveals tighter local clusters per genre — more interpr


# ═══════════════════════════════════════════════════════════════════════
# PAGE 8 — Insights
# ═══════════════════════════════════════════════════════════════════════
elif page == PAGES[7]:
    section_header("Insights", "Key Findings")

    all_insights = [
        ("Movies", [
            "Highest number of movies — nearly 1,600 — belong to Drama, follow
            "Release years span 1919–2000. Maximum movies released in 1996 and
            "Action, Comedy, Drama have grown steadily over the decades.",
            "Western and Animation proportions remained relatively constant.",
            "Crime and Thriller showed a sharp surge around 1995.",
        ]),
        ("Ratings", [
            "Rating 4 is the most common, followed by 3 then 5.",
            "Peak rating activity was in 2000 (~652k ratings), dropping sharpl
            "Most ratings occur during evening/late-night hours (18:00–23:00).
            "Fewer ratings between 06:00 and 17:00.",
        ]),
        ("Users", [
            "Males dominate at 72%; Females at 28%.",
            "Largest age cohort: 25–34 (2,096 users).",
            "Top occupation: 'college/grad student', followed by 'other' and '
            "Most users from zip-code 48104; four of the top 10 start with 55.
            "Largest demographic: Male, 18–24, college/grad student (~400 user
        ]),
        ("Models", [
            "Three algorithms: Pearson CF, Cosine+KNN, SVD.",
            "SVD achieved the best RMSE (0.8813) and MAPE (27.18%).",
            "PCA and t-SNE embeddings from SVD's latent space reveal genre clu
        ]),
    ]

    icons = {"Movies":"🎬","Ratings":"⭐","Users":"👥","Models":"🤖"}
    for category, items in all_insights:
        st.markdown(f"### {icons[category]} {category}")
        for item in items:
            insight(item)

    st.markdown("### Model Performance Summary")
    fig = go.Figure()
```

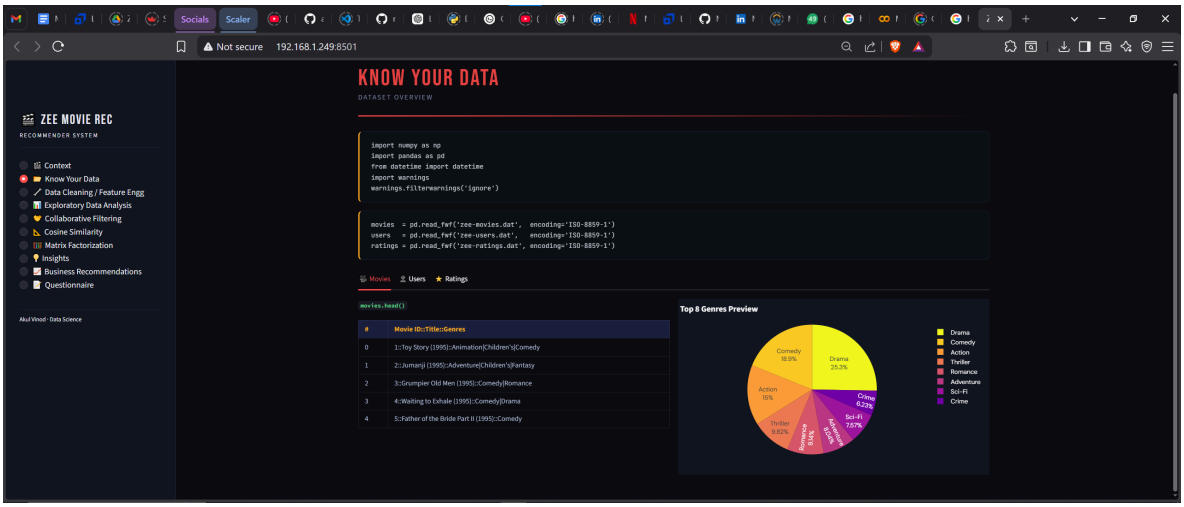
```
    fig.add_trace(go.Scatterpolar(
        r=[1/0.88, 1/0.96, 1/1.02, 10/27.18, 10/30.5, 10/34.1],
        theta=["SVD RMSE","KNN RMSE","Pearson RMSE","SVD MAPE","KNN MAPE","Pea
        fill="toself", name="Inverse score (higher = better)",
        line_color=ACCENT, fillcolor="rgba(232,69,69,0.15)"
    ))
    fig.update_layout(polar=dict(
        radialaxis=dict(visible=True, range=[0,2], gridcolor=GRID_COLOR),
        bgcolor=BG_COLOR
    ), paper_bgcolor=PAPER_COLOR, font=dict(color="#c8d4f0"),
       height=420, margin=dict(l=60,r=60,t=40,b=40))
    st.plotly_chart(fig, use_container_width=True)


# ═══════════════════════════════════════════════════════════════════════
# PAGE 9 — Business Recommendations
# ═══════════════════════════════════════════════════════════════════════
elif page == PAGES[8]:
    section_header("Business Recommendations", "Strategic Insights")

    recs = [
        ("Enhancing User Engagement", "🎯", [
            "**Personalized Recommendations:** Tailor suggestions based on use
            "**Diverse Suggestions:** Balance popular highly-rated movies with
        ]),
        ("Targeted Content Strategy", "🎬", [
            "**Genre Focus:** Drama, Comedy, Action dominate. Invest in Film-N
            "**Temporal Targeting:** Schedule releases and promotions during p
        ]),
        ("Demographic-Driven Marketing", "👥", [
            "**Age 25–34:** Largest user group. Tailor UI/UX and algorithm wei
            "**College Students:** Dominant occupation group — offer student d
            "**Female Users (28%):** Focus on content that diversifies engagem
        ]),
        ("Model Improvement", "🤖", [
            "**Reduce Sparsity:** Encourage more ratings via gamification (bad
            "**Hybrid Approach:** Combine content-based + collaborative filter
            "**A/B Test Algorithms:** Experiment between Pearson, KNN, and SVD
        ]),
    ]

    for i, (title, icon, points) in enumerate(recs):
        with st.expander(f"{icon}  {i+1}. {title}", expanded=(i==0)):
            for p in points:
                st.markdown(f"- {p}")

    st.markdown("### Strategic Priority Matrix")
    priority_df = pd.DataFrame({
        "Initiative":  ["Personalization Engine","Sparsity Reduction","Hybrid
                        "Demographic Marketing","Temporal Promotions","A/B Tes
        "Impact":      [9, 8, 7, 6, 5, 7],
        "Effort":      [7, 4, 8, 3, 2, 5],
        "Category":    ["Model","Data","Model","Marketing","Marketing","Model"
```

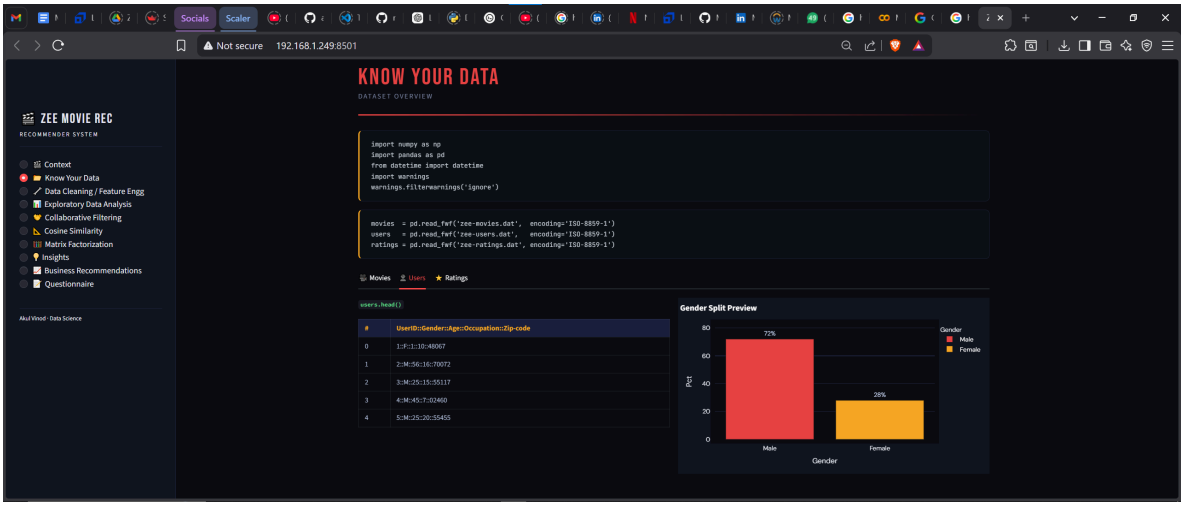
```
    })
    fig = px.scatter(priority_df, x="Effort", y="Impact",
                     text="Initiative", color="Category", size=[40]*6,
                     title="Initiative: Impact vs Effort",
                     color_discrete_sequence=[ACCENT, ACCENT2, ACCENT3, "#a888
    fig.update_traces(textposition="top center", marker_opacity=0.85)
    fig.add_hline(y=6.5, line_dash="dash", line_color=GRID_COLOR)
    fig.add_vline(x=5.5, line_dash="dash", line_color=GRID_COLOR)
    fig.add_annotation(x=2.5, y=9.6, text="Quick Wins", showarrow=False,
                       font=dict(color=ACCENT3, size=11))
    fig.add_annotation(x=7.5, y=9.6, text="Big Bets", showarrow=False,
                       font=dict(color=ACCENT2, size=11))
    st.plotly_chart(apply_theme(fig, 440), use_container_width=True)


# ═══════════════════════════════════════════════════════════════════════
# PAGE 10 — Questionnaire
# ═══════════════════════════════════════════════════════════════════════
elif page == PAGES[9]:
    section_header("Questionnaire", "Assessment & Answers")

    qa("1. Users of which age group have watched and rated the most number of
       "More than 2000 users belong to age group **25–34** who have watched ar
    qa("2. Users belonging to which profession have watched and rated the most
       "From the EDA plot, the answer is **'College/grad students'**.")
    qa("3. Most of the users in our dataset who've rated the movies are Male (
       "**True** – 72% of users are Males.")
    qa("4. Most movies were released in which decade?  a. 70s  b. 90s  c. 50s
       "Maximum movies released in 1995 and 1996. Answer: **b. 90s**.")
    qa("5. The movie with maximum no. of ratings is ___.",
       "**American Beauty** is the movie with the maximum number of ratings.")

    code_block("""max_ratings_movie  = movie_stats.loc[movie_stats['TotalRatir
movie_with_max     = movies_df.merge(
    movie_stats.loc[[movie_stats['TotalRatingsPerMovie'].idxmax()]], on='Movie
print(f"Movie Name: {movie_with_max['MovieName'].values[0]}")""")

    qa("6. Name the top 3 movies similar to 'Liar Liar' on the item-based appr
       "1. Mrs. Doubtfire\n2. Ace Ventura: Pet Detective\n3. Dumb & Dumber")

    code_block("""recommendations = recommend_movies("Liar Liar", user_item_ma
                                item_similarity_df, model_knn, n_neighbors=
print(recommendations)""")

    qa("7. Collaborative Filtering methods can be classified into ___-based ar
       "**User-based** and **item-based**.")
    qa("8. Pearson Correlation ranges from ___ to ___; Cosine Similarity from
       "**Pearson:** −1 to +1.\n\n**Cosine Similarity:** 0 to 1 (for non-negat
    qa("9. Mention the RMSE and MAPE from the Matrix Factorization model.",
       "**RMSE = 0.8813** and **MAPE = 27.18%**.")
    qa("10. Sparse 'row' representation for [[1 0] / [3 7]]:",
       "**(row_ptr, col_indices, values) = ([0, 1, 3], [0, 0, 1], [1, 3, 7])**
```

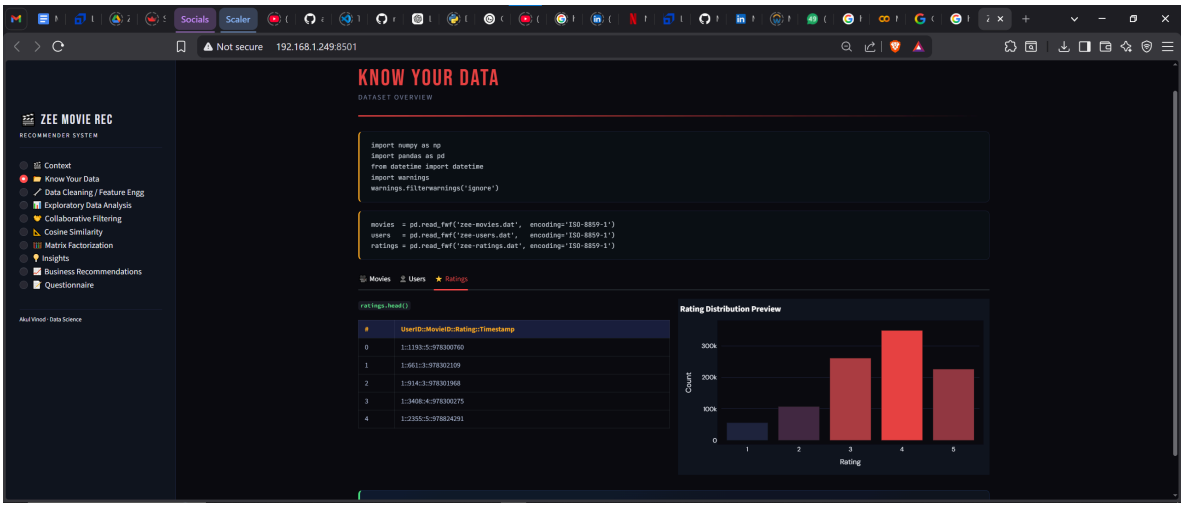
```
    st.markdown("### Question Difficulty Map")
    q_labels    = [f"Q{i}" for i in range(1,11)]
    difficulty  = [2,2,1,2,3,4,2,4,3,5]
    diff_labels = ["Easy" if d<=2 else "Medium" if d<=3 else "Hard" for d in d
    diff_colors = [ACCENT3 if d<=2 else ACCENT2 if d<=3 else ACCENT for d in d
    fig = go.Figure(go.Bar(
        x=q_labels, y=difficulty,
        marker_color=diff_colors,
        text=diff_labels, textposition="outside"
    ))
    fig.update_layout(yaxis_range=[0,6.5], title="Question Difficulty (1=Easy,
    st.plotly_chart(apply_theme(fig, 340), use_container_width=True)

    st.markdown("""
    <div style="text-align:center;padding:2rem 0;color:#3a4060;font-size:0.8re
         letter-spacing:1px;font-family:'JetBrains Mono';">
        Akul Vinod · DATA SCIENCE · MOVIE RECOMMENDER SYSTEM
    </div>""", unsafe_allow_html=True)
```

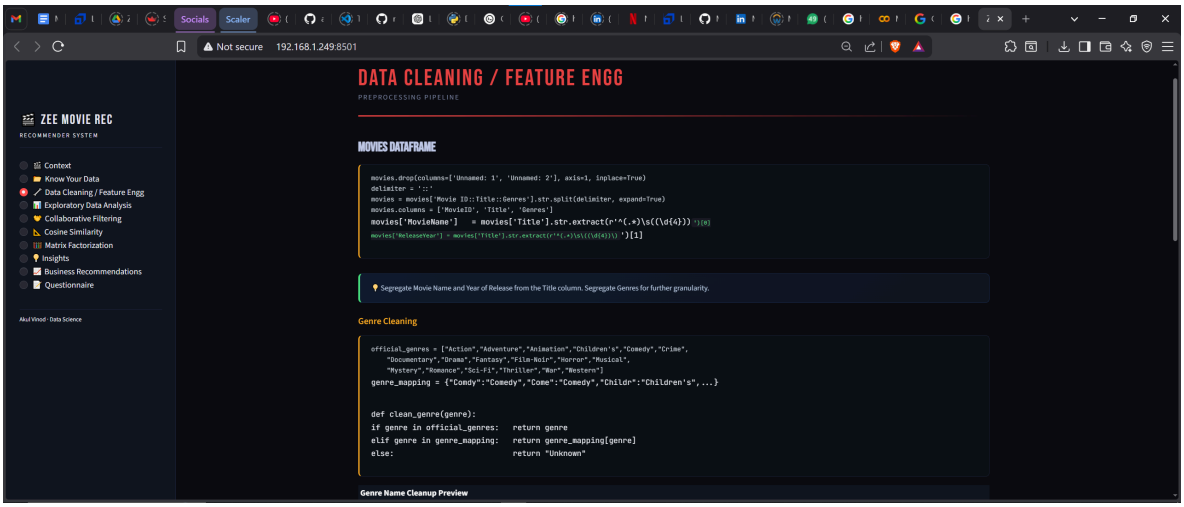
streamlit run movie_recommender_System.py

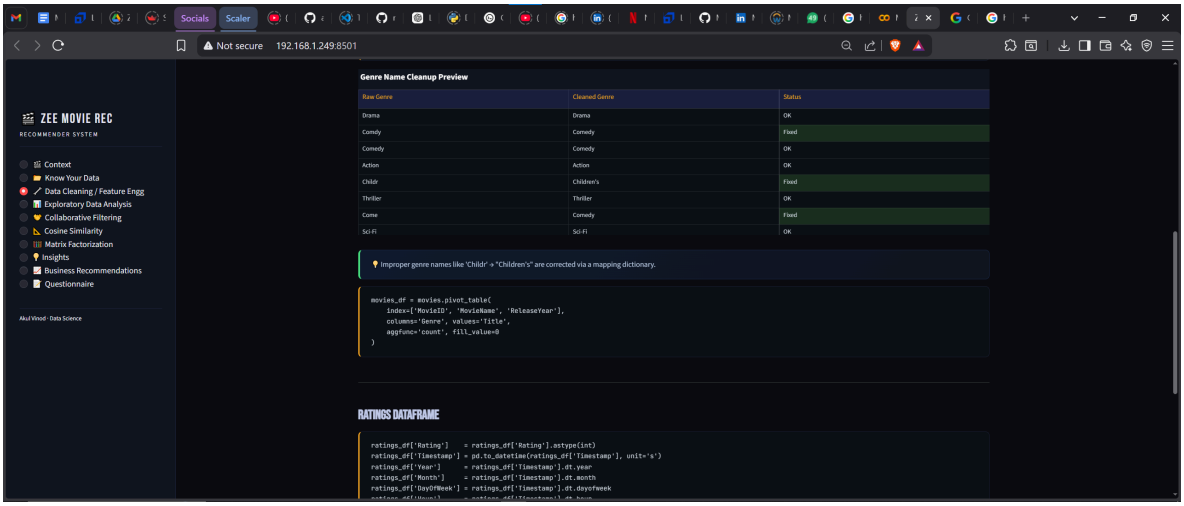


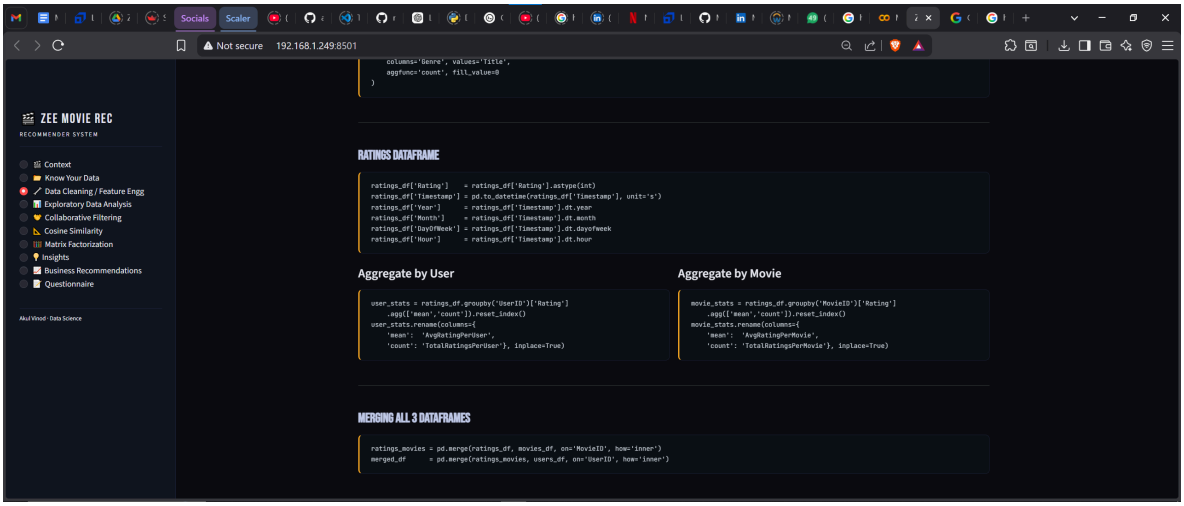

Socials
Scaler
Not secure 192.168.1.249:8501
ZEE MOVIE REC
RECOMMENDER SYSTEM
Context
Know Your Data
Data Cleaning / Feature Engg
Exploratory Data Analysis
Collaborative Filtering
Cosine Similarity
Matrix Factorization
Insights
Business Recommendations
Questionnaire
Akul Vinod - Data Science
KNOW YOUR DATA
DATASET OVERVIEW
import numpy as np
import pandas as pd
from datetime import datetime
import warnings
warnings.filterwarnings('ignore')
movies = pd.read_fwf('zee-movies.dat', encoding='ISO-8859-1')
users = pd.read_fwf('zee-users.dat', encoding='ISO-8859-1')
ratings = pd.read_fwf('zee-ratings.dat', encoding='ISO-8859-1')
Movies
Users
Ratings
movies.head()
# Movie ID::Title::Genres
0 1::Toy Story (1995)::Animation|Children's|Comedy
1 2::Jumanji (1995)::Adventure|Children's|Fantasy
2 3::Grumpier Old Men (1995)::Comedy|Romance
3 4::Waiting to Exhale (1995)::Comedy|Drama
4 5::Father of the Bride Part II (1995)::Comedy
Top 8 Genres Preview
Comedy 18.9%
Drama 25.3%
Action 15%
Thriller 9.82%
Romance 8.76%
Adventure 8.04%
Sci-Fi 7.57%
Crime 6.23%
Drama
Comedy
Action
Thriller
Romance
Adventure
Sci-Fi
Crime


Socials
Scaler
Not secure 192.168.1.249:8501
ZEE MOVIE REC
RECOMMENDER SYSTEM
Context
Know Your Data
Data Cleaning / Feature Engg
Exploratory Data Analysis
Collaborative Filtering
Cosine Similarity
Matrix Factorization
Insights
Business Recommendations
Questionnaire
Akul Vinod - Data Science
KNOW YOUR DATA
DATASET OVERVIEW
import numpy as np
import pandas as pd
from datetime import datetime
import warnings
warnings.filterwarnings('ignore')
movies = pd.read_fwf('zee-movies.dat', encoding='ISO-8859-1')
users = pd.read_fwf('zee-users.dat', encoding='ISO-8859-1')
ratings = pd.read_fwf('zee-ratings.dat', encoding='ISO-8859-1')
Movies
Users
Ratings
users.head()
# UserID::Gender::Age::Occupation::Zip-code
0 1::F::1::10::48067
1 2::M::56::16::70072
2 3::M::25::15::55117
3 4::M::45::7::02460
4 5::M::25::20::55455
Gender Split Preview
72%
28%
Male
Female
Gender
Pct


Socials
Scaler
Not secure 192.168.1.249:8501
ZEE MOVIE REC
RECOMMENDER SYSTEM
Context
Know Your Data
Data Cleaning / Feature Engg
Exploratory Data Analysis
Collaborative Filtering
Cosine Similarity
Matrix Factorization
Insights
Business Recommendations
Questionnaire
Akul Vinod - Data Science
KNOW YOUR DATA
DATASET OVERVIEW
import numpy as np
import pandas as pd
from datetime import datetime
import warnings
warnings.filterwarnings('ignore')
movies = pd.read_fwf('zee-movies.dat', encoding='ISO-8859-1')
users = pd.read_fwf('zee-users.dat', encoding='ISO-8859-1')
ratings = pd.read_fwf('zee-ratings.dat', encoding='ISO-8859-1')
Movies
Users
Ratings
ratings.head()
# UserID::MovieID::Rating::Timestamp
0 1::1193::5::978300760
1 1::661::3::978302109
2 1::914::3::978301968
3 1::3408::4::978300275
4 1::2355::5::978824291
Rating Distribution Preview
300k
200k
100k
0
Count
1
2
3
4
5
Rating


Not secure 192.168.1.249:8501
ZEE MOVIE REC
RECOMMENDER SYSTEM
Context
Know Your Data
Data Cleaning / Feature Engg
Exploratory Data Analysis
Collaborative Filtering
Cosine Similarity
Matrix Factorization
Insights
Business Recommendations
Questionnaire
Akul Vinod · Data Science
DATA CLEANING / FEATURE ENGG
PREPROCESSING PIPELINE
MOVIES DATAFRAME
movies.drop(columns=['Unnamed: 1', 'Unnamed: 2'], axis=1, inplace=True)
delimiter = '::'
movies = movies['Movie ID::Title::Genres'].str.split(delimiter, expand=True)
movies.columns = ['MovieID', 'Title', 'Genres']
movies['MovieName'] = movies['Title'].str.extract(r'^(.+)\s(\d{4}))
movies['ReleaseYear'] = movies['Title'].str.extract(r'^(.+)\s\((\d{4})\)')[1]
Segregate Movie Name and Year of Release from the Title column. Segregate Genres for further granularity.
Genre Cleaning
official_genres = ["Action","Adventure","Animation","Children's","Comedy","Crime",
"Documentary","Drama","Fantasy","Film-Noir","Horror","Musical",
"Mystery","Romance","Sci-Fi","Thriller","War","Western"]
genre_mapping = {"Condy":"Comedy","Come":"Comedy","Childr":"Children's",...}
def clean_genre(genre):
if genre in official_genres: return genre
elif genre in genre_mapping: return genre_mapping[genre]
else: return "Unknown"
Genre Name Cleanup Preview

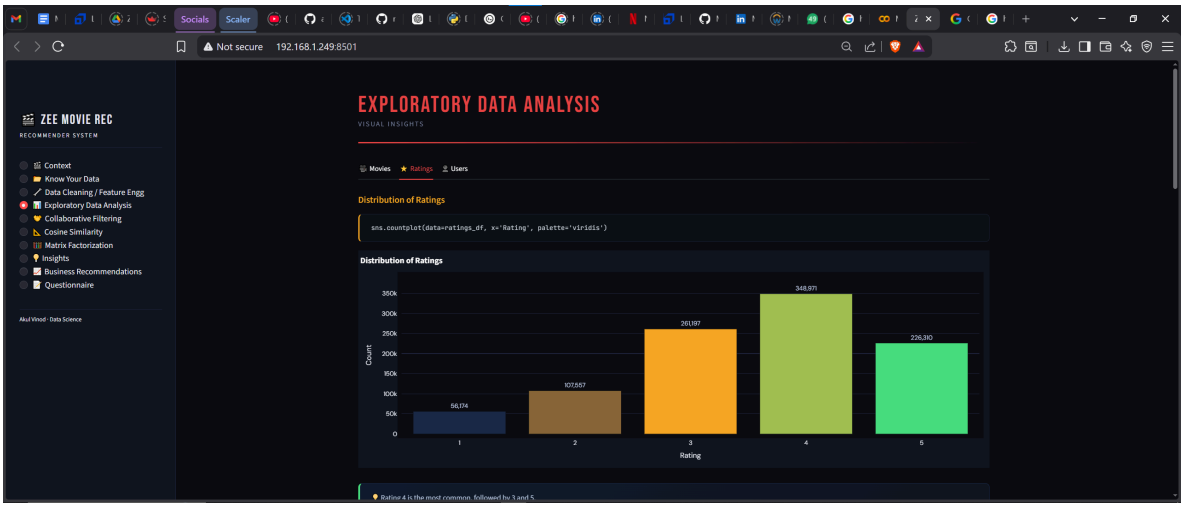

Not secure 192.168.1.249:8501
ZEE MOVIE REC
RECOMMENDER SYSTEM
Context
Know Your Data
Data Cleaning / Feature Engg
Exploratory Data Analysis
Collaborative Filtering
Cosine Similarity
Matrix Factorization
Insights
Business Recommendations
Questionnaire
Akul Vinod · Data Science
Genre Name Cleanup Preview
Raw Genre
Cleaned Genre
Status
Drama
Drama
OK
Comdy
Comedy
Fixed
Comedy
Comedy
OK
Action
Action
OK
Childr
Children's
Fixed
Thriller
Thriller
OK
Come
Comedy
Fixed
Sci-Fi
Sci-Fi
OK
Improper genre names like 'Childr' → "Children's" are corrected via a mapping dictionary.
movies_df = movies.pivot_table(
index=['MovieID', 'MovieName', 'ReleaseYear'],
columns='Genre', values='Title',
aggfunc='count', fill_value=0
)
RATINGS DATAFRAME
ratings_df['Rating'] = ratings_df['Rating'].astype(int)
ratings_df['Timestamp'] = pd.to_datetime(ratings_df['Timestamp'], unit='s')
ratings_df['Year'] = ratings_df['Timestamp'].dt.year
ratings_df['Month'] = ratings_df['Timestamp'].dt.month
ratings_df['DayOfWeek'] = ratings_df['Timestamp'].dt.dayofweek


Not secure 192.168.1.249:8501
ZEE MOVIE REC
RECOMMENDER SYSTEM
Context
Know Your Data
Data Cleaning / Feature Engg
Exploratory Data Analysis
Collaborative Filtering
Cosine Similarity
Matrix Factorization
Insights
Business Recommendations
Questionnaire
Akul Vinod · Data Science
columns='Genre', values='Title',
aggfunc='count', fill_value=0
)
RATINGS DATAFRAME
ratings_df['Rating'] = ratings_df['Rating'].astype(int)
ratings_df['Timestamp'] = pd.to_datetime(ratings_df['Timestamp'], unit='s')
ratings_df['Year'] = ratings_df['Timestamp'].dt.year
ratings_df['Month'] = ratings_df['Timestamp'].dt.month
ratings_df['DayOfWeek'] = ratings_df['Timestamp'].dt.dayofweek
ratings_df['Hour'] = ratings_df['Timestamp'].dt.hour
Aggregate by User
user_stats = ratings_df.groupby('UserID')['Rating']
.agg(['mean','count']).reset_index()
user_stats.rename(columns={
'mean': 'AvgRatingPerUser',
'count': 'TotalRatingsPerUser'}, inplace=True)
Aggregate by Movie
movie_stats = ratings_df.groupby('MovieID')['Rating']
.agg(['mean','count']).reset_index()
movie_stats.rename(columns={
'mean': 'AvgRatingPerMovie',
'count': 'TotalRatingsPerMovie'}, inplace=True)
MERGING ALL 3 DATAFRAMES
ratings_movies = pd.merge(ratings_df, movies_df, on='MovieID', how='inner')
merged_df = pd.merge(ratings_movies, users_df, on='UserID', how='inner')


Not secure 192.168.1.249:8501
ZEE MOVIE REC
RECOMMENDER SYSTEM
Context
Know Your Data
Data Cleaning / Feature Engg
Exploratory Data Analysis
Collaborative Filtering
Cosine Similarity
Matrix Factorization
Insights
Business Recommendations
Questionnaire
Akul Vinod - Data Science
EXPLORATORY DATA ANALYSIS
VISUAL INSIGHTS
Movies
Ratings
Users
Movies per Genre
genre_counts = movies_df.iloc[:, 3:].sum().sort_values(ascending=False)
genre_counts.plot(kind='bar', color='skyblue')
Show top N genres
18
Top 18 Genres by Movie Count
Count
1603
1200
952
623
580
510
480
395
340
296
240
205
190
165
120
95
72
40
Drama
Comedy
Action
Thriller
Romance
Adventure
Sci-Fi
Crime
Horror
Children's
Fantasy
Mystery
War
Animation
Musical
Documentary
Western
Film-Noir

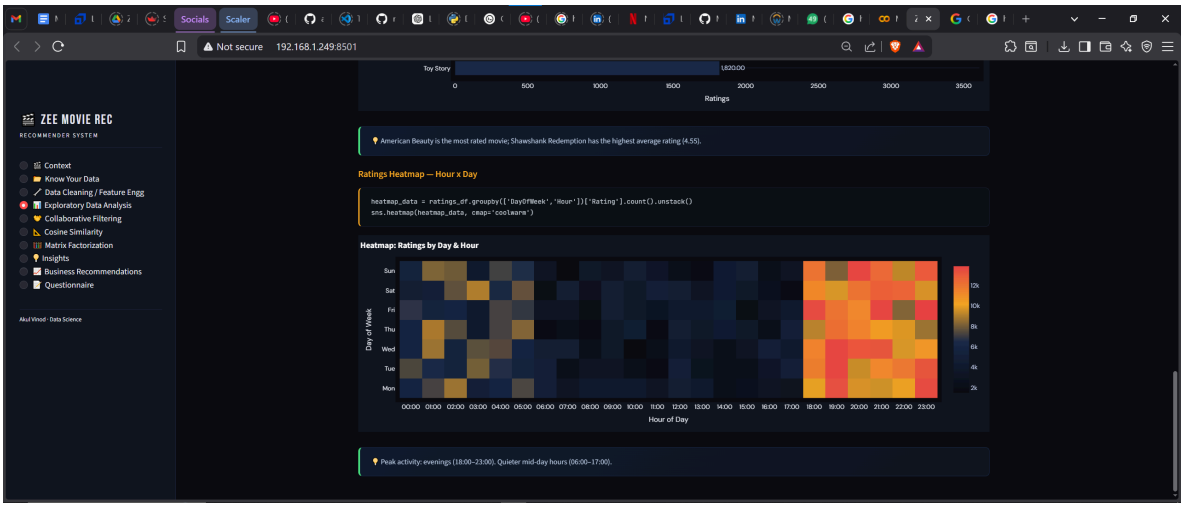

Not secure 192.168.1.249:8501
Genre
Highest number of movies — nearly 1,600 — belong to Drama, followed by Comedy and then Action.
Year Wise Trend
year_counts = movies_df['ReleaseYear'].value_counts().sort_index()
plt.plot(year_counts.index, year_counts.values, marker='o')
Year range
1960
2000
Movies Released Over Time
1996 peak
Count
Year


Not secure 192.168.1.249:8501
Release years span 1919–2000. Maximum movies were released in 1996 and 1995.
Genre Popularity by Year
genre_by_year.iloc[:, 2:].plot(kind='area', stacked=True, colormap='tab20')
plt.title('Genre Popularity Over Years')
Select genres
Drama
Comedy
Action
Thriller
Genre Popularity Over Years (Stacked Area)
Genre
Drama
Comedy
Action
Thriller
Count
Year
Action, Comedy, Drama grew over years. Crime and Thriller surged sharply around 1995.


Not secure 192.168.1.249:8501
ZEE MOVIE REC
RECOMMENDER SYSTEM
Context
Know Your Data
Data Cleaning / Feature Engg
Exploratory Data Analysis
Collaborative Filtering
Cosine Similarity
Matrix Factorization
Insights
Business Recommendations
Questionnaire
Akul Vinod - Data Science
EXPLORATORY DATA ANALYSIS
VISUAL INSIGHTS
Movies
Ratings
Users
Distribution of Ratings
sns.countplot(data=ratings_df, x='Rating', palette='viridis')
Distribution of Ratings
Count
Rating
56,174
107,557
261,197
348,971
226,310
Rating 4 is the most common, followed by 3 and 5.

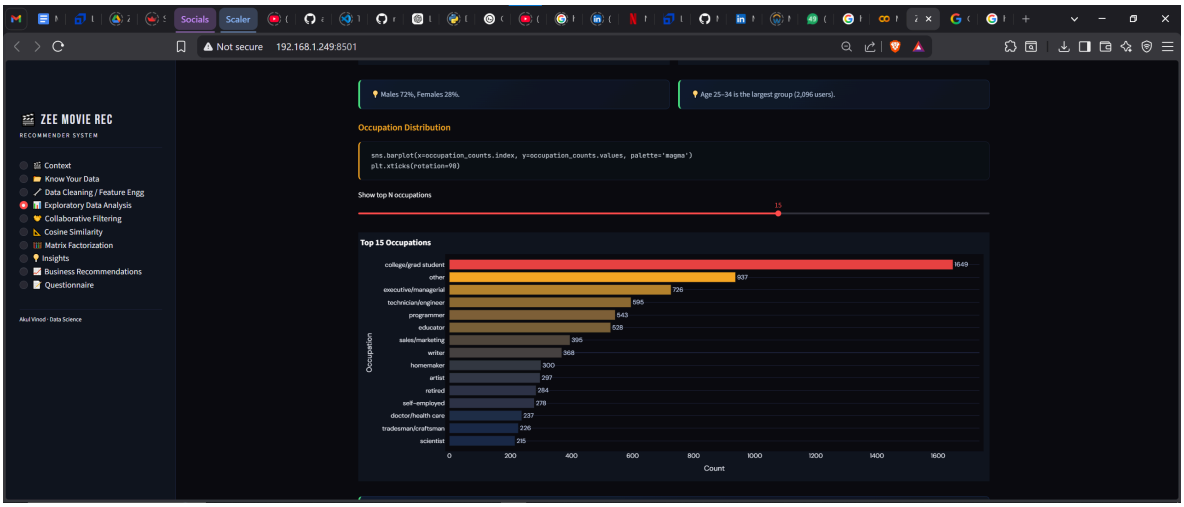

Not secure 192.168.1.249:8501
ZEE MOVIE REC
RECOMMENDER SYSTEM
Akul Vinod - Data Science
Rating 4 is the most common, followed by 3 and 5.
Ratings Over Time
yearly_ratings = ratings_df.groupby('Year')['Rating'].count()
plt.plot(yearly_ratings.index, yearly_ratings.values, marker='o')
Number of Ratings Per Year
Count
Year
Peak 2000
652,731
3,700
5,200
8,500
18,900
283,456
89,012
43,219
Peak rating activity in year 2000 (~652k). Sharply dropped in 2001.
Top 10 Most Rated Movies
top_movies = movie_stats.nlargest(10, 'TotalRatingsPerMovie')
sns.barplot(data=top_movies, x='TotalRatingsPerMovie', y='MovieID')

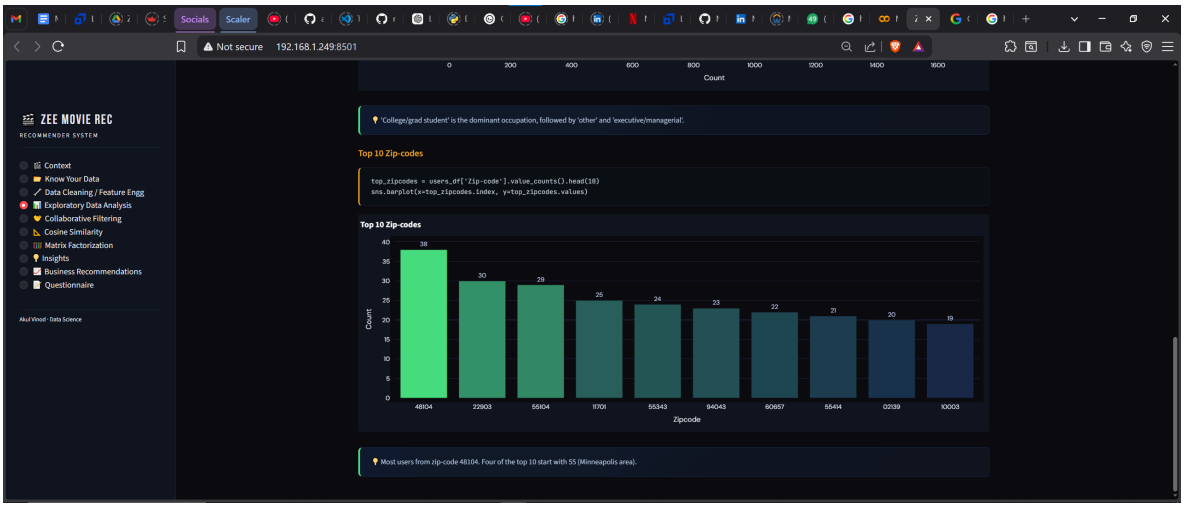

Not secure 192.168.1.249:8501
ZEE MOVIE REC
RECOMMENDER SYSTEM
Akul Vinod - Data Science
Peak rating activity in year 2000 (~652k). Sharply dropped in 2001.
Top 10 Most Rated Movies
top_movies = movie_stats.nlargest(10, 'TotalRatingsPerMovie')
sns.barplot(data=top_movies, x='TotalRatingsPerMovie', y='MovieID')
Sort by
Ratings
Avg Rating
Top 10 Movies by Ratings
Movie
Ratings
American Beauty
Star Wars: Ep IV
Schindler's List
Silence of the Lambs
Shawshank Redemption
Forrest Gump
Jurassic Park
Pulp Fiction
Back to the Future
Toy Story
3,428.00
2,991.00
2,884.00
2,583.00
2,560.00
2,194.00
2,121.00
2,056.00
1,878.00
1,820.00
American Beauty is the most rated movie; Shawshank Redemption has the highest average rating (4.55).

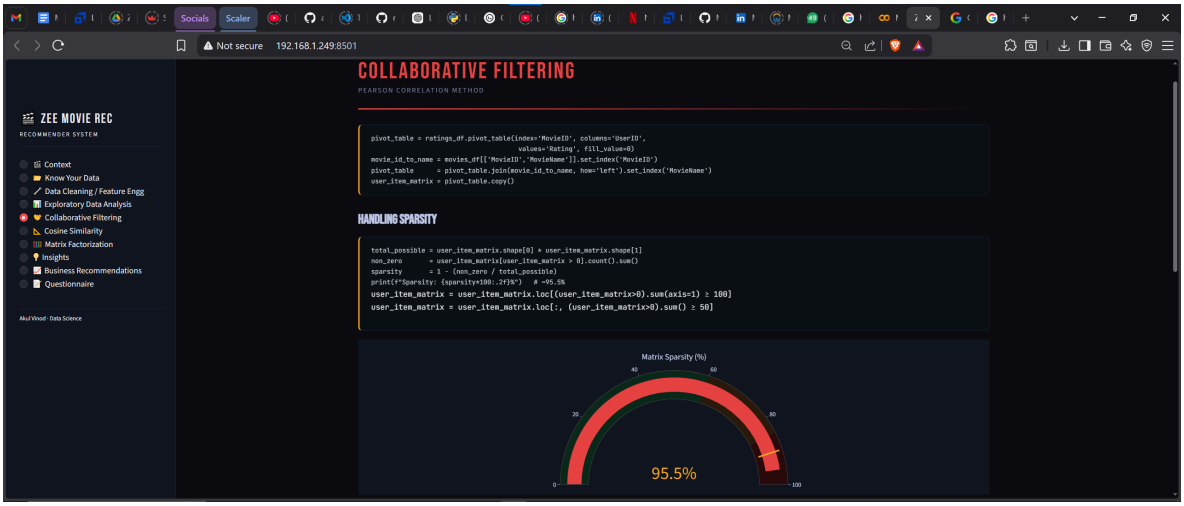

Not secure 192.168.1.249:8501
ZEE MOVIE REC
RECOMMENDER SYSTEM
Context
Know Your Data
Data Cleaning / Feature Engg
Exploratory Data Analysis
Collaborative Filtering
Cosine Similarity
Matrix Factorization
Insights
Business Recommendations
Questionnaire
Akul Vinod · Data Science
Toy Story
Ratings
American Beauty is the most rated movie; Shawshank Redemption has the highest average rating (4.55).
Ratings Heatmap — Hour x Day
heatmap_data = ratings_df.groupby(['DayOfWeek','Hour'])['Rating'].count().unstack()
sns.heatmap(heatmap_data, cmap='coolwarm')
Heatmap: Ratings by Day & Hour
Day of Week
Hour of Day
Peak activity: evenings (18:00–23:00). Quieter mid-day hours (06:00–17:00).

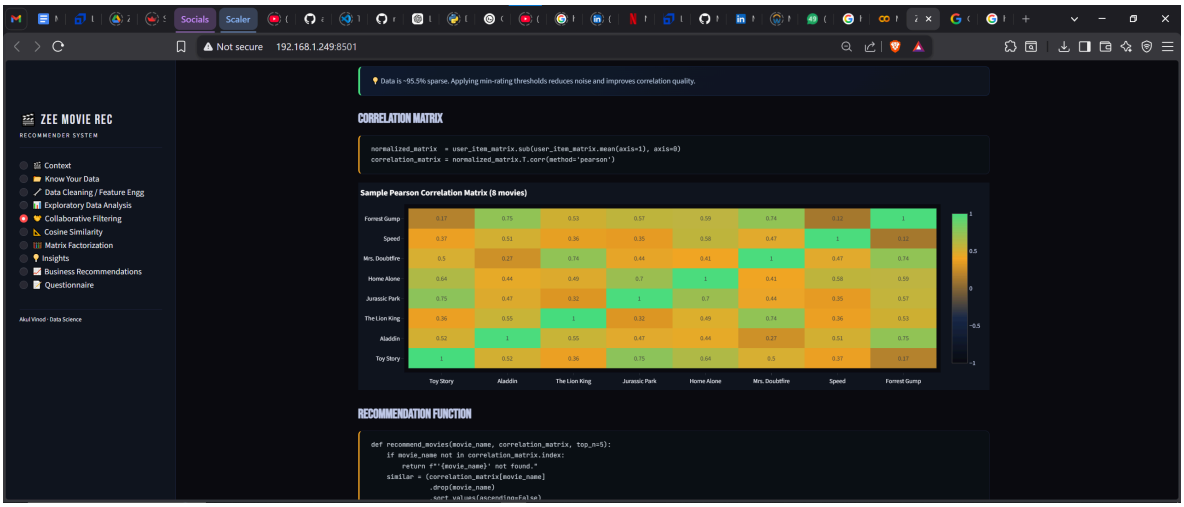

Not secure 192.168.1.249:8501
EXPLORATORY DATA ANALYSIS
VISUAL INSIGHTS
ZEE MOVIE REC
RECOMMENDER SYSTEM
Movies
Ratings
Users
Gender Distribution
plt.pie(gender_counts, labels=gender_counts.index, autopct='%1.1f%%')
Age Distribution
sns.barplot(x=age_counts.index, y=age_counts.values, palette='coolwarm')
Gender Proportion
Female 28%
Male 72%
Users by Age Group
Count
AgeGroup
Under 18
18-24
25-34
35-44
45-49
50-55
56+
Males 72%, Females 28%.
Age 25–34 is the largest group (2,096 users).
Occupation Distribution

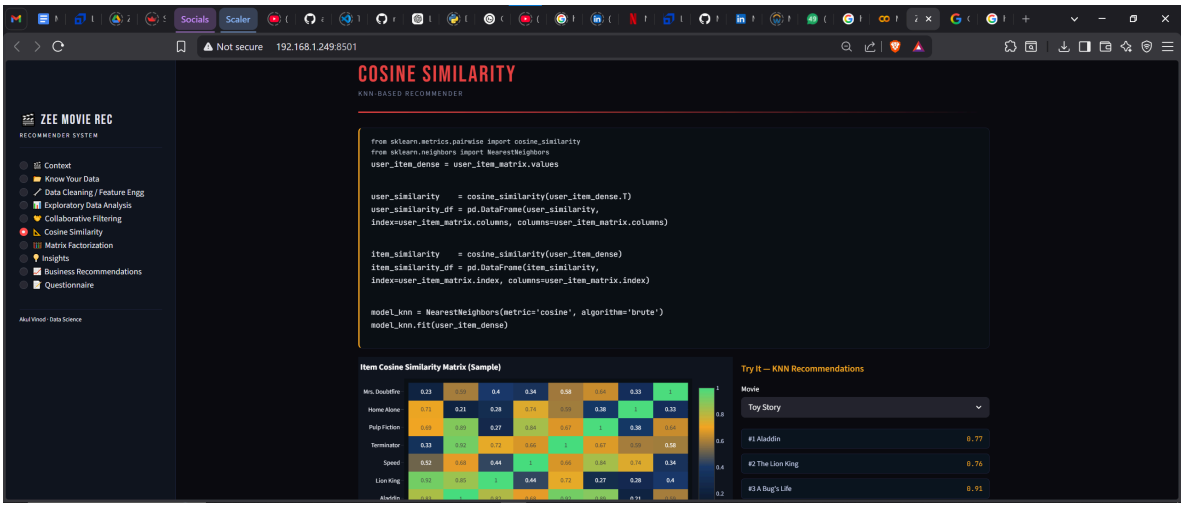
In [ ]:

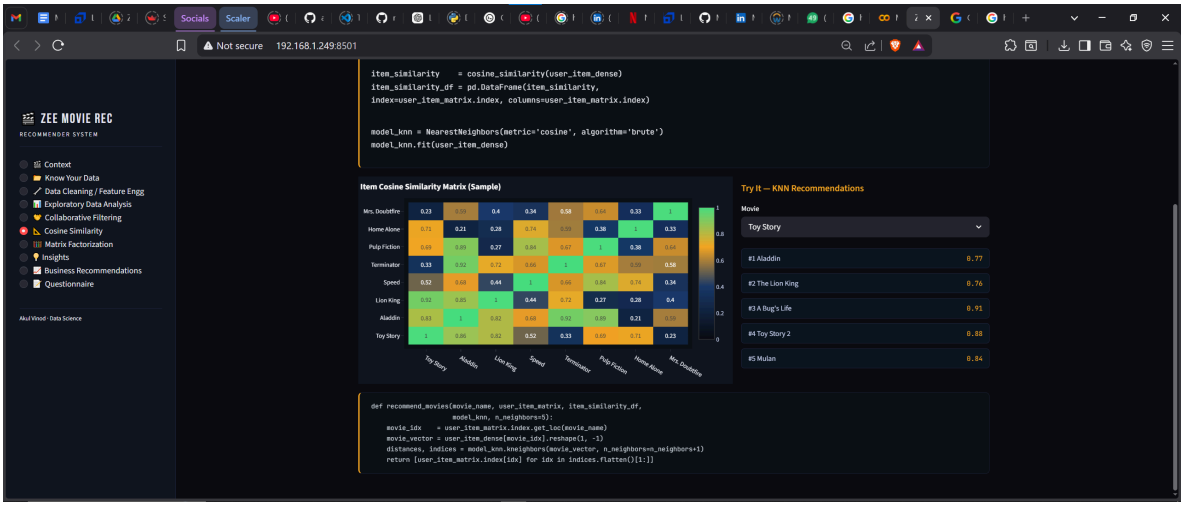

Not secure 192.168.1.249:8501
ZEE MOVIE REC
RECOMMENDER SYSTEM
Males 72%, Females 28%.
Age 25–34 is the largest group (2,096 users).
Occupation Distribution
sns.barplot(x=occupation_counts.index, y=occupation_counts.values, palette='magma')
plt.xticks(rotation=90)
Show top N occupations
Top 15 Occupations
college/grad student
other
executive/managerial
technician/engineer
programmer
educator
sales/marketing
writer
homemaker
artist
retired
self-employed
doctor/health care
tradesman/craftsman
scientist
Occupation
Count

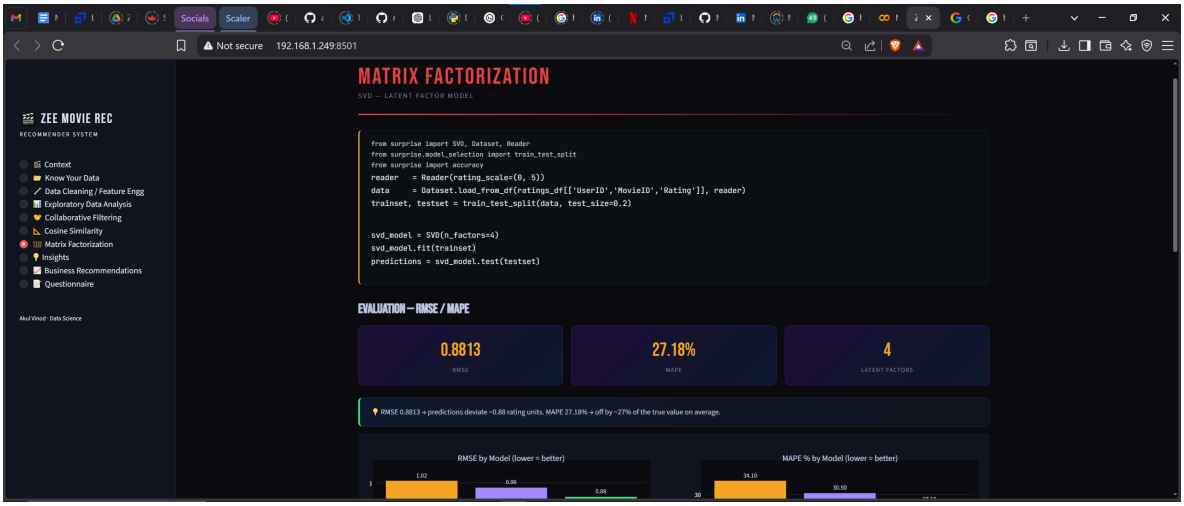

Socials
Scaler
Not secure 192.168.1.249:8501
ZEE MOVIE REC
RECOMMENDER SYSTEM
Context
Know Your Data
Data Cleaning / Feature Engg
Exploratory Data Analysis
Collaborative Filtering
Cosine Similarity
Matrix Factorization
Insights
Business Recommendations
Questionnaire
Akul Vinod · Data Science
Count
'College/grad student' is the dominant occupation, followed by 'other' and 'executive/managerial'.
Top 10 Zip-codes
top_zipcodes = users_df['Zip-code'].value_counts().head(10)
sns.barplot(x=top_zipcodes.index, y=top_zipcodes.values)
Top 10 Zip-codes
Count
Zipcode
48104
22903
55104
11701
55343
94043
60657
55414
02139
10003
Most users from zip-code 48104. Four of the top 10 start with 55 (Minneapolis area).

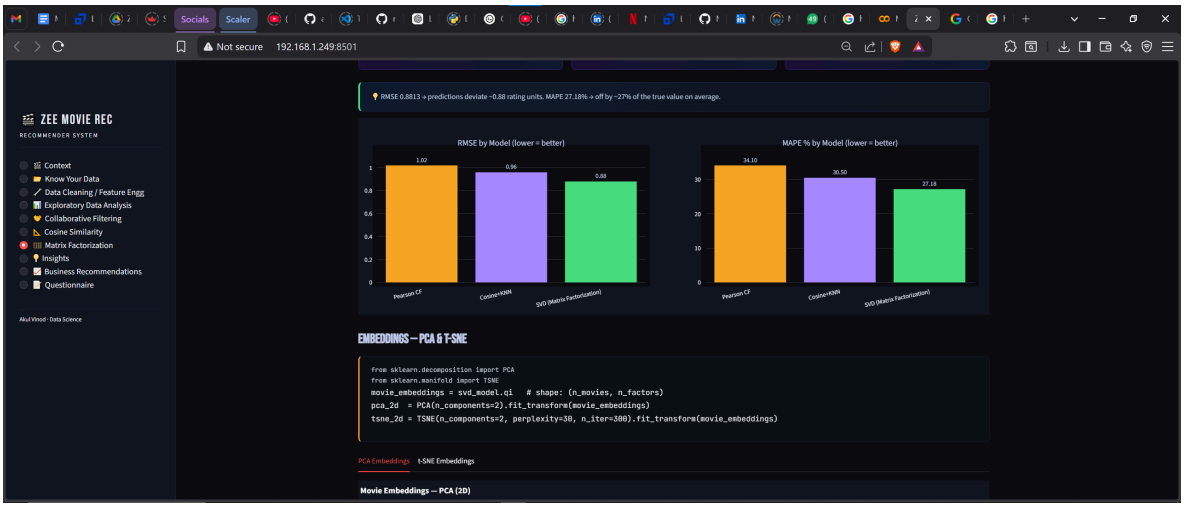

Socials
Scaler
Not secure 192.168.1.249:8501
ZEE MOVIE REC
RECOMMENDER SYSTEM
Context
Know Your Data
Data Cleaning / Feature Engg
Exploratory Data Analysis
Collaborative Filtering
Cosine Similarity
Matrix Factorization
Insights
Business Recommendations
Questionnaire
Akul Vinod · Data Science
COLLABORATIVE FILTERING
PEARSON CORRELATION METHOD
pivot_table = ratings_df.pivot_table(index='MovieID', columns='UserID',
values='Rating', fill_value=0)
movie_id_to_name = movies_df[['MovieID', 'MovieName']].set_index('MovieID')
pivot_table = pivot_table.join(movie_id_to_name, how='left').set_index('MovieName')
user_item_matrix = pivot_table.copy()
HANDLING SPARSITY
total_possible = user_item_matrix.shape[0] * user_item_matrix.shape[1]
non_zero = user_item_matrix[user_item_matrix > 0].count().sum()
sparsity = 1 - (non_zero / total_possible)
print(f"Sparsity: {sparsity*100:.2f}%") # ~95.5%
user_item_matrix = user_item_matrix.loc[(user_item_matrix>0).sum(axis=1) ≥ 100]
user_item_matrix = user_item_matrix.loc[:, (user_item_matrix>0).sum() ≥ 50]
Matrix Sparsity (%)
95.5%

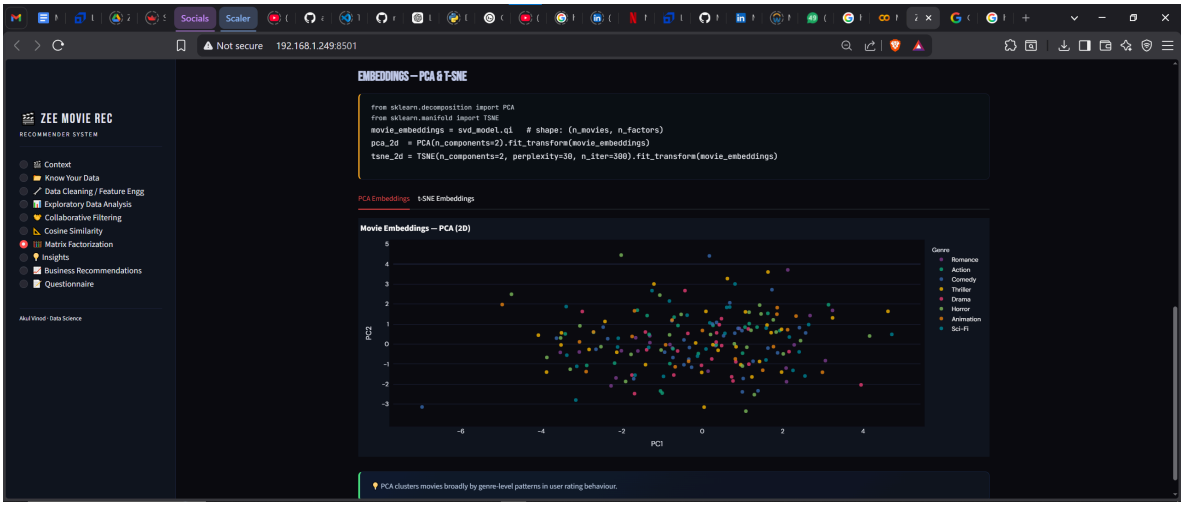

Socials
Scaler
Not secure 192.168.1.249:8501
ZEE MOVIE REC
RECOMMENDER SYSTEM
Context
Know Your Data
Data Cleaning / Feature Engg
Exploratory Data Analysis
Collaborative Filtering
Cosine Similarity
Matrix Factorization
Insights
Business Recommendations
Questionnaire
Akul Vinod · Data Science
Data is ~95.5% sparse. Applying min-rating thresholds reduces noise and improves correlation quality.
CORRELATION MATRIX
normalized_matrix = user_item_matrix.sub(user_item_matrix.mean(axis=1), axis=0)
correlation_matrix = normalized_matrix.T.corr(method='pearson')
Sample Pearson Correlation Matrix (8 movies)
Forrest Gump
Speed
Mrs. Doubtfire
Home Alone
Jurassic Park
The Lion King
Aladdin
Toy Story
RECOMMENDATION FUNCTION
def recommend_movies(movie_name, correlation_matrix, top_n=5):
if movie_name not in correlation_matrix.index:
return f"'{movie_name}' not found."
similar = (correlation_matrix[movie_name]
.drop(movie_name)
.sort_values(ascending=False)

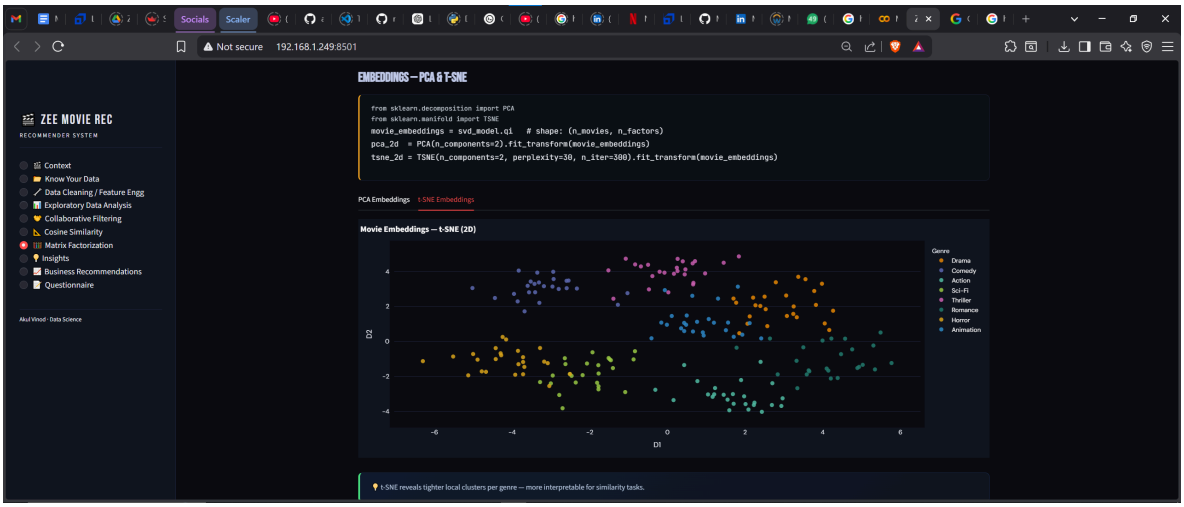

Not secure 192.168.1.249:8501
ZEE MOVIE REC
RECOMMENDER SYSTEM
Context
Know Your Data
Data Cleaning / Feature Engg
Exploratory Data Analysis
Collaborative Filtering
Cosine Similarity
Matrix Factorization
Insights
Business Recommendations
Questionnaire
Akul Vinod · Data Science
COSINE SIMILARITY
KNN-BASED RECOMMENDER
from sklearn.metrics.pairwise import cosine_similarity
from sklearn.neighbors import NearestNeighbors
user_item_dense = user_item_matrix.values
user_similarity = cosine_similarity(user_item_dense.T)
user_similarity_df = pd.DataFrame(user_similarity,
index=user_item_matrix.columns, columns=user_item_matrix.columns)
item_similarity = cosine_similarity(user_item_dense)
item_similarity_df = pd.DataFrame(item_similarity,
index=user_item_matrix.index, columns=user_item_matrix.index)
model_knn = NearestNeighbors(metric='cosine', algorithm='brute')
model_knn.fit(user_item_dense)
Item Cosine Similarity Matrix (Sample)
Mrs. Doubtfire
Home Alone
Pulp Fiction
Terminator
Speed
Lion King
Try It — KNN Recommendations
Movie
Toy Story
#1 Aladdin 0.77
#2 The Lion King 0.76
#3 A Bug's Life 0.91

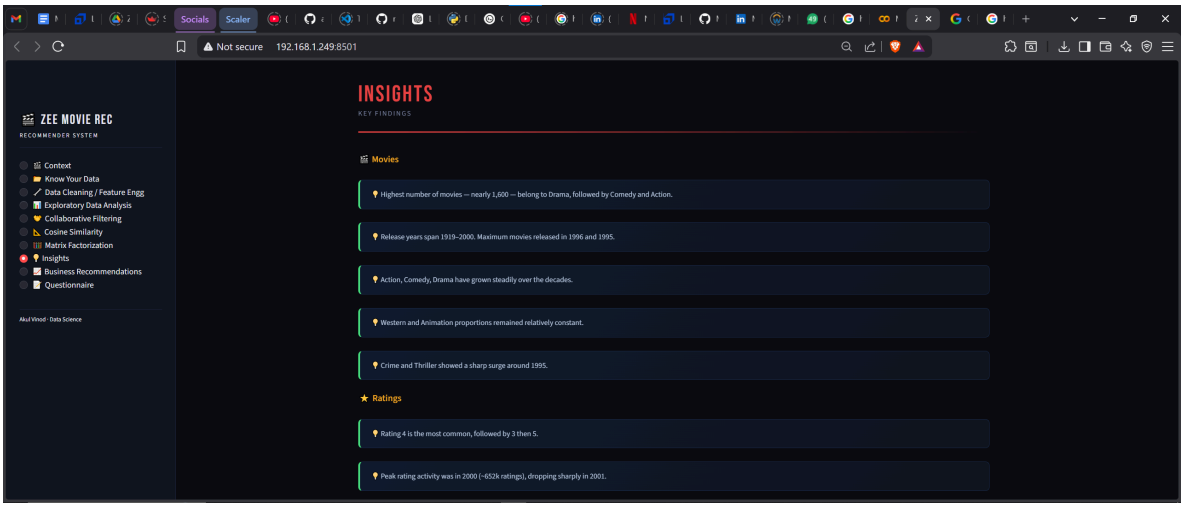

Not secure 192.168.1.249:8501
ZEE MOVIE REC
RECOMMENDER SYSTEM
Akul Vinod · Data Science
item_similarity = cosine_similarity(user_item_dense)
item_similarity_df = pd.DataFrame(item_similarity,
index=user_item_matrix.index, columns=user_item_matrix.index)
model_knn = NearestNeighbors(metric='cosine', algorithm='brute')
model_knn.fit(user_item_dense)
Item Cosine Similarity Matrix (Sample)
Toy Story
Aladdin
Lion King
Speed
Terminator
Pulp Fiction
Home Alone
Mrs. Doubtfire
Try It — KNN Recommendations
Movie
Toy Story
#1 Aladdin 0.77
#2 The Lion King 0.76
#3 A Bug's Life 0.91
#4 Toy Story 2 0.88
#5 Mulan 0.84
def recommend_movies(movie_name, user_item_matrix, item_similarity_df,
model_knn, n_neighbors=5):
movie_idx = user_item_matrix.index.get_loc(movie_name)
movie_vector = user_item_dense[movie_idx].reshape(1, -1)
distances, indices = model_knn.kneighbors(movie_vector, n_neighbors=n_neighbors+1)
return [user_item_matrix.index[idx] for idx in indices.flatten()[1:]]

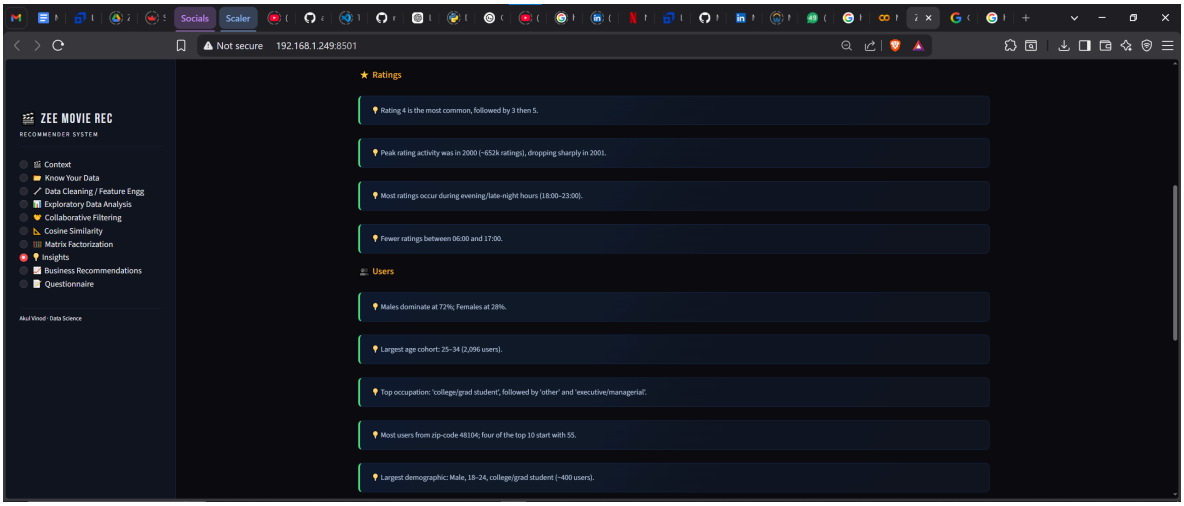

Not secure 192.168.1.249:8501
ZEE MOVIE REC
RECOMMENDER SYSTEM
Akul Vinod · Data Science
MATRIX FACTORIZATION
SVD — LATENT FACTOR MODEL
from surprise import SVD, Dataset, Reader
from surprise.model_selection import train_test_split
from surprise import accuracy
reader = Reader(rating_scale=(0, 5))
data = Dataset.load_from_df(ratings_df[['UserID','MovieID','Rating']], reader)
trainset, testset = train_test_split(data, test_size=0.2)
svd_model = SVD(n_factors=4)
svd_model.fit(trainset)
predictions = svd_model.test(testset)
EVALUATION — RMSE / MAPE
0.8813
RMSE
27.18%
MAPE
4
LATENT FACTORS
RMSE 0.8813 → predictions deviate ~0.88 rating units. MAPE 27.18% → off by ~27% of the true value on average.
RMSE by Model (lower = better)
MAPE % by Model (lower = better)

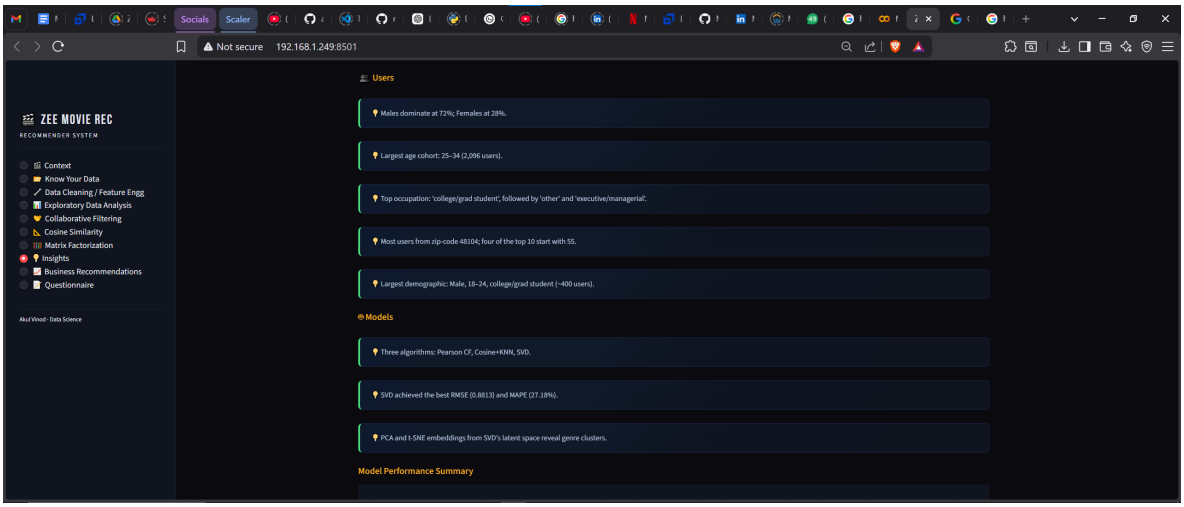

Not secure 192.168.1.249:8501
ZEE MOVIE REC
RECOMMENDER SYSTEM
Context
Know Your Data
Data Cleaning / Feature Engg
Exploratory Data Analysis
Collaborative Filtering
Cosine Similarity
Matrix Factorization
Insights
Business Recommendations
Questionnaire
Akul Vinod · Data Science
RMSE 0.8813 → predictions deviate ~0.88 rating units. MAPE 27.18% → off by ~27% of the true value on average.
RMSE by Model (lower = better)
1.02
0.96
0.88
Pearson CF
Cosine+KNN
SVD (Matrix Factorization)
MAPE % by Model (lower = better)
34.10
30.50
27.18
Pearson CF
Cosine+KNN
SVD (Matrix Factorization)
EMBEDDINGS — PCA & T-SNE
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE
movie_embeddings = svd_model.qi # shape: (n_movies, n_factors)
pca_2d = PCA(n_components=2).fit_transform(movie_embeddings)
tsne_2d = TSNE(n_components=2, perplexity=30, n_iter=300).fit_transform(movie_embeddings)
PCA Embeddings
t-SNE Embeddings
Movie Embeddings — PCA (2D)

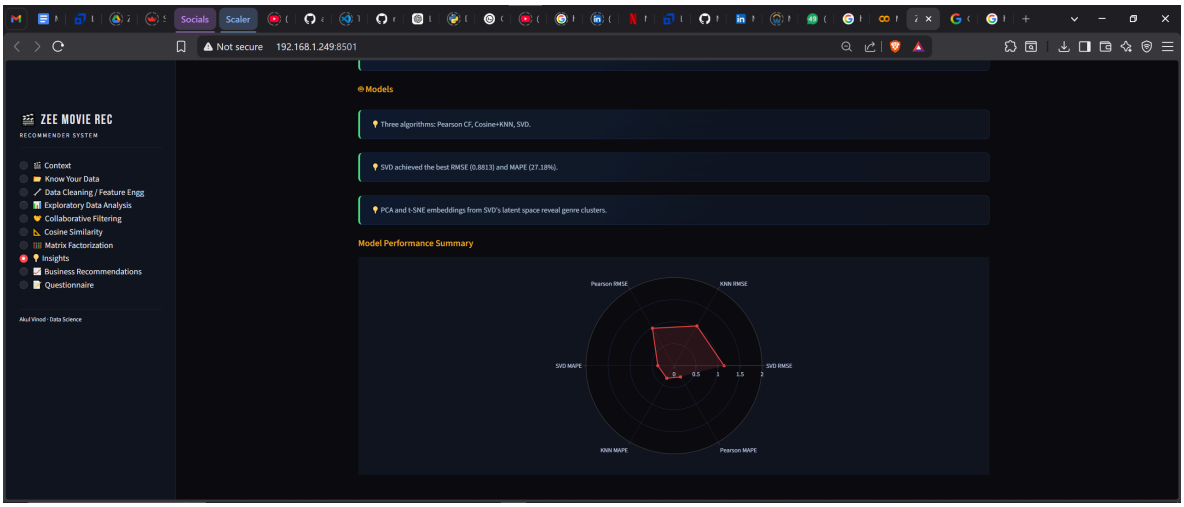

Not secure 192.168.1.249:8501
ZEE MOVIE REC
RECOMMENDER SYSTEM
Context
Know Your Data
Data Cleaning / Feature Engg
Exploratory Data Analysis
Collaborative Filtering
Cosine Similarity
Matrix Factorization
Insights
Business Recommendations
Questionnaire
Akul Vinod · Data Science
EMBEDDINGS — PCA & T-SNE
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE
movie_embeddings = svd_model.qi # shape: (n_movies, n_factors)
pca_2d = PCA(n_components=2).fit_transform(movie_embeddings)
tsne_2d = TSNE(n_components=2, perplexity=30, n_iter=300).fit_transform(movie_embeddings)
PCA Embeddings
t-SNE Embeddings
Movie Embeddings — PCA (2D)
PC1
PC2
Genre
Romance
Action
Comedy
Thriller
Drama
Horror
Animation
Sci-Fi
PCA clusters movies broadly by genre-level patterns in user rating behaviour.

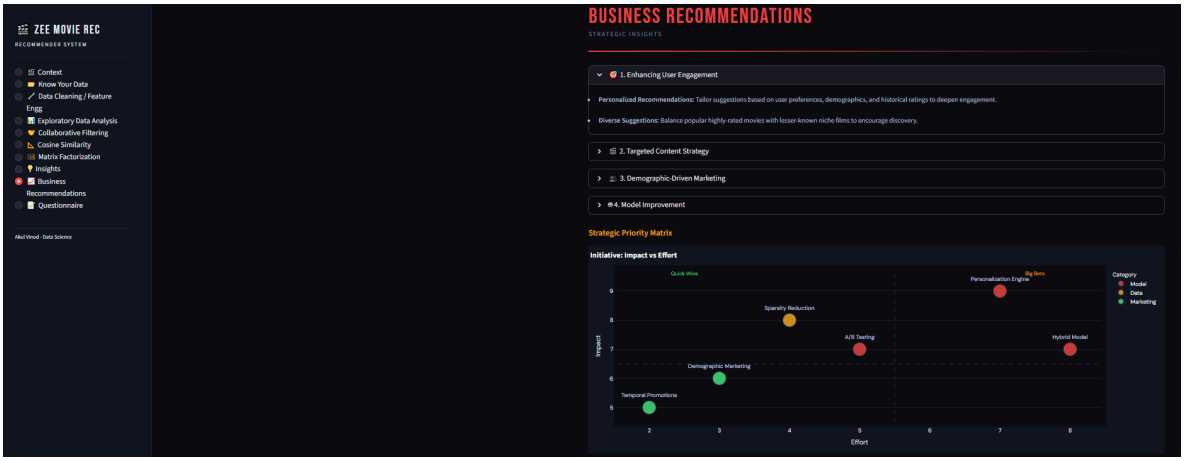

Not secure 192.168.1.249:8501
ZEE MOVIE REC
RECOMMENDER SYSTEM
Context
Know Your Data
Data Cleaning / Feature Engg
Exploratory Data Analysis
Collaborative Filtering
Cosine Similarity
Matrix Factorization
Insights
Business Recommendations
Questionnaire
Akul Vinod · Data Science
EMBEDDINGS — PCA & T-SNE
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE
movie_embeddings = svd_model.qi # shape: (n_movies, n_factors)
pca_2d = PCA(n_components=2).fit_transform(movie_embeddings)
tsne_2d = TSNE(n_components=2, perplexity=30, n_iter=300).fit_transform(movie_embeddings)
PCA Embeddings
t-SNE Embeddings
Movie Embeddings — t-SNE (2D)
D1
D2
Genre
Drama
Comedy
Action
Sci-Fi
Thriller
Romance
Horror
Animation
t-SNE reveals tighter local clusters per genre — more interpretable for similarity tasks.

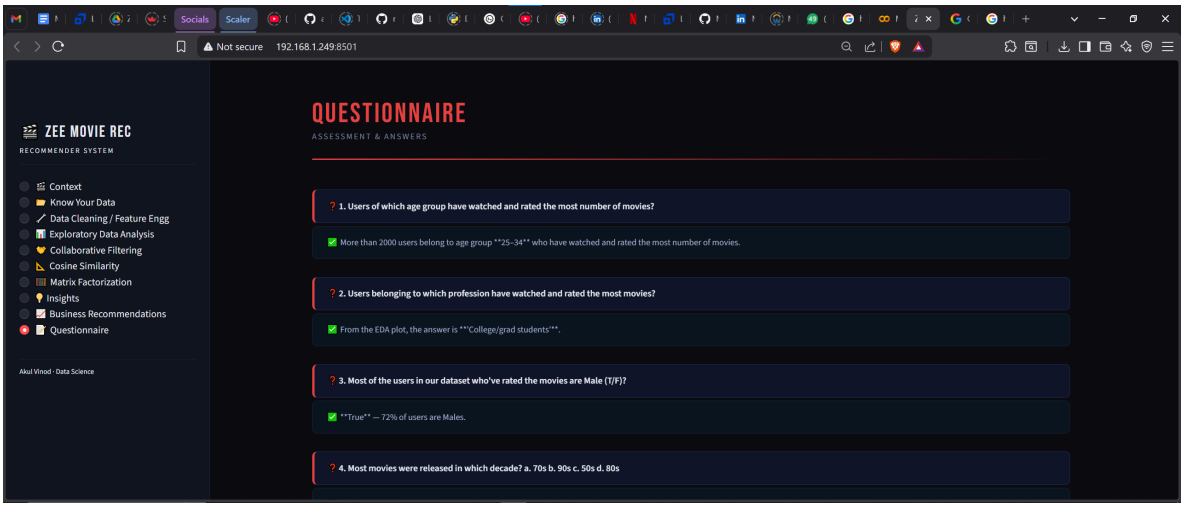

Socials
Scaler
Not secure
192.168.1.249:8501
ZEE MOVIE REC
RECOMMENDER SYSTEM
Context
Know Your Data
Data Cleaning / Feature Engg
Exploratory Data Analysis
Collaborative Filtering
Cosine Similarity
Matrix Factorization
Insights
Business Recommendations
Questionnaire
Akul Vinod · Data Science
INSIGHTS
KEY FINDINGS
Movies
Highest number of movies — nearly 1,600 — belong to Drama, followed by Comedy and Action.
Release years span 1919–2000. Maximum movies released in 1996 and 1995.
Action, Comedy, Drama have grown steadily over the decades.
Western and Animation proportions remained relatively constant.
Crime and Thriller showed a sharp surge around 1995.
Ratings
Rating 4 is the most common, followed by 3 then 5.
Peak rating activity was in 2000 (~652k ratings), dropping sharply in 2001.

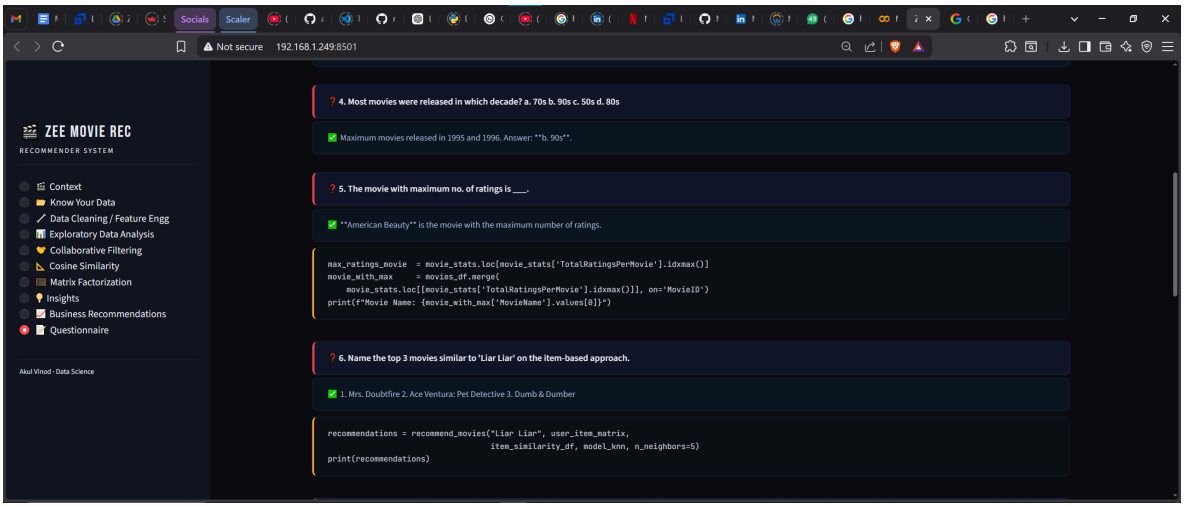

Socials
Scaler
Not secure
192.168.1.249:8501
ZEE MOVIE REC
RECOMMENDER SYSTEM
Context
Know Your Data
Data Cleaning / Feature Engg
Exploratory Data Analysis
Collaborative Filtering
Cosine Similarity
Matrix Factorization
Insights
Business Recommendations
Questionnaire
Akul Vinod · Data Science
Ratings
Rating 4 is the most common, followed by 3 then 5.
Peak rating activity was in 2000 (~652k ratings), dropping sharply in 2001.
Most ratings occur during evening/late-night hours (18:00–23:00).
Fewer ratings between 06:00 and 17:00.
Users
Males dominate at 72%; Females at 28%.
Largest age cohort: 25–34 (2,096 users).
Top occupation: 'college/grad student', followed by 'other' and 'executive/managerial'.
Most users from zip-code 48104; four of the top 10 start with 55.
Largest demographic: Male, 18–24, college/grad student (~400 users).

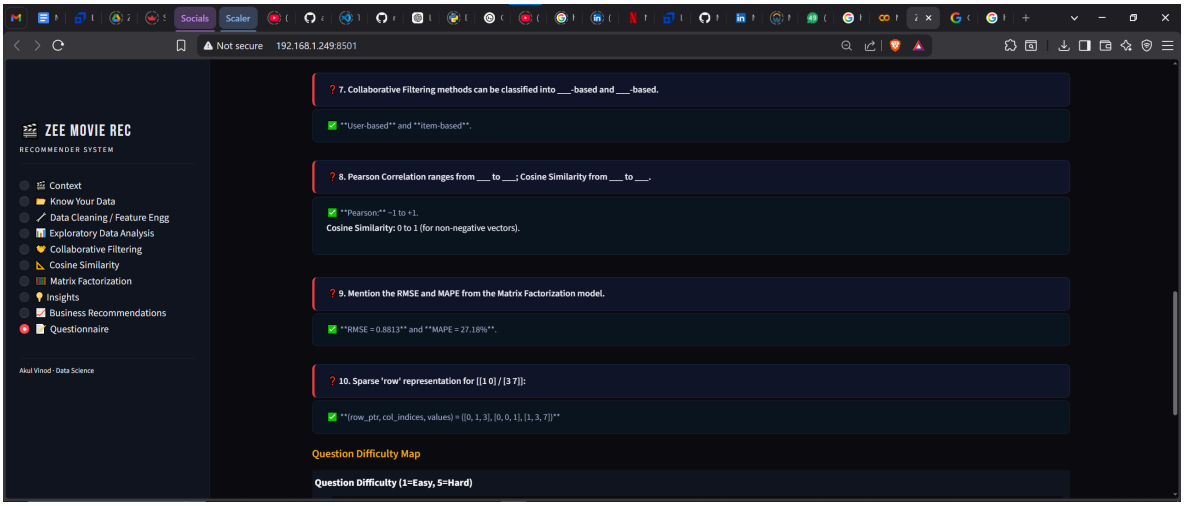

Socials
Scaler
Not secure
192.168.1.249:8501
ZEE MOVIE REC
RECOMMENDER SYSTEM
Context
Know Your Data
Data Cleaning / Feature Engg
Exploratory Data Analysis
Collaborative Filtering
Cosine Similarity
Matrix Factorization
Insights
Business Recommendations
Questionnaire
Akul Vinod · Data Science
Users
Males dominate at 72%; Females at 28%.
Largest age cohort: 25–34 (2,096 users).
Top occupation: 'college/grad student', followed by 'other' and 'executive/managerial'.
Most users from zip-code 48104; four of the top 10 start with 55.
Largest demographic: Male, 18–24, college/grad student (~400 users).
Models
Three algorithms: Pearson CF, Cosine+KNN, SVD.
SVD achieved the best RMSE (0.8813) and MAPE (27.18%).
PCA and t-SNE embeddings from SVD's latent space reveal genre clusters.
Model Performance Summary


Socials
Scaler
Not secure
192.168.1.249:8501
ZEE MOVIE REC
RECOMMENDER SYSTEM
Context
Know Your Data
Data Cleaning / Feature Engg
Exploratory Data Analysis
Collaborative Filtering
Cosine Similarity
Matrix Factorization
Insights
Business Recommendations
Questionnaire
Akul Vinod - Data Science
Models
Three algorithms: Pearson CF, Cosine+KNN, SVD.
SVD achieved the best RMSE (0.8813) and MAPE (27.18%).
PCA and t-SNE embeddings from SVD's latent space reveal genre clusters.
Model Performance Summary
Pearson RMSE
KNN RMSE
SVD MAPE
SVD RMSE
KNN MAPE
Pearson MAPE


ZEE MOVIE REC
RECOMMENDER SYSTEM
Context
Know Your Data
Data Cleaning / Feature Engg
Exploratory Data Analysis
Collaborative Filtering
Cosine Similarity
Matrix Factorization
Insights
Business Recommendations
Questionnaire
Akul Vinod - Data Science
BUSINESS RECOMMENDATIONS
STRATEGIC INSIGHTS
1. Enhancing User Engagement
Personalized Recommendations: Tailor suggestions based on user preferences, demographics, and historical ratings to deepen engagement.
Diverse Suggestions: Balance popular highly-rated movies with lesser-known niche films to encourage discovery.
2. Targeted Content Strategy
3. Demographic-Driven Marketing
4. Model Improvement
Strategic Priority Matrix
Initiative: Impact vs Effort
Quick Wins
Big Bets
Personalization Engine
Sparsity Reduction
A/B Testing
Hybrid Model
Demographic Marketing
Temporal Promotions
Impact
Effort
Category
Model
Data
Marketing


Socials
Scaler
Not secure
192.168.1.249:8501
ZEE MOVIE REC
RECOMMENDER SYSTEM
Context
Know Your Data
Data Cleaning / Feature Engg
Exploratory Data Analysis
Collaborative Filtering
Cosine Similarity
Matrix Factorization
Insights
Business Recommendations
Questionnaire
Akul Vinod - Data Science
QUESTIONNAIRE
ASSESSMENT & ANSWERS
1. Users of which age group have watched and rated the most number of movies?
More than 2000 users belong to age group **25-34** who have watched and rated the most number of movies.
2. Users belonging to which profession have watched and rated the most movies?
From the EDA plot, the answer is **College/grad students**.
3. Most of the users in our dataset who've rated the movies are Male (T/F)?
True — 72% of users are Males.
4. Most movies were released in which decade? a. 70s b. 90s c. 50s d. 80s


Not secure 192.168.1.249:8501
ZEE MOVIE REC
RECOMMENDER SYSTEM
Context
Know Your Data
Data Cleaning / Feature Engg
Exploratory Data Analysis
Collaborative Filtering
Cosine Similarity
Matrix Factorization
Insights
Business Recommendations
Questionnaire
Akul Vinod · Data Science
? 4. Most movies were released in which decade? a. 70s b. 90s c. 50s d. 80s
Maximum movies released in 1995 and 1996. Answer: **b. 90s**.
? 5. The movie with maximum no. of ratings is ___.
American Beauty is the movie with the maximum number of ratings.
max_ratings_movie = movie_stats.loc[movie_stats['TotalRatingsPerMovie'].idxmax()]
movie_with_max = movies_df.merge(
movie_stats.loc[[movie_stats['TotalRatingsPerMovie'].idxmax()]], on='MovieID')
print(f"Movie Name: {movie_with_max['MovieName'].values[0]}")
? 6. Name the top 3 movies similar to 'Liar Liar' on the item-based approach.
1. Mrs. Doubtfire 2. Ace Ventura: Pet Detective 3. Dumb & Dumber
recommendations = recommend_movies("Liar Liar", user_item_matrix,
item_similarity_df, model_knn, n_neighbors=5)
print(recommendations)


Not secure 192.168.1.249:8501
ZEE MOVIE REC
RECOMMENDER SYSTEM
Context
Know Your Data
Data Cleaning / Feature Engg
Exploratory Data Analysis
Collaborative Filtering
Cosine Similarity
Matrix Factorization
Insights
Business Recommendations
Questionnaire
Akul Vinod · Data Science
? 7. Collaborative Filtering methods can be classified into ___-based and ___-based.
User-based and **item-based**.
? 8. Pearson Correlation ranges from ___ to ___; Cosine Similarity from ___ to ___.
Pearson: −1 to +1.
Cosine Similarity: 0 to 1 (for non-negative vectors).
? 9. Mention the RMSE and MAPE from the Matrix Factorization model.
RMSE = 0.8813 and **MAPE = 27.18%**.
? 10. Sparse 'row' representation for [[1 0] / [3 7]]:
(row_ptr, col_indices, values) = ([0, 1, 3], [0, 0, 1], [1, 3, 7])
Question Difficulty Map
Question Difficulty (1=Easy, 5=Hard)


Not secure 192.168.1.249:8501
ZEE MOVIE REC
RECOMMENDER SYSTEM
Context
Know Your Data
Data Cleaning / Feature Engg
Exploratory Data Analysis
Collaborative Filtering
Cosine Similarity
Matrix Factorization
Insights
Business Recommendations
Questionnaire
Akul Vinod · Data Science
RMSE = 0.8813 and **MAPE = 27.18%**.
? 10. Sparse 'row' representation for [[1 0] / [3 7]]:
(row_ptr, col_indices, values) = ([0, 1, 3], [0, 0, 1], [1, 3, 7])
Question Difficulty Map
Question Difficulty (1=Easy, 5=Hard)
Easy
Easy
Easy
Easy
Medium
Hard
Easy
Hard
Medium
Hard
Q1
Q2
Q3
Q4
Q5
Q6
Q7
Q8
Q9
Q10
Akul Vinod · DATA SCIENCE · MOVIE RECOMMENDER SYSTEM

In [ ]: